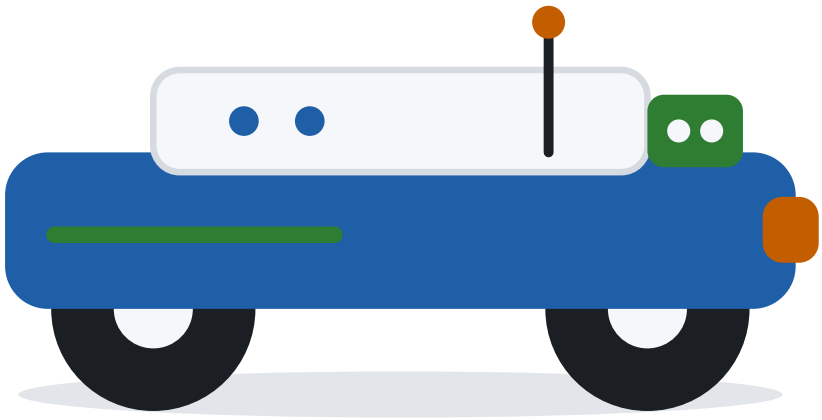
ELEGOO SMART ROBOT CAR V4.0

Programmieren lernen

mit dem Elegoo Smart Robot Car V4.0



Ein Lernbuch für Arduino und Robotik

Für Kinder ab 12

Version 1.1 · Stand Oktober 2026

So liest du dieses Buch

Jedes Kapitel beginnt mit einer kurzen Mission. Danach baust du Schritt für Schritt ein Programm auf. Die Kästen helfen dir beim Lesen: Sie sind immer mit einem Wort beschriftet, damit du sie auch im Schwarz-Weiß-Druck erkennst.

Mission

Die Mission sagt dir, was du am Ende des Kapitels kannst oder gebaut hast.

Deine Aufgabe

Bei einer Aufgabe bekommst du Code mit Lücken. Du ergänzt die fehlenden Zeilen selbst, bevor du weiterliest.

Lösung Die Lösung steht immer auf einer neuen Seite. Vergleiche erst nach dem eigenen Versuch.

Merken

Merken fasst eine Regel zusammen, die du dir dauerhaft einprägen solltest.

Tipp

Ein Tipp spart Zeit. Er zeigt dir einen einfachen Weg oder eine Abkürzung.

Achtung

Achtung warnt vor einem Fehler, der Bauteile oder dein Programm gefährden kann.

Inhalt

Programmieren lernen ... 7
Die Arduino IDE in Kürze ... 7
So ist ein Sketch aufgebaut ... 7
Dein erstes Programm ... 7
Der Serial Monitor ... 9
Variablen ... 10
Entscheidungen ... 11
Schleifen ... 12
Funktionen und Arrays ... 13
Fehler und Merken ... 15
Motoren ... 16
Die erste Fahrt ... 16
Warum setup nie fahren darf ... 18
Die H-Brücke ... 20
Was der Code macht ... 20
Geschwindigkeit in Stufen ... 21
Der Trimm ... 21
Mission: Quadrat fahren ... 21
Fehler und Merken ... 26
Ultraschall ... 28
Wie Schall den Abstand misst ... 28
Der Sensor am Auto ... 28
Abstand messen ... 28
Wenn die Messung danebenliegt ... 31
Mission: Notbremse ... 31
Linie folgen ... 37
Drei Sensoren sehen die Linie ... 37
Rohwerte lesen ... 37
Die Schwelle wählen ... 39
Lenken mit einem Fehlerwert ... 39
Mission: Linie folgen ... 40
Servo ... 46
Ein Motor mit Gedächtnis ... 46
Der Kopf dreht sich ... 46
Mission: Der freie Weg ... 48
MPU6050 ... 57
Zwei Sensoren in einem Chip ... 57
Anschluss und Adresse ... 57
Grad statt Bogenmaß ... 57
Neigung messen ... 58
Zittern und Driften ... 61
Mission: Stoßalarm ... 61
Licht und Fernbedienung ... 67
Drei Farben in einer LED ... 67
Die Fernbedienung auslesen ... 68
Mission: Fahren per Fernbedienung ... 69
Kamera und WLAN ... 78
Ein eigenes WLAN im Auto ... 78

Mit dem Handy verbinden ... 78
Der Schalter Upload und Cam ... 78
Wie Uno und ESP32 reden ... 79
Zwei Gehirne im Auto ... 79
Warum der Uno kein Bild liest ... 80
Gesichtserkennung ausprobieren ... 81
Follow nutzt den Ultraschall ... 81
Mission: Buchstaben statt JSON ... 81
Aufgabe: Schneller und langsamer ... 85
Grossprojekt Rettungsrover ... 93
Fünf Zustände statt vieler if-Blöcke ... 93
Das vollständige Programm ... 95
Sicherheit vor Tempo ... 97
Mission: zwei Funktionen fehlen ... 98
Erweitere selbst ... 104
ESP32 selbst programmieren ... 106
Warum eigene Firmware ... 106
Firmware aufspielen ... 106
Werksfirmware zurückholen ... 106
Die Kamera-Pins ... 107
Der Algorithmus in fünf Schritten ... 107
Aufgabe: Grün statt Rot ... 109
Der Uno empfängt ... 111
Aufgabe: Ruhiger lenken ... 114
Grenzen ... 117
Projekte zum Weiterbauen ... 119
Vier Ideen für dein Auto ... 119
Parkwächter ... 119
Tanzroboter ... 125
Geheimcode ... 130
Kamera-Mission ... 133
Code ordnen und eigene Klassen ... 135
Wenn der Sketch wächst ... 135
Header und Includes ... 135
Eine Klasse ist ein Bauplan ... 136
Mission: Notbremse mit Klasse ... 137
Cheatsheet ... 143
Grundgeruest und Serial Monitor ... 143
Datentypen und Operatoren ... 143
Ablauf: Bedingungen, Schleifen, Funktionen ... 143
Immer wieder gebraucht ... 143
Sensoren und Aktoren ... 144
IR-Codetabelle ... 144
Pin-Tabelle ... 144
Fehlerbilder ... 145
Bibliotheksversionen ... 145
Glossar ... 146
Beispielnummern ... 146
Anhang ... 147
Fuer Eltern und Lehrkraefte ... 147

Wenn nichts geht 147
Teileliste 148
Beispielverzeichnis 149
Änderungen in Version 1.1 149
Quellen und Weiterlesen 150

Bevor du loslegst

Nimm dir kurz Zeit für diese Checkliste. Dann klappt der Start ohne Umwege.

> **Achtung**
>
> **Welche Variante hast du?** Schau auf das Etikett des Kits. Dieses Buch passt zur Variante mit **TB6612** und **MPU6050/GY-521**. Andere Varianten haben eine andere Verdrahtung. Für sie gelten die Original-Dokumente von Elegoo, nicht dieses Buch.

> **Tipp**
>
> Lege Akku und Kabel bereit. Lade den Akku vor dem ersten Fahren auf und prüfe, ob alle Kabel fest sitzen.

- [ ] Ich habe die Variante mit TB6612 und MPU6050/GY-521 erkannt.
- [ ] Der Akku ist geladen und alle Kabel sitzen fest.
- [ ] Arduino IDE 2.x ist installiert.
- [ ] Unter `Werkzeuge > Board` ist **Arduino Uno** gewählt.
- [ ] Unter `Werkzeuge > Port` ist der richtige Port gewählt.
- [ ] Die Bibliotheken sind in den getesteten Versionen installiert.
- [ ] Der Schalter steht auf **Upload** (nicht auf **Cam**).

Pin	Bedeutung
FastLED	3.9.15
IRremote	4.7.1
Adafruit MPU6050	2.2.9
Servo	1.2.2
ESP32-Core	2.0.17 (nur für Kapitel 10)

1 Programmieren lernen

Ziel

Du lädst dein erstes Programm auf den Arduino. Du siehst, wie `setup` und `loop` zusammenarbeiten, und schreibst Zahlen und Texte in den Serial Monitor. Du lernst Variablen, Entscheidungen und Schleifen. Am Ende baust du eine eigene Funktion, die aus drei Messwerten einen Mittelwert rechnet.

Die Arduino IDE in Kürze

Der Arduino ist ein kleiner Computer ohne Bildschirm. Du schreibst das Programm auf dem PC. Ein Klick schiebt es über das USB-Kabel in den Arduino. Die Programmdatei heißt Sketch.

Board wählen. Öffne die Arduino IDE. Unter `Werkzeuge > Board` wählst du `Arduino Uno`.

Port wählen. Stecke das USB-Kabel an. Unter `Werkzeuge > Port` steht ein Eintrag wie `COM3`. Wähle ihn aus.

Hochladen. Drücke den Pfeil nach rechts. Die IDE übersetzt den Sketch und schickt ihn zum Arduino. Nach dem Hochladen startet das Programm sofort.

Serial Monitor. Klicke oben rechts auf das Symbol für den Serial Monitor. Stelle unten `9600 Baud` ein. Jetzt siehst du die Texte, die der Arduino schickt.

Achtung

Auf dem Shield sitzt der Schalter `Upload/Cam`. Für jedes Hochladen steht er auf `Upload`. Nur für die Kamera brauchst du `Cam`. Steht der Schalter falsch, findet die IDE den Arduino nicht.

Mission

Bringe den Arduino dazu, im Serial Monitor zu sprechen. Danach rechnest du mit Variablen und baust eine Funktion für den Mittelwert aus drei Zahlen.

So ist ein Sketch aufgebaut

Ein Sketch besteht aus Zeilen. Vier Regeln helfen dir beim Lesen.

Kommentare. Zwei Schrägstriche `//` beginnen einen Kommentar. Der Arduino ignoriert alles bis zum Zeilenende. Kommentare erklären, was der Code tut.

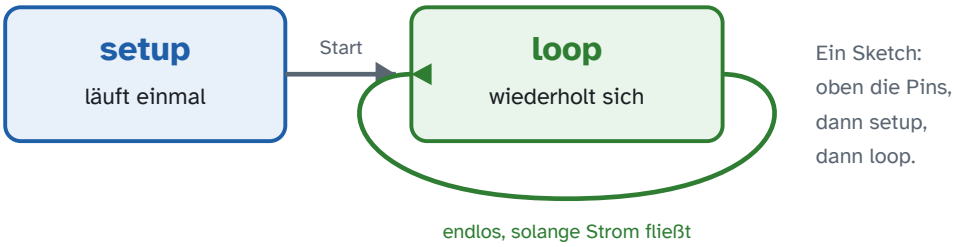
Anweisungen. Jede Anweisung endet mit einem Semikolon. Fehlt es, meldet die IDE einen Fehler.

Blöcke. Geschweifte Klammern `{` und `}` fassen mehrere Zeilen zusammen. Eine Funktion beginnt mit ihrem Namen und einem Block.

Funktionen. `setup` und `loop` sind Funktionen. Du erkennst sie an den runden Klammern. Was darin steht, sind Parameter.

Dein erstes Programm

Jeder Sketch hat zwei feste Teile. `setup` läuft genau einmal, direkt nach dem Einschalten oder nach dem Hochladen. `loop` heißt Schleife und wiederholt sich endlos, solange der Arduino Strom hat.



Ein Sketch:
oben die Pins,
dann setup,
dann loop.

Hallo Arduino 001_hello_world_solution.ino

```
1  // Kapitel 1: Hallo Arduino, Zaehler im Serial Monitor
2  const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
3  const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
4  const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
5
6  int counter = 0;  // zaehlt die Runden in loop()
7
8  void stopMotors() {
9    analogWrite(PIN_PWMA, 0);
10   analogWrite(PIN_PWMB, 0);
11   digitalWrite(PIN_STBY, LOW);
12 }
13
14 void setup() {
15   pinMode(PIN_PWMA, OUTPUT);
16   pinMode(PIN_PWMB, OUTPUT);
17   pinMode(PIN_STBY, OUTPUT);
18   stopMotors();  // Motoren bleiben aus
19
20   Serial.begin(9600);
21   Serial.println(F("Hallo! Ich bin der Arduino."));
22 }
23
24 void loop() {
25   counter = counter + 1;
26   Serial.print(F("Runde "));
27   Serial.println(counter);
28   delay(500);
29 }
```

Zeile 2 bis 4 legen fest, an welchen Pins die Motoren hängen. Zeile 6 legt den Zähler an. Der Zähler startet bei `0`.

Zeile 8 bis 12 schalten die Motoren ab. Diese Funktion rufen wir in `setup` auf, damit das Auto beim Üben sicher stehen bleibt. Die Motoren sind hier nur vorbereitet, gefahren wird in Kapitel 2.

Zeile 14 bis 22 sind `setup`. Die drei `pinMode`-Zeilen machen aus den Pins Ausgänge. `Serial.begin(9600)` öffnet den Serial Monitor mit 9600 Baud. Die nächste Zeile schickt einen Text.

Zeile 24 bis 29 sind `loop`. Der Arduino zählt eins hoch, schreibt `Runde` und die Zahl, dann wartet er 500 Millisekunden. Danach beginnt `loop` von vorn. Im Monitor läuft die Zahl deshalb immer weiter.

> **Probiere**
>
> Ändere `delay(500)` in `delay(2000)`. Lade neu hoch. Jetzt kommt nur noch alle zwei Sekunden eine neue Runde.

Der Serial Monitor

Der Monitor zeigt, was der Arduino sendet. `Serial.begin(9600)` legt die Geschwindigkeit fest. 9600 Baud bedeutet 9600 Bit pro Sekunde. Bei der üblichen Einstellung 8N1 sind das höchstens etwa 960 Zeichen pro Sekunde. Unten im Monitor muss dieselbe Zahl stehen, sonst erscheinen seltsame Zeichen.

`Serial.print` schreibt ohne Zeilenumbruch. `Serial.println` beginnt danach eine neue Zeile. Deshalb stehen `Runde 1`, `Runde 2` und `Runde 3` untereinander. So sieht die Ausgabe aus:

```
Hallo! Ich bin der Arduino.
Runde 1
Runde 2
Runde 3
```

Der Arduino sendet nur, solange er Strom hat. Ziehe das USB-Kabel ab, dann bleibt der Monitor stehen. Nach dem nächsten Hochladen beginnt die Ausgabe wieder von vorn.

> **Tipp**
>
> Unten im Monitor gibt es den Schalter `Autoscroll`. Ist er an, springt die Anzeige automatisch nach unten. Bei langen Ausgaben schaltest du ihn aus und liest in Ruhe.

Variablen

Eine Variable ist ein beschrifteter Platz im Speicher. Sie hat einen Typ, einen Namen und einen Wert. Der Typ sagt, welche Art von Wert hineinpasst.

Vier Typen 006_variables_solution.ino

```
1  // Kapitel 1: Variablen und eine kleine Rechnung
2  const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
3  const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
4  const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
5  int distanceLeft = 30;   // cm, linker Messwert
6  int distanceRight = 42;  // cm, rechter Messwert
7  void stopMotors() {
8    analogWrite(PIN_PWMA, 0);
9    analogWrite(PIN_PWMB, 0);
10   digitalWrite(PIN_STBY, LOW);
11 }
12 void setup() {
13   pinMode(PIN_PWMA, OUTPUT);
14   pinMode(PIN_PWMB, OUTPUT);
15   pinMode(PIN_STBY, OUTPUT);
16   stopMotors();
17   Serial.begin(9600);
18   float average = (distanceLeft + distanceRight) / 2.0;
19   bool freeWay = average > 20;
20   char sensor = 'A';
21   Serial.print(F("Sensor "));
22   Serial.print(sensor);
23   Serial.print(F(": "));
24   Serial.print(average);
25   Serial.println(F(" cm"));
26   Serial.print(F("Freie Fahrt? "));
27   Serial.println(freeWay ? F("ja") : F("nein"));
28 }
29 void loop() {
30 }
```

Zeile 5 und 6 legen zwei ganze Zahlen an. `int` steht für eine ganze Zahl ohne Komma, zum Beispiel 30 oder 42. Die Kommentare nennen die Einheit Zentimeter.

Zeile 18 rechnet den Mittelwert. `2.0` enthält einen Punkt. Dadurch rechnet der Arduino mit Kommazahlen. Bei `2` ohne Punkt würde er abrunden.

Der Typ entscheidet über das Ergebnis. `30 / 2` ergibt 15. `31 / 2` ergibt ebenfalls 15, weil beide Zahlen `int` sind. Der Rest fällt weg. Mit `31 / 2.0` kommt 15.5 heraus. Eine Kommazahl im Ausdruck genügt.

Zeile 19 prüft, ob der Mittelwert größer als 20 ist. Das Ergebnis ist `true` oder `false`. Eine `bool`-Variable merkt sich genau diesen Wahrheitswert.

Zeile 20 legt einen einzelnen Buchstaben an. `char` speichert ein Zeichen und steht in einfachen Anführungszeichen. Zeile 27 gibt `ja` oder `nein` aus. Das Fragezeichen wählt den passenden Text aus.

Pin	Bedeutung
`int`	ganze Zahl, zum Beispiel 30
`float`	Kommazahl, zum Beispiel 36.0
`bool`	`true` oder `false`
`char`	ein einzelnes Zeichen, zum Beispiel `A`

Entscheidungen

Ein Programm trifft Entscheidungen mit `if`. Der Ausdruck in der Klammer wird geprüft. Ist er wahr, läuft der Block darunter. Sonst prüft `else if` die nächste Bedingung. `else` fängt alle übrigen Fälle auf.

Zu nah, passt, weit weg `011_decisions_solution.ino`

```
1  // Kapitel 1: Entscheidungen mit if und else
2  const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
3  const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
4  const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
5  const int distance = 25;  // cm, angenommener Messwert
6  const int NEAR_CM = 20;   // naeher als das ist zu nah
7  const int FAR_CM = 60;    // weiter als das ist weit weg
8  void stopMotors() {
9    analogWrite(PIN_PWMA, 0);
10   analogWrite(PIN_PWMB, 0);
11   digitalWrite(PIN_STBY, LOW);
12 }
13 void setup() {
14   pinMode(PIN_PWMA, OUTPUT);
15   pinMode(PIN_PWMB, OUTPUT);
16   pinMode(PIN_STBY, OUTPUT);
17   stopMotors();
18   Serial.begin(9600);
19   if (distance < NEAR_CM) {
20     Serial.println(F("Zu nah!"));
21   } else if (distance <= FAR_CM) {
22     Serial.println(F("Passt."));
23   } else {
24     Serial.println(F("Weit weg."));
25   }
26 }
27 void loop() {
28 }
```

Zeile 5 bis 7 geben drei Werte vor. `distance` ist der Messwert. `NEAR_CM` und `FAR_CM` sind die beiden Schwellen. Große Namen in Großbuchstaben zeigen an, dass sich der Wert nicht ändert.
Zeile 19 fragt: Ist `distance` kleiner als 20? Dann ist das Hindernis zu nah. Zeile 21 fragt sonst: Ist `distance` kleiner oder gleich 60? Dann passt der Abstand. Alles darüber landet im `else`.
Diese Zeichen vergleichen zwei Werte:

Zeichen	Bedeutung
==	gleich
!=	ungleich
<	kleiner
>	größer
<=	kleiner oder gleich
>=	größer oder gleich

Zwei Gleichheitszeichen prüfen. Ein einzelnes = weist einen Wert zu. Das ist der häufigste Tippfehler am Anfang.

Schleifen

Eine Schleife wiederholt Befehle. Die `for`-Schleife zählt eine Variable hoch oder herunter. Die `while`-Schleife läuft, solange ihre Bedingung wahr ist.

Zählen mit for und while `016_loops_solution.ino`

```
1  // Kapitel 1: for und while
2  const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
3  const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
4  const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
5  void stopMotors() {
6    analogWrite(PIN_PWMA, 0);
7    analogWrite(PIN_PWMB, 0);
8    digitalWrite(PIN_STBY, LOW);
9  }
10 void setup() {
11   pinMode(PIN_PWMA, OUTPUT);
12   pinMode(PIN_PWMB, OUTPUT);
13   pinMode(PIN_STBY, OUTPUT);
14   stopMotors();
15   Serial.begin(9600);
16   Serial.println(F("for-Schleife:"));
17   for (int i = 3; i >= 1; i--) {
18     Serial.print(i);
19     Serial.println(F(" ..."));
20   }
21   Serial.println(F("while-Schleife:"));
22   int battery = 3;
23   while (battery > 0) {
24     Serial.println(battery);
25     battery = battery - 1;
26   }
27   Serial.println(F("Akku leer."));
28 }
29 void loop() {
30 }
```

Zeile 17 bis 20 sind die `for`-Schleife. `int i = 3` setzt den Start. `i >= 1` ist die Bedingung. `i--` zieht nach jeder Runde eins ab. Die Schleife schreibt also 3, 2 und 1.

Zeile 22 legt den Akkustand auf 3. Zeile 23 prüft vor jeder Runde, ob noch etwas übrig ist. Zeile 25 zieht eins ab. Bei 0 endet die Schleife.

> **Merken**
>
> `setup` läuft einmal. `loop` läuft immer weiter. Eine `for`-Schleife zählt eine feste Anzahl von Runden. Eine `while`-Schleife hängt von einer Bedingung ab.

Funktionen und Arrays

Eine Funktion fasst Befehle zusammen und bekommt einen Namen. Du rufst sie mit ihrem Namen auf. Ein Array ist eine Reihe von Werten unter einem Namen. Die Werte haben Plätze von 0 an gezählt.

In der Aufgabe liegen drei Messwerte in einem Array. Du schreibst die Funktion `average`. Sie nimmt drei ganze Zahlen, addiert sie und teilt die Summe durch `3.0`. Das Ergebnis ist eine Kommazahl.

Das Array `values` hat drei Plätze. `values[0]` ist 24, `values[1]` ist 30, `values[2]` ist 27. Die Summe ist 81. Geteilt durch 3.0 ergibt 27.0. Diesen Wert gibt die Funktion mit `return` zurück.

Deine Aufgabe

Ergänze die Funktion `average`. Bilde die Summe aus `a`, `b` und `c` und teile sie durch `3.0`. Rufe die Funktion danach mit den drei Array-Werten auf und speichere das Ergebnis in `mean`.

Dein Code 021_average_exercise.ino

```
// Kapitel 1: Aufgabe Mittelwert
const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
int values[3] = {24, 30, 27};  // drei Messwerte in cm
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
// TODO: Ergänze die Funktion average().
float average(int a, int b, int c) {
  // TODO: Summe bilden, durch 3.0 teilen, Ergebnis zurueckgeben.
  return 0;
}
void setup() {
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  stopMotors();
  Serial.begin(9600);
  // TODO: average() mit den drei Array-Werten aufrufen.
  float mean = 0;
  Serial.print(F("Mittelwert: "));
  Serial.print(mean);
  Serial.println(F(" cm"));
}
void loop() {
}
```

Ergänze die fehlenden Zeilen.

Lösung Mittelwert

Mittelwert 021_average_solution.ino

```
1  // Kapitel 1: Loesung Mittelwert
2  const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
3  const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
4  const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
5  int values[3] = {24, 30, 27};  // drei Messwerte in cm
6  void stopMotors() {
7    analogWrite(PIN_PWMA, 0);
8    analogWrite(PIN_PWMB, 0);
9    digitalWrite(PIN_STBY, LOW);
10 }
11 // Nimmt drei ganze Zahlen, gibt eine Kommazahl zurueck.
12 float average(int a, int b, int c) {
13   int sum = a + b + c;
14   return sum / 3.0;  // 3.0 erzwingt Kommazahlen
15 }
16 void setup() {
17   pinMode(PIN_PWMA, OUTPUT);
18   pinMode(PIN_PWMB, OUTPUT);
19   pinMode(PIN_STBY, OUTPUT);
20   stopMotors();
21   Serial.begin(9600);
22   float mean = average(values[0], values[1], values[2]);
23   Serial.print(F("Mittelwert: "));
24   Serial.print(mean);
25   Serial.println(F(" cm"));
26 }
27 void loop() {
28 }
```

Vergleiche deine Lösung mit dem Listing. Die Summe entsteht in Zeile 13, geteilt wird in Zeile 14 durch `3.0`. Zeile 22 ruft die Funktion mit den drei Array-Plätzen auf.

Fehler und Merken

Häufiger Fehler

So sieht es aus	Das hilft
Der Monitor bleibt leer	Prüfe `Serial.begin(9600)` und stelle unten im Monitor `9600 Baud` ein.
`Runde` erscheint ohne Zahl	Nutze `Serial.println(counter)` statt `Serial.print`.
Die Zahl steigt nicht	Der Zähler muss in `loop` stehen, in `setup` läuft er nur einmal.
`average` liefert `0`	Gib `sum / 3.0` zurück. Ein `return 0` bleibt bei `0`.

> **Merken**
>
> Jede Anweisung endet mit einem Semikolon. `setup` läuft einmal, `loop` wiederholt sich. `int` speichert ganze Zahlen, `float` Kommazahlen. Ein Array zählt seine Plätze ab `0`.

- [ ] Ich kann einen Sketch hochladen und den Port wählen
- [ ] Ich sehe Texte im Serial Monitor bei 9600 Baud
- [ ] Ich kenne den Unterschied zwischen `setup` und `loop`
- [ ] Ich habe eine Funktion mit Rückgabewert geschrieben

2 Motoren

Ziel

Du verstehst, wie der Motortreiber die Drehrichtung umschaltet und wie du die Geschwindigkeit in 255 Stufen einstellst. Du baust ein Start-Gate mit Taste und Countdown. Danach fährt dein Auto vorwärts, rückwärts und im Quadrat. Zum Schluss stellst du den Geradeauslauf mit dem Trimm ein.

Die erste Fahrt

Zwei Programme stehen am Anfang. Das erste fährt vier Manöver. Das zweite erhöht die Geschwindigkeit in Stufen. Lies zuerst den Code, danach die Erklärung. Beide Programme starten erst nach einem Tastendruck. Abbildung 2.1 zeigt diesen Start als Plan.

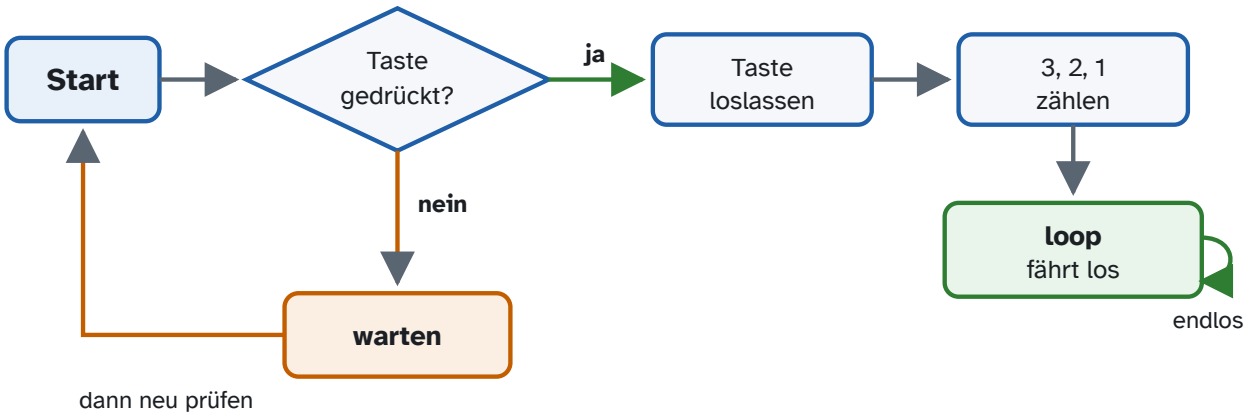


Abbildung 2.1: Das Start-Gate. Erst Tastendruck, Loslassen und der Countdown geben die Fahrt frei.

Vor, zurück und drehen 026_drive_test_solution.ino

```
// Kapitel 2: Erste Fahrt nach dem Start-Gate
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const int TEST_SPEED = 140;  // 140 von 255
const int LEFT_TRIM = 0;     // hier spaeter korrigieren
const int RIGHT_TRIM = 0;    // hier spaeter korrigieren
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void driveForward(int speed) {
  drive(speed + LEFT_TRIM, speed + RIGHT_TRIM);
}
void driveBackward(int speed) {
  drive(-speed - LEFT_TRIM, -speed - RIGHT_TRIM);
}
```

Fortsetzung 026_drive_test_solution.ino

```
53  void setup() {
54    pinMode(PIN_PWMA, OUTPUT);
55    pinMode(PIN_PWMB, OUTPUT);
56    pinMode(PIN_AIN1, OUTPUT);
57    pinMode(PIN_BIN1, OUTPUT);
58    pinMode(PIN_STBY, OUTPUT);
59    pinMode(PIN_BUTTON, INPUT_PULLUP);
60    stopMotors();
61    Serial.begin(9600);
62  }
63  void loop() {
64    waitForStart();
65    Serial.println(F("Vorwaerts"));
66    driveForward(TEST_SPEED);
67    delay(1000);
68    stopMotors();
69    delay(500);
70    Serial.println(F("Rueckwaerts"));
71    driveBackward(TEST_SPEED);
72    delay(1000);
73    stopMotors();
74    delay(500);
75    Serial.println(F("Links drehen"));
76    drive(-TEST_SPEED, TEST_SPEED);
77    delay(600);
78    stopMotors();
79    delay(500);
80    Serial.println(F("Rechts drehen"));
81    drive(TEST_SPEED, -TEST_SPEED);
82    delay(600);
83    stopMotors();
84    Serial.println(F("Fertig. Taste druecken."));
85  }
```

Warum setup nie fahren darf

Beim Hochladen hängt das Auto am USB-Kabel. Der Arduino startet direkt nach dem Flashen, noch bevor du das Kabel weglegen kannst. Stünde ein Fahrbefehl in `setup`, führe das Auto sofort los. Es zöge am Kabel oder über die Tischkante.

Deshalb steht jeder Fahrbefehl in `loop` hinter einem Start-Gate. Du drückst die Taste auf dem Shield und lässt sie wieder los. Dann zählt das Programm 3, 2, 1. In dieser Zeit legst du das Kabel zur Seite und setzt das Auto hin. Erst nach dem Countdown fährt es. Abbildung 2.1 zeigt das Gate als Plan: Solange die Taste nicht gedrückt ist, läuft das Programm im Kreis und wartet.

> **Mission**
>
> Baue ein Auto, das erst nach Tastendruck und Countdown fährt. Teste vorwärts, rückwärts und beide Drehungen. Fahre danach ein Quadrat mit eigenen Fahrfunktionen.

Rampe von 0 bis 255 031_speed_ramp_solution.ino

```ino
// Kapitel 2: Geschwindigkeit in Stufen erhoehen
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield

const int RAMP_STEP = 51;      // 0, 51, 102, 153, 204, 255
const int STEP_DELAY_MS = 400;  // Zeit zwischen Schritten
const int LEFT_TRIM = 0;  // Korrektur linker Motor
const int RIGHT_TRIM = 0;  // Korrektur rechter Motor

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
```

Fortsetzung 031_speed_ramp_solution.ino

```
53
54  void driveForward(int speed) {
55    drive(speed + LEFT_TRIM, speed + RIGHT_TRIM);
56  }
57
58  void setup() {
59    pinMode(PIN_PWMA, OUTPUT);
60    pinMode(PIN_PWMB, OUTPUT);
61    pinMode(PIN_AIN1, OUTPUT);
62    pinMode(PIN_BIN1, OUTPUT);
63    pinMode(PIN_STBY, OUTPUT);
64    pinMode(PIN_BUTTON, INPUT_PULLUP);
65    stopMotors();
66    Serial.begin(9600);
67  }
68
69  void loop() {
70    waitForStart();
71    Serial.println(F("Rampe vorwaerts"));
72    for (int pwm = 0; pwm ⩽ 255; pwm += RAMP_STEP) {
73      driveForward(pwm);
74      Serial.print(F("PWM "));
75      Serial.println(pwm);
76      delay(STEP_DELAY_MS);
77    }
78    stopMotors();
79    Serial.println(F("Aus."));
80  }
```

Die H-Brücke

Der Motortreiber enthält pro Seite eine H-Brücke. Sie polt die Spannung um. Ein Richtungspin pro Seite gibt die Richtung vor. Der PWM-Pin gibt die Geschwindigkeit vor. `analogWrite` schaltet schnell ein und aus: `0` ist aus, 255 ist volle Kraft, `120` ist knapp die Hälfte.

Was der Code macht

Zeile 11 bis 15 stoppen beide Motoren. `STBY` auf `LOW` trennt den Treiber. Zeile 16 bis 24 sind `drive`: zwei Geschwindigkeiten, `constrain` auf –255 bis 255, positiv vorwärts, negativ rückwärts, `abs` für `analogWrite`. Abbildung 2.2 trennt die eine Zahl in zwei Aufgaben. Das Vorzeichen wählt die Drehrichtung, der Betrag ohne Vorzeichen bestimmt die Geschwindigkeit.

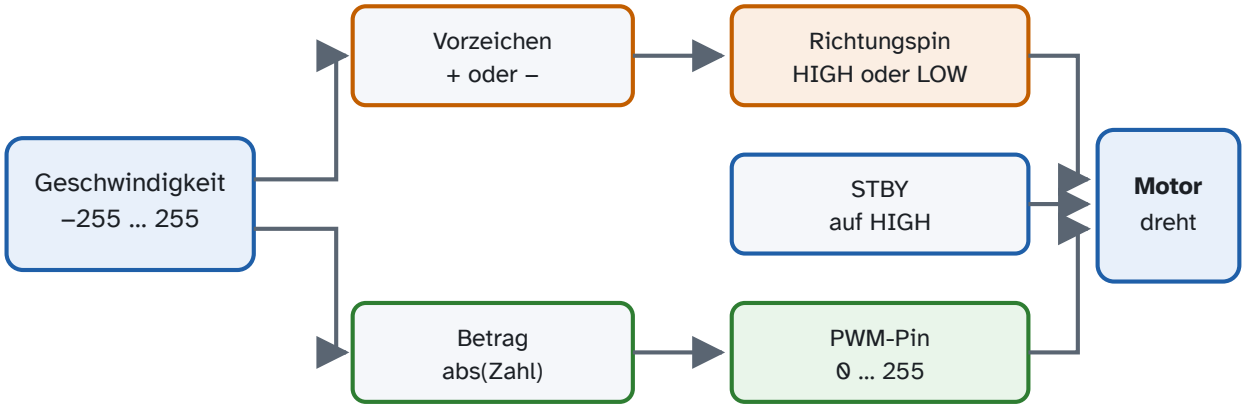


Abbildung 2.2: `drive` macht aus einer Zahl zwischen –255 und 255 Richtung und Geschwindigkeit.

Zeile 25 bis 31 lesen die Taste. `INPUT_PULLUP` hält sie auf `HIGH`, gedrückt liefert `LOW`. Zeile 32 bis 46 sind das Gate: warten auf den Druck, warten aufs Loslassen, dann 3, 2, 1.

Zeile 63 bis 85 fahren. Vorwärts und rückwärts dauern eine Sekunde. Für die Drehung laufen die Räder gegenläufig.

Achtung

Stelle das Auto so auf eine Kiste, dass beide Räder frei drehen und das Auto nicht vom Tisch fallen kann. Lege das USB-Kabel locker, sodass es nirgends zieht. Halte eine Hand am Ein-/Aus-Schalter. Der Countdown hilft beim Timing, ist aber keine Garantie: Prüfe vor dem Start, dass das Auto sicher steht und das Kabel frei liegt.

Geschwindigkeit in Stufen

`analogWrite` kennt 0 bis 255. Kleine Werte drehen die Achse oft nicht. Zeile 9 legt die Stufe auf 51: 0, 51, 102, 153, 204, 255. Zeile 72 bis 77 fährt jede Stufe an. Beobachte, ab welcher Zahl die Räder drehen.

Probiere

Ändere `RAMP_STEP` auf 17. Jetzt gibt es 16 Stufen. Ändere danach `STEP_DELAY_MS` auf `200`, dann läuft der Test schneller.

Der Trimm

Zwei Motoren laufen nie gleich, das Auto zieht zur Seite. `LEFT_TRIM` und `RIGHT_TRIM` in Zeile 9 und 10 gleichen das aus. Zieht es nach links, erhöhe `RIGHT_TRIM`; zieht es nach rechts, erhöhe `LEFT_TRIM`. Werte von –20 bis 20.

Tipp

Fährt das Auto rückwärts, obwohl `driveForward` aufgerufen wird, tausche in `drive` die beiden Richtungspins: `PIN_BIN1` und `PIN_AIN1` in Zeile 19 und 20. Dann stimmt die Richtung für dein Shield.

Mission: Quadrat fahren

Ein Quadrat hat vier gleiche Seiten und vier Drehungen um 90 Grad. In der Aufgabe fehlen die Funktionen `driveOneSide` und `driveSquare`.

Deine Aufgabe

Fülle `driveOneSide`: vorwärts fahren, `SIDE_MS` warten, anhalten. Fülle `driveSquare`: eine Seite fahren, links drehen, `TURN_MS` warten, anhalten. Wiederhole das vier Mal.

Dein Code 036_square_exercise.ino

```
// Kapitel 2: Aufgabe Quadrat fahren
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield

const int DRIVE_SPEED = 130;   // 130 von 255
const int TURN_SPEED = 120;    // 120 von 255
const int SIDE_MS = 1200;      // Zeit fuer eine Seite
const int TURN_MS = 620;       // Zeit fuer 90 Grad
const int LEFT_TRIM = 0;   // Korrektur linker Motor
const int RIGHT_TRIM = 0;   // Korrektur rechter Motor

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

void driveForward(int speed) {
  drive(speed + LEFT_TRIM, speed + RIGHT_TRIM);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
```

Fortsetzung 036_square_exercise.ino

```
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

// TODO: Fahre eine Seite, warte SIDE_MS, halte an.
void driveOneSide() {
}

// TODO: Wiederhole vier Mal: Seite fahren, links drehen.
void driveSquare() {
  for (int side = 0; side < 4; side++) {
  }
}

void setup() {
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  Serial.begin(9600);
}

void loop() {
  waitForStart();
  Serial.println(F("Quadrat startet"));
  driveSquare();
  Serial.println(F("Fertig"));
}
```

Ergänze die fehlenden Zeilen.

Lösung Quadrat

Quadrat 036_square_solution.ino

```
// Kapitel 2: Loesung Quadrat fahren
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const int DRIVE_SPEED = 130;  // 130 von 255
const int TURN_SPEED = 120;   // 120 von 255
const int SIDE_MS = 1200;     // Zeit fuer eine Seite
const int TURN_MS = 620;      // Zeit fuer 90 Grad
const int LEFT_TRIM = 0;  // Korrektur linker Motor
const int RIGHT_TRIM = 0;  // Korrektur rechter Motor
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
void driveForward(int speed) {
  drive(speed + LEFT_TRIM, speed + RIGHT_TRIM);
}
void turnLeft(int speed) {
  drive(-speed, speed);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
```

Fortsetzung 036_square_solution.ino

```
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void driveOneSide() {
  driveForward(DRIVE_SPEED);
  delay(SIDE_MS);
  stopMotors();
  delay(200);
}

void driveSquare() {
  for (int side = 0; side < 4; side++) {
    driveOneSide();
    turnLeft(TURN_SPEED);
    delay(TURN_MS);
    stopMotors();
    delay(200);
  }
}

void setup() {
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  Serial.begin(9600);
}

void loop() {
  waitForStart();
  Serial.println(F("Quadrat startet"));
  driveSquare();
  Serial.println(F("Fertig"));
}
```

Stelle die Zeiten selbst ein. `SIDE_MS` bestimmt die Seitenlänge, `TURN_MS` den Drehwinkel.

Fehler und Merken

Häufiger Fehler

So sieht es aus	Das hilft
Die Räder stehen still	Prüfe, ob `drive` den Treiber mit `PIN_STBY` auf `HIGH` schaltet.
Das Auto fährt rückwärts	Tausche in `drive` die Pins `PIN_BIN1` und `PIN_AIN1`.
Das Auto zieht nach links	Erhöhe `RIGHT_TRIM` um einige Stufen.
Die Taste startet nichts	Prüfe `pinMode(PIN_BUTTON, INPUT_PULLUP)`. Gedrückt liefert `LOW`.

Merken

`setup` enthält keinen Fahrbefehl. Das Gate wartet auf die Taste und zählt 3, 2, 1. `drive` bekommt links und rechts getrennte Geschwindigkeiten. Positiv fährt vorwärts, negativ rückwärts.

- [ ] Ich kann erklären, warum `setup` nie fährt
- [ ] Ich habe das Gate mit Taste und Countdown getestet
- [ ] Ich habe vorwärts, rückwärts und beide Drehungen gesehen
- [ ] Ich habe ein Quadrat mit eigenen Fahrfunktionen gebaut

3
Ultraschall

Ziel

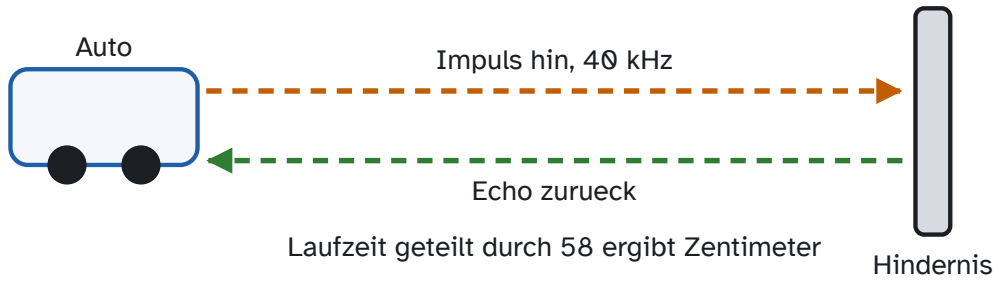
Du misst mit dem Ultraschallsensor den Abstand zu einer Wand. Du verstehst, warum man die Laufzeit durch 58 teilt. Du baust eine Notbremse, die vor einem Hindernis anhält.

Mission

Baue ein Auto, das den Abstand misst und vor einem Hindernis rechtzeitig stehen bleibt.

Wie Schall den Abstand misst

Der Ultraschallsensor arbeitet wie ein Echo in einer Höhle. Er schickt einen kurzen Schallimpuls nach vorn. Der Schall fliegt bis zur Wand, wird dort zurückgeworfen und kommt als Echo wieder an. Der Sensor misst die Zeit zwischen Senden und Empfangen.



Schall legt in der Luft etwa 34 000 Zentimeter pro Sekunde zurück. Das sind 0.034 Zentimeter in einer Mikrosekunde. Der Schall fliegt den Weg zweimal: hin und zurück. Ein Zentimeter Abstand kostet deshalb `2 / 0.034 = 58` Mikrosekunden Laufzeit. Aus der gemessenen Zeit bekommst du den Abstand mit einer einzigen Division:

$$\text{Abstand in cm} = \frac{\text{Laufzeit in µs}}{58}$$

Der Arduino wartet höchstens 25 000 Mikrosekunden auf das Echo. Das sind gut vier Meter. Kommt in dieser Zeit nichts zurück, ist die Messung ungültig. Das passiert, wenn das Hindernis den Schall schluckt oder schräg wegwirft. Der Sketch gibt dann `-1` zurück, und das Auto hält an.

Der Sensor am Auto

Der Sensor sitzt vorn am Auto. Zwei Pins verbinden ihn mit dem Arduino: D13 schickt den Impuls, D12 empfängt das Echo.

Pin	Bedeutung
D13	TRIG: löst einen Schallimpuls aus
D12	ECHO: bleibt HIGH, solange das Echo läuft
5V, GND	Versorgung des Sensors

Abstand messen

Der Sketch misst alle 200 Millisekunden. Damit die Anzeige ruhiger wird, mittelt er drei Messungen.

Abstand anzeigen 041_distance_solution.ino

```
// Kapitel 3: Abstand messen und im Serial Monitor anzeigen.

const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;    // Ultraschall Senden
const uint8_t PIN_ECHO = 12;    // Ultraschall Empfangen

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltige Messung
  }
  return (int)(duration / 58UL);
```

Fortsetzung 041_distance_solution.ino

```
53  }
54
55  int readAverageCm() {
56    long sum = 0;
57    for (int i = 0; i < 3; i++) {
58      int cm = readDistanceCm();
59      if (cm < 0) {
60        return -1;  // eine ungueltige Messung genuegt
61      }
62      sum += cm;
63      delay(20);
64    }
65    return (int)(sum / 3);
66  }
67
68  void setup() {
69    Serial.begin(9600);
70    pinMode(PIN_PWMA, OUTPUT);
71    pinMode(PIN_PWMB, OUTPUT);
72    pinMode(PIN_AIN1, OUTPUT);
73    pinMode(PIN_BIN1, OUTPUT);
74    pinMode(PIN_STBY, OUTPUT);
75    pinMode(PIN_BUTTON, INPUT_PULLUP);
76    pinMode(PIN_TRIG, OUTPUT);
77    pinMode(PIN_ECHO, INPUT);
78    stopMotors();
79    waitForStart();
80  }
81
82  void loop() {
83    int cm = readAverageCm();
84    if (cm < 0) {
85      Serial.println(F("keine Messung"));
86    } else {
87      Serial.print(cm);
88      Serial.println(F(" cm"));
89    }
90    delay(200);
91  }
```

Zeile 42 bis 53 enthalten `readDistanceCm`. Zuerst zieht der Arduino TRIG für 2 Mikrosekunden auf LOW, dann für 10 Mikrosekunden auf HIGH. Diese 10 Mikrosekunden sind der Startbefehl für den Sensor.

Danach misst `pulseIn` in Zeile 48, wie lange ECHO auf HIGH bleibt. Die Zahl 25000 ist das Zeitlimit in Mikrosekunden. Ohne eigenes Limit würde `pulseIn` etwa eine Sekunde warten. Das ist zu lang für eine Notbremse, deshalb steht das kurze Limit hier ausdrücklich im Aufruf.

Zeile 49 bis 52 prüfen das Ergebnis. Bei 0 gibt der Sensor `-1` zurück. Sonst teilt Zeile 52 die Laufzeit durch 58 und schneidet die Nachkommastellen ab.

Zeile 55 bis 66 mitteln drei Messungen. Ist eine davon ungültig, gibt auch der Mittelwert `-1` zurück. Zwischen den Messungen wartet der Sketch 20 Millisekunden, damit der Sensor nicht in sein eigenes Echo hineinhört. Zeile 82 bis 91 geben den Wert aus und warten 200 Millisekunden bis zur nächsten Runde.

Probiere

Halte deine Hand in verschiedenen Abständen vor den Sensor. Vergleiche die Zahlen im Serial Monitor mit einem Lineal. Ändere dann in Zeile 90 die Pause von 200 auf 500 Millisekunden.

Wenn die Messung danebenliegt

Der Sensor ist kein Lineal. Drei Fälle machen ihm zu schaffen:

- **Weiche Stoffe** wie Wolle oder ein Pullover schlucken einen Teil des Schalls. Das Echo ist leise, der Wert wird zu groß.
- **Schräge Flächen** werfen den Schall zur Seite. Er kommt nicht zum Sensor zurück, und der Sketch meldet `-1`.
- **Unter 2 cm** kann der Sensor Senden und Empfangen nicht trennen. Die Messung wird ungenau oder liefert `-1`.

Für die Notbremse ist das wichtig. Ein fehlendes Echo heißt nicht, dass der Weg frei ist. Das Auto behandelt `-1` wie ein Hindernis und hält an. So bleibt es auch bei einem weichen oder schrägen Hindernis stehen, statt weiterzufahren.

Mission: Notbremse

Jetzt fährt das Auto los und bremst bei 20 cm. Der Übungssketch ist schon vollständig. Er misst, entscheidet und fährt. Es fehlen nur die beiden `TODO`-Zeilen in `loop()`. Die neuen Konstanten stehen oben bei den Pins.

Abbildung 3.1 zeigt die Entscheidung: Ist der Abstand kleiner als 20 cm, stoppt das Auto. Sonst fährt es weiter und misst in der nächsten Runde neu.

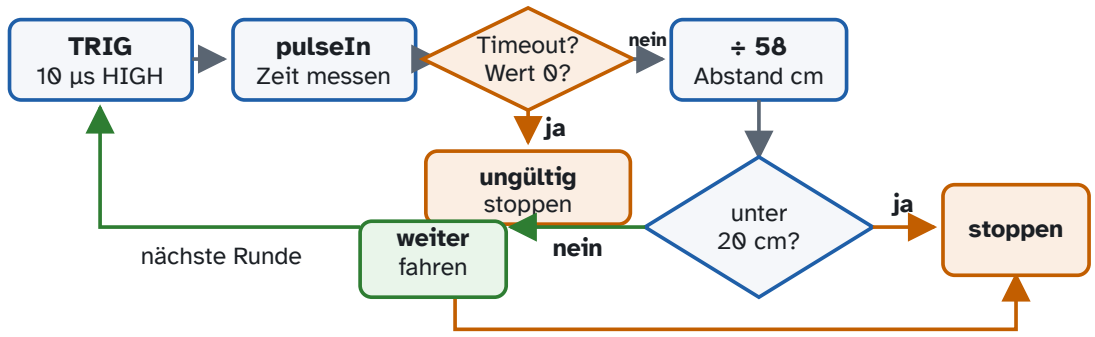


Abbildung 3.1: Messen und entscheiden. Ein Timeout ist eine ungültige Messung, dann stoppt das Auto.

Deine Aufgabe

Ergänze die beiden `TODO`-Zeilen. Überlege zuerst: Welche Funktion hält das Auto an, welche lässt es fahren? Die Schwelle steht in `STOP_DISTANCE_CM`.

Dein Code 046_stop_before_wall_exercise.ino

```ino
// Kapitel 3, Aufgabe: Notbremse vor einem Hindernis.
// Ergaenze die beiden TODO-Zeilen in loop().
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;    // Ultraschall Senden
const uint8_t PIN_ECHO = 12;    // Ultraschall Empfangen
const int STOP_DISTANCE_CM = 20;  // naeher heran: anhalten
const int DRIVE_SPEED = 120;      // 120 von 255

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
```

Fortsetzung 046_stop_before_wall_exercise.ino

```
53
54  int readDistanceCm() {
55    digitalWrite(PIN_TRIG, LOW);
56    delayMicroseconds(2);
57    digitalWrite(PIN_TRIG, HIGH);
58    delayMicroseconds(10);
59    digitalWrite(PIN_TRIG, LOW);
60    unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
61    if (duration == 0) {
62      return -1;  // kein Echo: ungueltige Messung
63    }
64    return (int)(duration / 58UL);
65  }
66
67  void setup() {
68    Serial.begin(9600);
69    pinMode(PIN_PWMA, OUTPUT);
70    pinMode(PIN_PWMB, OUTPUT);
71    pinMode(PIN_AIN1, OUTPUT);
72    pinMode(PIN_BIN1, OUTPUT);
73    pinMode(PIN_STBY, OUTPUT);
74    pinMode(PIN_BUTTON, INPUT_PULLUP);
75    pinMode(PIN_TRIG, OUTPUT);
76    pinMode(PIN_ECHO, INPUT);
77    stopMotors();
78    waitForStart();
79  }
80
81  void loop() {
82    int cm = readDistanceCm();
83    if (cm < 0) {
84      Serial.println(F("keine Messung"));
85    } else {
86      Serial.print(cm);
87      Serial.println(F(" cm"));
88    }
89
90    // Auch -1 bedeutet: anhalten.
91    if (cm < 0 || cm < STOP_DISTANCE_CM) {
92      // TODO: beide Motoren anhalten
93    } else {
94      // TODO: geradeaus fahren
95    }
96    delay(200);
97  }
```

Ergänze die fehlenden Zeilen.

Lösung Notbremse bei 20 cm

Notbremse bei 20 cm 046_stop_before_wall_solution.ino

```
// Kapitel 3, Loesung: Notbremse vor einem Hindernis.
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;    // Ultraschall Senden
const uint8_t PIN_ECHO = 12;    // Ultraschall Empfangen
const int STOP_DISTANCE_CM = 20;  // naeher heran: anhalten
const int DRIVE_SPEED = 120;      // 120 von 255
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
```

Fortsetzung 046_stop_before_wall_solution.ino

```
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltige Messung
  }
  return (int)(duration / 58UL);
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  stopMotors();
  waitForStart();
}

void loop() {
  int cm = readDistanceCm();
  if (cm < 0) {
    Serial.println(F("keine Messung"));
  } else {
    Serial.print(cm);
    Serial.println(F(" cm"));
  }

  // Auch -1 bedeutet: anhalten. Ein fehlendes Echo ist nie "frei".
  if (cm < 0 || cm < STOP_DISTANCE_CM) {
    stopMotors();
  } else {
```

Fortsetzung 046_stop_before_wall_solution.ino

```
88    // Auch -1 bedeutet: anhalten. Ein fehlendes Echo ist nie "frei".
89    if (cm < 0 || cm < STOP_DISTANCE_CM) {
90      stopMotors();
91    } else {
92      drive(DRIVE_SPEED, DRIVE_SPEED);
93    }
94    delay(200);
95  }
```

Häufiger Fehler

So sieht es aus	Das hilft
Im Monitor steht dauernd „keine Messung“.	Prüfe TRIG auf D13 und ECHO auf D12. Halte den Sensor frei und richte ihn gerade auf eine Wand. Kommt kein Echo, gibt der Sketch `-1` aus.
Die Werte springen wild hin und her.	Mittle drei Messungen wie in `readAverageCm`. Halte mindestens 2 cm Abstand und ziele auf eine ebene Fläche.
Das Auto bremst nie.	Lies im Monitor mit: Fällt der Wert wirklich unter 20? Sonst erhöhe `STOP_DISTANCE_CM` auf 30.

Merken

Der Sensor misst die Laufzeit des Schalls, nicht den Abstand direkt. Ein Zentimeter Hin- und Rückweg dauert 58 Mikrosekunden. `pulseIn` bekommt hier ein kurzes Zeitlimit. Ohne eigenes Limit wartet es etwa eine Sekunde. Weiche und schräge Flächen liefern keine gültige Messung. Der Sketch gibt dann `-1` zurück und das Auto hält an.

- [ ] Ich kann den Abstand im Serial Monitor ablesen
- [ ] Ich weiß, warum durch 58 geteilt wird
- [ ] Ich kann das Zeitlimit von `pulseIn` erklären
- [ ] Mein Auto hält vor einem Hindernis an

4
Linie folgen

Ziel

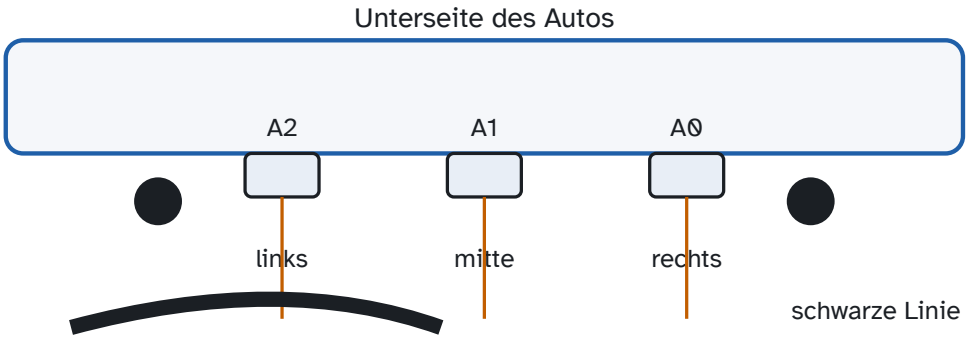
Du liest die Rohwerte von drei Liniensensoren. Du wählst eine eigene Schwelle für schwarz und weiß. Du baust ein Auto, das einer schwarzen Linie folgt.

Mission

Baue ein Auto, das eine schwarze Linie auf hellem Grund findet und ihr bis zum Ende folgt.

Drei Sensoren sehen die Linie

Unter dem Auto sitzen drei kleine Sensoren nebeneinander. Jeder Sensor hat eine Infrarot-LED und einen Fototransistor. Die LED strahlt nach unten. Ein heller Untergrund wirft viel Licht zurück, schwarzer Untergrund schluckt das meiste davon.



Der Arduino liest die Helligkeit als Zahl zwischen 0 und 1023. Auf Schwarz liegen die Werte ungefähr zwischen 250 und 850. Auf Weiß sind es etwa 1000. Liegen alle drei Werte über 950, sieht kein Sensor eine dunkle Linie. Das passiert am Linienende oder wenn du das Auto hochhebst. Diese Zahlen hängen von Raum, Akku und Untergrund ab, deshalb misst du sie zuerst selbst.

Rohwerte lesen

Dieser Sketch zeigt die Werte aller drei Sensoren im Serial Monitor. Er fährt nicht, sondern wartet nur auf den Start und druckt dann weiter.

Abbildung 4.1 zeigt schon einmal den ganzen Weg: von den drei Sensoren über den Fehlerwert bis zu den beiden Motorgeschwindigkeiten.

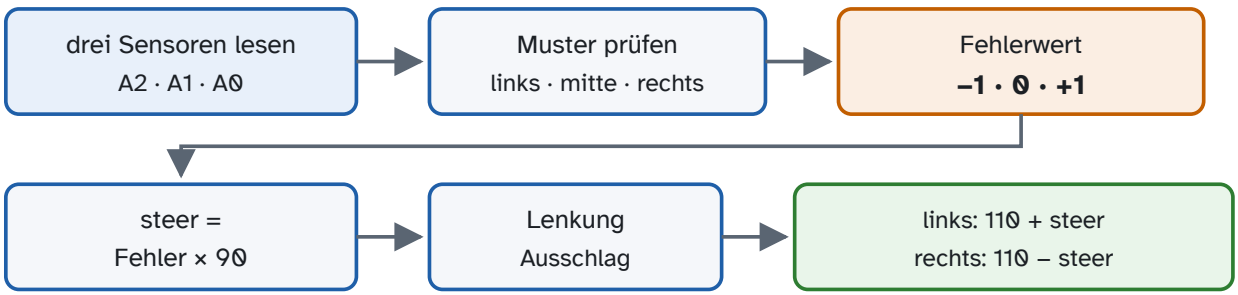


Abbildung 4.1: Linie lesen und lenken. Aus dem Muster wird ein Fehler, aus dem Fehler die Lenkung.

Rohwerte anzeigen 051_calibrate_line_solution.ino

```
// Kapitel 4: Rohwerte der drei Liniensensoren anzeigen.

const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_LINE_LEFT = A2;   // Linien-Sensor links
const uint8_t PIN_LINE_MIDDLE = A1;   // Linien-Sensor Mitte
const uint8_t PIN_LINE_RIGHT = A0;   // Linien-Sensor rechts

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_LINE_LEFT, INPUT);
  pinMode(PIN_LINE_MIDDLE, INPUT);
```

Fortsetzung 051_calibrate_line_solution.ino

```
53    pinMode(PIN_LINE_RIGHT, INPUT);
54    stopMotors();
55    waitForStart();
56  }
57
58  void loop() {
59    Serial.print(F("links "));
60    Serial.print(analogRead(PIN_LINE_LEFT));
61    Serial.print(F("  mitte "));
62    Serial.print(analogRead(PIN_LINE_MIDDLE));
63    Serial.print(F("  rechts "));
64    Serial.println(analogRead(PIN_LINE_RIGHT));
65    delay(300);
66  }
```

Zeile 58 bis 65 bilden den Kern. `analogRead` liest die Spannung am Pin und macht daraus eine Zahl. Links steht auf A2, in der Mitte auf A1 und rechts auf A0. Die Ausgabe wiederholt sich alle 300 Millisekunden.

Die Schwelle wählen

Lege das Auto auf eine weiße Fläche und lies die drei Werte ab. Schiebe es dann so, dass alle drei Sensoren auf der schwarzen Linie stehen, und lies wieder ab. Notiere beide Zahlenpaare.

Deine Messung

Schwarz: links … · mitte … · rechts …
Weiß: links … · mitte … · rechts …
Meine Schwelle: …

Die Schwelle liegt zwischen Schwarz und Weiß. Bei den Werten aus dem Kasten sind das zum Beispiel 600. Alles unter 600 gilt als schwarz. Trage deine eigene Zahl als `LINE_THRESHOLD` in den nächsten Sketch ein.

Achtung

Die Schwelle gilt nur für diesen Raum und diesen Akkustand. Ein leerer Akku macht die LED dunkler, und alle Werte sinken. Miss deshalb vor jeder Fahrt neu und passe `LINE_THRESHOLD` an.

Lenken mit einem Fehlerwert

Der Sketch rechnet aus der Sensorposition einen Fehlerwert:

Sensor	Fehler	Bedeutung
links schwarz	−1	die Linie liegt links
mitte schwarz	0	die Linie liegt mittig
rechts schwarz	+1	die Linie liegt rechts

Aus dem Fehler wird die Lenkung: `steer = error * 90`. Bei Fehler +1 ist `steer` gleich 90. Der linke Motor bekommt `110 + 90 = 200`, der rechte `110 - 90 = 20`. Das Auto zieht nach rechts. Bei −1 dreht es sich andersherum. Bei 0 fahren beide Motoren gleich schnell geradeaus.

Abbildung 4.1 zeigt diesen Weg noch einmal als Plan.

Das nennt man proportionale Lenkung: Je weiter die Linie seitlich liegt, desto stärker der Ausschlag. Ein PID-Regler macht das feiner, braucht aber Kommazahlen und viel Erklärung. Für den Anfang reicht diese einfache Regel.

Zwei Sonderfälle bekommen ein festes Verhalten. Sehen alle drei Sensoren weiß, ist die Linie zu Ende oder das Auto steht neben ihr. Dann stoppt das Auto. Sehen alle drei Sensoren schwarz, kann das eine

breite Linie oder eine Querlinie sein. Auch dann stoppt das Auto kurz. Das Hochheben des Autos ist kein Test für „alle schwarz“: In der Luft sehen die Sensoren meist weiß.

Mission: Linie folgen

Der Übungssketch ist vollständig und baut so, wie er ist. Es fehlen nur die `TODO`-Zeilen in `loop()`.

> **Deine Aufgabe**
>
> Ergänze die `TODO`-Zeilen in `loop()`. Setze den Fehlerwert und berechne daraus `steer`. Welche Geschwindigkeit bekommt links, welche rechts? Lege fest: Alle drei Sensoren weiß heißt Linienende, das Auto stoppt.

Dein Code 056_line_follow_exercise.ino

```ino
// Kapitel 4, Aufgabe: Der Linie folgen.
// Ergaenze die TODOs in loop(). Der Rest ist fertig und baut schon.
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_LINE_LEFT = A2;   // Linien-Sensor links
const uint8_t PIN_LINE_MIDDLE = A1;   // Linien-Sensor Mitte
const uint8_t PIN_LINE_RIGHT = A0;   // Linien-Sensor rechts
const int LINE_THRESHOLD = 600;   // kleiner: schwarz
const int BASE_SPEED = 110;       // Grundtempo, 110 von 255
const int STEER_GAIN = 90;        // Lenkstaerke pro Fehlerschritt

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
```

Fortsetzung 056_line_follow_exercise.ino

```
}

bool onBlack(uint8_t pin) {
  return analogRead(pin) < LINE_THRESHOLD;
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_LINE_LEFT, INPUT);
  pinMode(PIN_LINE_MIDDLE, INPUT);
  pinMode(PIN_LINE_RIGHT, INPUT);
  stopMotors();
  waitForStart();
}

void loop() {
  bool onLeft = onBlack(PIN_LINE_LEFT);
  bool onMiddle = onBlack(PIN_LINE_MIDDLE);
  bool onRight = onBlack(PIN_LINE_RIGHT);
  // TODO: alle drei weiss: stopMotors(), Meldung, delay, return.
  // TODO: alle drei schwarz: stopMotors(), delay, return.
  int error = 0;
  // TODO: links schwarz ergibt -1, rechts schwarz ergibt +1.
  // TODO: steer berechnen und drive aufrufen.
  stopMotors();  // Platzhalter, damit der Sketch jetzt schon baut
  delay(10);
}
```

Ergänze die fehlenden Zeilen.

Lösung Der Linie folgen

Der Linie folgen 056_line_follow_solution.ino

```
// Kapitel 4, Loesung: Der Linie folgen.
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_LINE_LEFT = A2;  // Linien-Sensor links
const uint8_t PIN_LINE_MIDDLE = A1;  // Linien-Sensor Mitte
const uint8_t PIN_LINE_RIGHT = A0;  // Linien-Sensor rechts
const int LINE_THRESHOLD = 600;  // kleiner: schwarz
const int BASE_SPEED = 110;      // Grundtempo, 110 von 255
const int STEER_GAIN = 90;       // Lenkstaerke pro Fehlerschritt

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
```

Fortsetzung 056_line_follow_solution.ino

```
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

bool onBlack(uint8_t pin) {
  return analogRead(pin) < LINE_THRESHOLD;
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_LINE_LEFT, INPUT);
  pinMode(PIN_LINE_MIDDLE, INPUT);
  pinMode(PIN_LINE_RIGHT, INPUT);
  stopMotors();
  waitForStart();
}

void loop() {
  bool onLeft = onBlack(PIN_LINE_LEFT);
  bool onMiddle = onBlack(PIN_LINE_MIDDLE);
  bool onRight = onBlack(PIN_LINE_RIGHT);
  if (!onLeft && !onMiddle && !onRight) {
    stopMotors();  // alle weiss: Linie zu Ende
    Serial.println(F("keine Linie"));
    delay(100);
    return;
  }
  if (onLeft && onMiddle && onRight) {
    stopMotors();  // breite Linie oder Querlinie
    delay(100);
    return;
  }
  int error = 0;
  if (onLeft) {
    error = -1;
  } else if (onRight) {
    error = 1;
```

Fortsetzung 056_line_follow_solution.ino

```
    error = -1;
  } else if (onRight) {
    error = 1;
  }
  int steer = error * STEER_GAIN;
  drive(BASE_SPEED + steer, BASE_SPEED - steer);
  delay(10);
}
```

Probiere

Ändere `BASE_SPEED` von 110 auf 150. Verschiebe danach `LINE_THRESHOLD` um 100 nach oben und beobachte die Fahrt.

Häufiger Fehler

So sieht es aus	Das hilft
Das Auto dreht sich im Kreis.	Links und rechts sind vertauscht. Prüfe, ob A2 wirklich links und A0 wirklich rechts sitzt.
Das Auto fährt über die Linie hinaus.	Die Schwelle passt nicht. Lies die Rohwerte neu und wähle eine Zahl zwischen Schwarz und Weiß.
Das Auto hält an und meldet „keine Linie“.	Alle drei Sensoren sehen weiß. Das ist das Linienende. Setze das Auto wieder auf die Linie oder starte neu.
Das Auto hält auf einer breiten Linie an.	Alle drei Sensoren sehen schwarz. Das kann eine breite Linie oder eine Querlinie sein. Das Anhalten ist so gewollt.

Merken

Schwarz schluckt Infrarotlicht, Weiß wirft es zurück. Die Schwelle liegt zwischen beiden Werten. Der Fehler –1, 0, +1 sagt, wo die Linie liegt, und `steer = error * 90` macht daraus eine Lenkbewegung. Sehen alle drei Sensoren weiß, ist die Linie zu Ende, und das Auto stoppt.

- [ ] Ich kann die drei Rohwerte im Monitor lesen
- [ ] Ich habe eine eigene Schwelle gemessen und kenne den Fehlerwert
- [ ] Mein Auto folgt der Linie über eine Kurve

5
Servo

Ziel

Ein Servo ist ein kleiner Motor, der eine genaue Stellung halten kann. Du steuerst ihn mit einem Winkel von 0 bis 180 Grad. Am Ende dreht dein Auto den Kopf, misst links, in der Mitte und rechts und fährt in die Richtung mit dem größten Abstand.

Mission

Baue einen schwenkbaren Kopf. Er schaut nach links, geradeaus und nach rechts. Danach fährt das Auto dorthin, wo am meisten Platz ist.

Ein Motor mit Gedächtnis

Ein normaler Motor dreht sich einfach weiter, solange Strom fließt. Ein Servo ist anders. In seinem Gehäuse sitzen ein Getriebe und eine kleine Steuerung. Du sagst ihm einen Winkel, zum Beispiel 90 Grad. Der Servo dreht dorthin und bleibt stehen.

Der Servo hängt an Pin D10. Drei Adern verbinden ihn mit dem Shield: Masse, 5 Volt und das Steuersignal. Das Signal sagt dem Servo, welchen Winkel er anfahren soll.

In diesem Buch gilt: 0 Grad schaut nach links, 90 Grad geradeaus und 180 Grad nach rechts. Je nach Zusammenbau kann das gespiegelt sein. Dann tauschst du links und rechts im Code.

Achtung

Dreh den Servo nie von Hand, solange das Auto Strom hat. Das Getriebe arbeitet gegen deine Hand und kann kaputtgehen. Schalte vorher aus.

Der Kopf dreht sich

Das erste Programm schickt den Kopf einmal durch seinen ganzen Bereich. `write(90)` heißt: fahre auf 90 Grad. Danach braucht der Servo Zeit. Ohne Wartezeit würde das Programm sofort den nächsten Winkel schicken.

Kopf drehen 061_servo_sweep_solution.ino

```ino
// Kopf langsam nach links, geradeaus, rechts und zurueck drehen.
#include <Servo.h>

const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_SERVO = 10;   // Servo
const int STEP_DELAY_MS = 400; // Zeit zwischen zwei Winkeln
Servo headServo;
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void lookAt(int angle) {
  headServo.write(angle);
  delay(STEP_DELAY_MS);
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  headServo.attach(PIN_SERVO);
  lookAt(90);  // geradeaus starten
  waitForStart();
```

Fortsetzung 061_servo_sweep_solution.ino

```
53  }
54  void loop() {
55    lookAt(0);     // links
56    lookAt(90);    // geradeaus
57    lookAt(180);   // rechts
58    lookAt(90);    // zurueck in die Mitte
59    delay(800);
60  }
```

Zeile 2 bindet die Servo-Bibliothek ein. Sie gehört zum Arduino und ist schon installiert. Zeile 4 bis 10 legen die Pins fest. Zeile 12 erzeugt das Objekt `headServo`, damit steuerst du den Servo später.
Zeile 40 bis 43 fassen das Drehen zusammen. `lookAt()` stellt den Winkel ein und wartet 400 Millisekunden. Zeile 50 verbindet den Servo mit Pin D10. Ab Zeile 54 läuft die Schleife: links, geradeaus, rechts und zurück in die Mitte. Dann wartet das Programm 800 Millisekunden und beginnt von vorn.

Achtung

Der Servo blockiert Timer1 des Uno. Timer1 erzeugt auch das PWM-Signal auf Pin 9 und Pin 10. Darum darfst du die Motoren nicht über Pin 9 oder 10 regeln. Bei diesem Auto liegen die Motoren auf D5 und D6, die bleiben frei. Der Servo selbst sitzt auf D10.

Probiere

Verkleinere den Bereich auf 30 bis 150 Grad. Der Kopf schwenkt dann nur noch ein Stück. Erhöhe danach `STEP_DELAY_MS` auf 600. Der Kopf wird langsamer, kommt aber sicher an jedem Winkel an.

Mission: Der freie Weg

Jetzt bekommt der Kopf eine Aufgabe. Er misst nacheinander links, in der Mitte und rechts den Abstand. Die Funktion `readDistanceCm()` kennst du aus dem Ultraschallkapitel. Danach vergleicht das Programm die drei Werte und fährt in die Richtung mit dem größten Abstand.

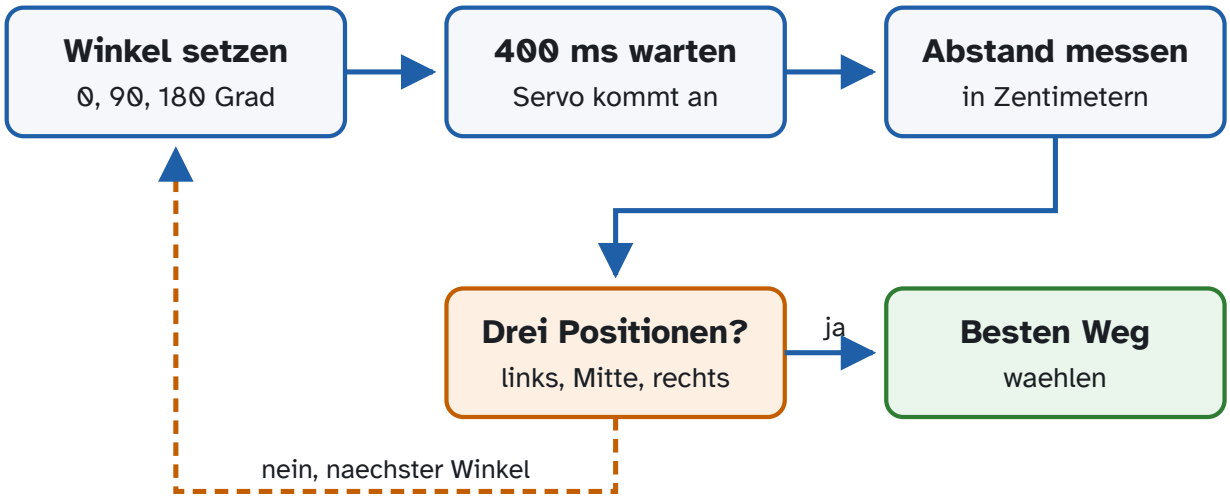


Abbildung 5.1: Eine Messrunde: Winkel setzen, 400 Millisekunden warten, Abstand messen. Nach drei Positionen wählt das Programm den besten Weg.

Abbildung 5.1 zeigt die vier Schritte. Nach jeder Messung dreht der Kopf zum nächsten Winkel. Erst wenn alle drei Werte vorliegen, entscheidet das Programm.

Der Übungssketch ist vollständig und baut so, wie er ist. Nur die Funktion `chooseAndDrive()` entscheidet noch nicht; im Übungssketch stoppt sie das Auto. Sie bekommt die drei Messwerte und entscheidet, wohin das Auto fährt.

Deine Aufgabe

Setze `bestCm` und `bestAngle` für links und rechts. Stelle den Kopf vor dem Fahren auf 90 Grad. Fahre nur, wenn `bestCm` mindestens `MIN_FREE_CM` ist, sonst stoppe mit `stopMotors()`. Drehe dann in die beste Richtung und fahre geradeaus.

Dein Code 066_look_around_exercise.ino

```
// Kapitel 5, Aufgabe: Ergaenze nur chooseAndDrive().
// Der Rest des Sketches ist fertig und baut so, wie er ist.
#include <Servo.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;   // Ultraschall Senden
const uint8_t PIN_ECHO = 12;   // Ultraschall Empfangen
const uint8_t PIN_SERVO = 10;  // Servo
const int STEP_MS = 400;       // Wartezeit nach jedem Winkel
const int TURN_SPEED = 120;    // Drehgeschwindigkeit, 120 von 255
const int DRIVE_SPEED = 150;   // Fahrgeschwindigkeit, 150 von 255
const int MIN_FREE_CM = 30;    // weniger Platz: nicht fahren
Servo headServo;
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
```

Fortsetzung 066_look_around_exercise.ino

```
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltige Messung
  }
  return (int)(duration / 58UL);
}
int lookAndMeasure(int angle) {
  headServo.write(angle);
  delay(STEP_MS);
  return readDistanceCm();
}
void chooseAndDrive(int leftCm, int middleCm, int rightCm) {
  int bestAngle = 90;  // 0 = links, 90 = Mitte, 180 = rechts
  int bestCm = middleCm;
  // TODO: Ist leftCm groesser? Dann bestCm und bestAngle setzen.
  // TODO: Ist rightCm groesser? Dann bestCm und bestAngle setzen.
  // TODO: Kopf auf 90 Grad stellen und STEP_MS warten.
  // TODO: Ist bestCm kleiner als MIN_FREE_CM? Dann stoppen und return.
  // TODO: Sonst in Richtung bestAngle drehen und geradeaus fahren.
  stopMotors();  // Platzhalter, damit der Sketch jetzt schon baut
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  stopMotors();
  headServo.attach(PIN_SERVO);
  headServo.write(90);
  waitForStart();
}
void loop() {
  int leftCm = lookAndMeasure(0);
  int middleCm = lookAndMeasure(90);
  int rightCm = lookAndMeasure(180);
  chooseAndDrive(leftCm, middleCm, rightCm);
}
```

Ergänze die fehlenden Zeilen.

Lösung Mission freier Weg

Mission freier Weg 066_look_around_solution.ino

```ino
#include <Servo.h>
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;    // Ultraschall Senden
const uint8_t PIN_ECHO = 12;    // Ultraschall Empfangen
const uint8_t PIN_SERVO = 10;   // Servo
const int STEP_MS = 400;        // Wartezeit nach jedem Winkel
const int TURN_SPEED = 120;     // Drehgeschwindigkeit, 120 von 255
const int DRIVE_SPEED = 150;    // Fahrgeschwindigkeit, 150 von 255
const int MIN_FREE_CM = 30;     // weniger Platz: nicht fahren
Servo headServo;
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
```

Fortsetzung 066_look_around_solution.ino

```
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltige Messung
  }
  return (int)(duration / 58UL);
}
int lookAndMeasure(int angle) {
  headServo.write(angle);
  delay(STEP_MS);
  return readDistanceCm();
}
void chooseAndDrive(int leftCm, int middleCm, int rightCm) {
  int bestAngle = 90;  // 0 = links, 90 = Mitte, 180 = rechts
  int bestCm = middleCm;
  if (leftCm > bestCm) {
    bestCm = leftCm;
    bestAngle = 0;
  }
  if (rightCm > bestCm) {
    bestCm = rightCm;
    bestAngle = 180;
  }

  headServo.write(90);  // Kopf vor der Fahrt gerade stellen
  delay(STEP_MS);

  if (bestCm < MIN_FREE_CM) {
    stopMotors();  // ueberall zu eng oder keine Messung
    Serial.println(F("ueberall zu eng"));
    return;
  }

  int turn = 0;
  if (bestAngle == 0) {
```

Fortsetzung 066_look_around_solution.ino

```
92      turn = -TURN_SPEED;
93    } else if (bestAngle == 180) {
94      turn = TURN_SPEED;
95    }
96    drive(turn, -turn);  // auf der Stelle drehen
97    delay(600);
98    drive(DRIVE_SPEED, DRIVE_SPEED);
99    delay(700);
100   stopMotors();
101   delay(1000);
102 }
103 void setup() {
104   Serial.begin(9600);
105   pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
106   pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
107   pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
108   pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
109   stopMotors();
110   headServo.attach(PIN_SERVO);
111   headServo.write(90);
112   waitForStart();
113 }
114 void loop() {
115   int leftCm = lookAndMeasure(0);
116   int middleCm = lookAndMeasure(90);
117   int rightCm = lookAndMeasure(180);
118   chooseAndDrive(leftCm, middleCm, rightCm);
119 }
```

Im Lösungssketch beginnt die Entscheidung bei Zeile 69. Zuerst gilt die Mitte mit 90 Grad. Zwei `if`-Abfragen prüfen, ob links oder rechts mehr Platz ist. `bestAngle` merkt sich die Richtung.

Vor dem Fahren stellt Zeile 81 den Kopf wieder gerade. Nur so zeigt der Ultraschall in die Fahrtrichtung. Zeile 84 prüft `bestCm` gegen `MIN_FREE_CM`. Ist der beste Abstand zu klein oder die Messung ungültig (`-1`), stoppt das Auto sofort. Sonst dreht es auf der Stelle und fährt ein kurzes Stück. `stopMotors()` beendet die Fahrt.

Häufiger Fehler

So sieht es aus	Das hilft
Der Servo brummt und zittert.	Der Winkel liegt außerhalb von 0 bis 180 Grad, oder der Kopf stößt an. Prüfe den Bereich und drehe den Kopf von Hand nur bei ausgeschaltetem Auto.
Der Kopf springt nur ruckartig.	Die Wartezeit ist zu kurz. Erhöhe `STEP_DELAY_MS` auf 600 Millisekunden.
Das Auto fährt sofort nach dem Einschalten los.	Der Start fehlt. In jedem fahrenden Sketch muss `waitForStart()` vor der ersten Fahrt stehen.
Der Motor auf D5 oder D6 läuft nicht.	Prüfe, ob `PIN_PWMA` und `PIN_PWMB` im Sketch zu den Kabeln passen. Der Servo darf nicht auf D5 oder D6 hängen.

Merken

> Ein Servo fährt einen Winkel an und hält ihn. `write(90)` stellt den Kopf geradeaus. Nach jedem Winkel braucht der Servo eine kurze Pause. Der Servo belegt Timer1, deshalb bleiben D5 und D6 für die Motoren. Vor jeder Fahrt wartet das Programm auf die Taste.

- [ ] Ich kann den Servo an D10 anschließen und mit `write()` drehen.
- [ ] Ich weiß, dass 90 Grad geradeaus bedeutet.
- [ ] Ich kann drei Abstände messen und den größten Wert finden.
- [ ] Ich kann das Auto in die freie Richtung fahren lassen.

6
MPU6050

Ziel

Der MPU6050 ist ein Chip mit zwei Sensoren. Der eine misst Beschleunigung, der andere die Drehung. Du liest beide über zwei dünne Kabel aus. Am Ende zeigt dein Auto seine Neigung an und warnt bei einem kräftigen Stoß mit roter LED.

Mission

Baue einen Neigungsmesser und einen Stoßalarm. Der Bildschirm zeigt Roll und Pitch in Grad. Bei einem Stoß leuchtet die LED zwei Sekunden lang.

Zwei Sensoren in einem Chip

Der MPU6050 steckt auf einer kleinen Platine, dem GY-521-Modul. In seinem Gehäuse arbeiten zwei Sensoren.

Der Beschleunigungssensor misst, wie stark das Auto beschleunigt wird. Er spürt auch die Erdanziehung. Ein Auto in Ruhe misst deshalb nicht 0, sondern etwa 9,8 Meter pro Sekunde zum Quadrat, kurz m/s^2. Das ist die Fallbeschleunigung `g`.

Der Gyrosensor misst, wie schnell sich das Auto dreht. Die Adafruit- Bibliothek liefert diesen Wert in Radiant pro Sekunde, kurz rad/s. Der nächste Abschnitt zeigt, was Radiant bedeutet.

Anschluss und Adresse

Das Modul hängt am I2C-Bus. I2C braucht nur zwei Datenleitungen. Beim Uno sind das A4 und A5. A4 ist `SDA` und trägt die Daten. A5 ist `SCL` und gibt den Takt vor. Das Shield verbindet beide Leitungen mit dem MPU6050.

Jedes I2C-Gerät hat eine Adresse. Der MPU6050 antwortet auf `0x68`. Diese Zahl steht in der Bibliothek als Standardwert, du musst sie nicht selbst setzen.

Tipp

`0x68` ist eine Hexadezimalzahl. Hex-Zahlen haben 16 Ziffern: erst 0 bis 9, dann A, B, C, D, E und F. Dabei ist A gleich 10, B gleich 11 und so weiter bis F gleich 15. Die zwei Zeichen `0x` zeigen nur: Diese Zahl ist zur Basis 16 geschrieben. `0x68` heißt also 6 mal 16 plus 8, das sind 104. Für den Sketch musst du nichts umrechnen. Du tippst die Adresse genau so ab, wie sie in der Bibliothek steht.

Pin	Bedeutung
A4	SDA, Datenleitung
A5	SCL, Taktleitung
0x68	I2C-Adresse des MPU6050

Grad statt Bogenmaß

Ein Winkel lässt sich auf zwei Arten schreiben: in Grad oder in Bogenmaß. Das Bogenmaß benutzt die Länge des Kreisbogens. Ein Bogen der Länge r spannt den Winkel 1 rad auf.

Eine volle Drehung sind 360 Grad. Als Bogenmaß sind das 2 pi rad, also 6,28 rad. Daraus folgt: 1 rad sind rund 57,3 Grad.

Bogenmass und Grad

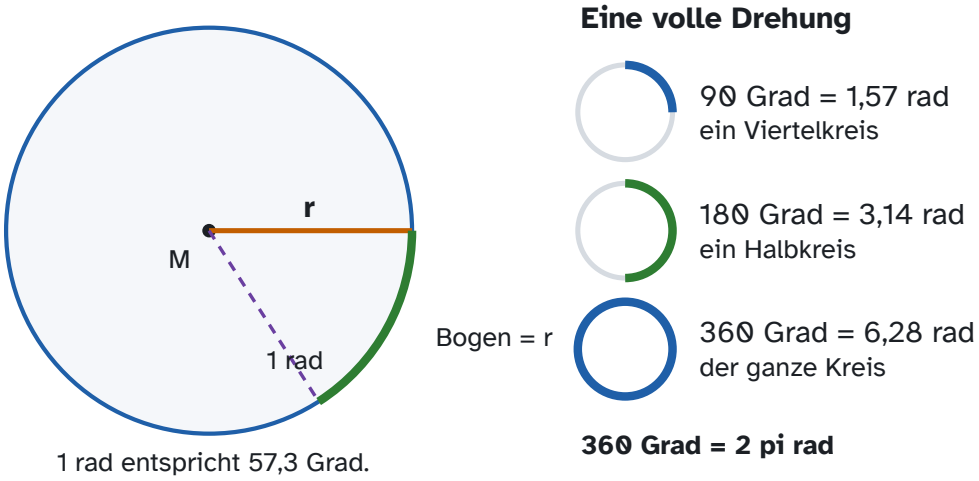


Abbildung 6.1: Ein Bogen der Länge r ergibt 1 rad. 90 Grad sind 1,57 rad, 180 Grad sind 3,14 rad, 360 Grad sind 6,28 rad.

Zwei Merksätze reichen:

- 360 Grad = 2 pi rad = 6,28 rad.
- 1 rad = 57,3 Grad.

In Abbildung 6.1 siehst du beide Schreibweisen nebeneinander. Der Sketch rechnet die Gyro-Werte mit `GYRO_RAD_TO_DEG` um. In der Ausgabe steht dann `deg/s` statt `rad/s`.

Neigung messen

Das erste Programm liest den Sensor alle 200 Millisekunden. Aus den drei Beschleunigungswerten `ax`, `ay` und `az` berechnet es zwei Winkel. Roll kippt nach links oder rechts. Pitch hebt oder senkt die Nase.

Roll und Pitch 071_tilt_solution.ino

```ino
// Neigung in Grad anzeigen: Roll (links/rechts) und Pitch (vorn/hinten).
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const float GYRO_RAD_TO_DEG = 57.2958;  // rad/s in Grad/s

Adafruit_MPU6050 mpu;
sensors_event_t accel, gyro, temp;

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void setup() {
  Serial.begin(9600);

  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
```

Fortsetzung 071_tilt_solution.ino

```
53    pinMode(PIN_BIN1, OUTPUT);
54    pinMode(PIN_STBY, OUTPUT);
55    pinMode(PIN_BUTTON, INPUT_PULLUP);
56    stopMotors();
57
58    if (!mpu.begin()) {
59      Serial.println(F("MPU6050 nicht gefunden. Kabel pruefen."));
60      while (true) {
61        delay(100);
62      }
63    }
64    mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
65    mpu.setGyroRange(MPU6050_RANGE_250_DEG);
66    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
67    delay(1000);  // Sensor kurz beruhigen
68    waitForStart();
69  }
70
71  void loop() {
72    mpu.getEvent(&accel, &gyro, &temp);
73
74    float ax = accel.acceleration.x;
75    float ay = accel.acceleration.y;
76    float az = accel.acceleration.z;
77
78    // Roll: Drehung um die Fahrtrichtung (links/rechts kippen).
79    float roll = atan2(ay, az) * 180.0 / PI;
80    // Pitch: Nase hoch oder runter.
81    float pitch = atan2(-ax, sqrt(ay * ay + az * az)) * 180.0 / PI;
82
83    // Gyro: Drehrate von rad/s in Grad pro Sekunde umrechnen.
84    float gyroX = gyro.gyro.x * GYRO_RAD_TO_DEG;
85    float gyroY = gyro.gyro.y * GYRO_RAD_TO_DEG;
86    float gyroZ = gyro.gyro.z * GYRO_RAD_TO_DEG;
87
88    Serial.print(F("Roll "));
89    Serial.print(roll, 1);
90    Serial.print(F(" Grad, Pitch "));
91    Serial.print(pitch, 1);
92    Serial.println(F(" Grad"));
93    Serial.print(F("Drehrate "));
94    Serial.print(gyroX, 1);
95    Serial.print(F(", "));
96    Serial.print(gyroY, 1);
97    Serial.print(F(", "));
98    Serial.print(gyroZ, 1);
99    Serial.println(F(" deg/s"));
100   delay(200);
101 }
```

Zeile 2 bis 4 binden die Bibliotheken ein. Zeile 15 erzeugt das Objekt `mpu`, Zeile 16 drei Speicher für die Messwerte. Ab Zeile 59 startet `mpu.begin()` den Sensor. Schlägt das fehl, bleibt das Programm in einer Endlosschleife und schreibt eine Meldung auf den seriellen Monitor.

Zeile 73 holt einen neuen Satz Messwerte. `getEvent()` füllt `accel`, `gyro` und `temp` auf einmal. Die drei Beschleunigungswerte stehen in Zeile 75 bis 77. Zeile 80 rechnet Roll, Zeile 82 rechnet Pitch. Beide Formeln nutzen `atan2()`, eine Umkehrfunktion des Tangens.

Zeile 84 bis 100 rechnen die Drehrate in Grad pro Sekunde um. Die drei Werte stehen als `deg/s` im seriellen Monitor.
Die Achsen hängen vom Einbau ab. Hier gilt: x zeigt nach vorn, y nach links, z nach oben. Liegt das Auto gerade, zeigt `az` ungefähr 9,8 und die beiden anderen Werte liegen bei 0. Kippst du zur Seite, ändert sich `ay`.

Achtung

`mpu.begin()` kalibriert den Sensor nicht auf die aktuelle Lage. Es startet nur die Kommunikation. Die angezeigten Winkel sind immer eine frische Messung. Lege das Auto für einen festen Nullpunkt gerade hin und notiere die Werte als Vergleich. Bewege das USB-Kabel nicht, es zieht am Auto.

Probiere

Kippe das Auto langsam nach vorn. Beobachte, wie Pitch größer wird. Tausche dann in Zeile 78 `ay` und `az`. Schon zeigt Roll etwas anderes. Mit diesem Tausch findest du heraus, wie dein Modul eingebaut ist.

Zittern und Driften

Der Beschleunigungssensor ist für Neigung gut geeignet. Der Gyrosensor misst Drehungen, doch sein Wert ist nie exakt null. Kleine Fehler summieren sich auf, wenn du ihn über die Zeit addierst. Man nennt das Driften.
Mit dem Gyro könntest du eine 90-Grad-Drehung abschätzen. Dazu müsstest du die Drehgeschwindigkeit jeden Moment mit der Zeit multiplizieren und alles aufaddieren. Nach wenigen Sekunden wäre das Ergebnis sichtbar falsch. Deshalb bauen wir diese Drehung nicht. Für eine gerade Fahrt und für Neigung reicht der Sensor trotzdem gut aus.

Mission: Stoßalarm

Jetzt misst das Programm die Länge des Beschleunigungsvektors. Dafür quadriert es `ax`, `ay` und `az`, addiert die drei Quadrate und zieht die Wurzel. In Ruhe ergibt das etwa 9,8 m/s². Ein kräftiger Stoß treibt den Wert über 14 m/s².

Deine Aufgabe

Berechne `total` mit der Wurzel aus den drei Quadraten. Vergleiche den Wert mit `SHAKE_LIMIT`. Ist er größer, rufe `warnTwoSeconds(total)` auf.

Dein Code 076_shake_exercise.ino

```ino
// Vollstaendiger Sketch. Ergaenze die Stosserkennung in loop.
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_LED = 4;     // WS2812
const float SHAKE_LIMIT = 14.0;  // Schwelle in m/s^2
const float GYRO_RAD_TO_DEG = 57.2958;  // rad/s in Grad/s
const int WARN_MS = 2000;  // Dauer der Warnung
Adafruit_MPU6050 mpu;
sensors_event_t accel, gyro, temp;
CRGB leds[1];
unsigned long lastRatePrint = 0;  // Zeit der letzten Ausgabe

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void warnTwoSeconds(float total) {
  Serial.print(F("Stoss erkannt: "));
  Serial.println(total, 1);
  leds[0] = CRGB::Red;
  FastLED.show();
```

Fortsetzung 076_shake_exercise.ino

```
  unsigned long start = millis();
  while (millis() - start < WARN_MS) {
    delay(20);
  }
  leds[0] = CRGB::Black;
  FastLED.show();
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(40);
  leds[0] = CRGB::Black; FastLED.show();
  if (!mpu.begin()) {
    Serial.println(F("MPU6050 nicht gefunden. Kabel pruefen."));
    while (true) {
      delay(100);
    }
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
  mpu.setGyroRange(MPU6050_RANGE_250_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  delay(1000);  // Sensor kurz beruhigen
  waitForStart();
}
void loop() {
  mpu.getEvent(&accel, &gyro, &temp);
  // Drehrate hoechstens alle 200 ms als Grad pro Sekunde zeigen.
  unsigned long now = millis();
  if (now - lastRatePrint >= 200) {
    lastRatePrint = now;
    Serial.print(F("Drehrate "));
    Serial.print(gyro.gyro.x * GYRO_RAD_TO_DEG, 1);
    Serial.print(F(", "));
    Serial.print(gyro.gyro.y * GYRO_RAD_TO_DEG, 1);
    Serial.print(F(", "));
    Serial.print(gyro.gyro.z * GYRO_RAD_TO_DEG, 1);
    Serial.println(F(" deg/s"));
  }
  float ax = accel.acceleration.x;
  float ay = accel.acceleration.y;
  float az = accel.acceleration.z;
  // TODO: Laenge des Vektors berechnen:
  // total = sqrt(ax * ax + ay * ay + az * az)
  float total = 0.0;
  // TODO: total mit SHAKE_LIMIT vergleichen und bei einem Stoss
  // warnTwoSeconds(total) aufrufen.
  delay(50);
}
```

Ergänze die fehlenden Zeilen.

Lösung Mission Stoßalarm

Mission Stoßalarm 076_shake_solution.ino

```ino
// Erkennt einen Stoss und warnt 2 Sekunden mit LED und Serial.
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_LED = 4;     // WS2812
const float SHAKE_LIMIT = 14.0;  // Schwelle in m/s^2
const float GYRO_RAD_TO_DEG = 57.2958;  // rad/s in Grad/s
const int WARN_MS = 2000;  // Dauer der Warnung
Adafruit_MPU6050 mpu;
sensors_event_t accel, gyro, temp;
CRGB leds[1];
unsigned long lastRatePrint = 0;  // Zeit der letzten Ausgabe

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
```

Fortsetzung 076_shake_solution.ino

```
  }
  Serial.println(F("Los!"));
}
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void warnTwoSeconds(float total) {
  Serial.print(F("Stoss erkannt: "));
  Serial.println(total, 1);
  leds[0] = CRGB::Red;
  FastLED.show();
  unsigned long start = millis();
  while (millis() - start < WARN_MS) {
    delay(20);
  }
  leds[0] = CRGB::Black;
  FastLED.show();
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(40);
  leds[0] = CRGB::Black; FastLED.show();
  if (!mpu.begin()) {
    Serial.println(F("MPU6050 nicht gefunden. Kabel pruefen."));
    while (true) {
      delay(100);
    }
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
  mpu.setGyroRange(MPU6050_RANGE_250_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  delay(1000);  // Sensor kurz beruhigen
  waitForStart();
}
void loop() {
  mpu.getEvent(&accel, &gyro, &temp);
  // Drehrate hoechstens alle 200 ms als Grad pro Sekunde zeigen.
  unsigned long now = millis();
  if (now - lastRatePrint ⩾ 200) {
    lastRatePrint = now;
    Serial.print(F("Drehrate "));
    Serial.print(gyro.gyro.x * GYRO_RAD_TO_DEG, 1);
    Serial.print(F(", "));
    Serial.print(gyro.gyro.y * GYRO_RAD_TO_DEG, 1);
    Serial.print(F(", "));
```

Fortsetzung 076_shake_solution.ino

```
92      Serial.print(gyro.gyro.z * GYRO_RAD_TO_DEG, 1);
93      Serial.println(F(" deg/s"));
94    }
95    float ax = accel.acceleration.x;
96    float ay = accel.acceleration.y;
97    float az = accel.acceleration.z;
98    float total = sqrt(ax * ax + ay * ay + az * az);
99    if (total > SHAKE_LIMIT) {
100     warnTwoSeconds(total);
101     delay(300);  // kurz warten, damit der Stoss abklingt
102   }
103   delay(50);
104 }
```

Die Schwelle steht in Zeile 13. Zeile 49 bis 60 zeigen die Warnung: rote LED, Meldung auf dem seriellen Monitor, zwei Sekunden warten, LED aus. Die Zeit misst das Programm mit `millis()`. Zeile 99 berechnet die Länge, Zeile 100 vergleicht sie mit der Schwelle.

Die LED auf dem Shield ist ein WS2812. Sie braucht die Bibliothek FastLED, die du im Lichtkapitel genauer kennenlernst.

Häufiger Fehler

So sieht es aus	Das hilft
Der Monitor zeigt: MPU6050 nicht gefunden.	Prüfe die Kabel. A4 muss an SDA liegen, A5 an SCL. Sitzt das Modul fest auf dem Shield?
Roll und Pitch springen wild.	Das USB-Kabel wackelt oder das Auto steht nicht ruhig. Lege das Kabel locker und starte neu.
Der Stoßalarm löst nie aus.	Die Schwelle ist zu hoch. Setze `SHAKE_LIMIT` auf 12 und klopfe leicht gegen das Gehäuse.
Die LED bleibt rot.	Das Programm hängt in der Warteschleife. Prüfe, ob `WARN_MS` eine ganze Zahl ist und ob `FastLED.show()` aufgerufen wird.

Merken

Der MPU6050 enthält einen Beschleunigungssensor und einen Gyrosensor. Er hängt an A4 (SDA) und A5 (SCL) und antwortet auf Adresse `0x68`. `getEvent()` liefert Beschleunigung, Drehung und Temperatur. Aus `ax`, `ay` und `az` berechnest du Roll und Pitch. Der Gyro driftet. Eine Drehung per Addition über die Zeit wird ungenau.

- [ ] Ich kann den MPU6050 anschließen und im seriellen Monitor auslesen.
- [ ] Ich weiß, was Beschleunigung und was Drehgeschwindigkeit ist.
- [ ] Ich kann Roll und Pitch in Grad berechnen.
- [ ] Ich kann einen Stoß erkennen und zwei Sekunden warnen.

7
Licht und Fernbedienung

Ziel

Du bringst die RGB-LED auf dem Shield zum Leuchten. Du liest die Befehle deiner Fernbedienung aus und fährst das Auto mit den Tasten.

Mission

Baue ein Auto, das auf einen Tastendruck fährt, mit `OK` anhält und dabei eine farbige LED leuchten lässt. Ohne neuen Befehl stoppt es nach einer Sekunde von selbst.

Drei Farben in einer LED

Die LED auf dem Shield heißt WS2812. In ihr stecken drei winzige Leuchtdioden: rot, grün und blau. Aus diesen drei Farben mischt sie alle anderen. Der Datenpin ist D4.

Die drei Zahlen in `CRGB(255, 120, 0)` stehen für Rot, Grün und Blau. Jede Zahl geht von 0 bis 255. 255 ist die volle Helligkeit. Zusammen ergibt das Orange. `CRGB::Red` ist eine Abkürzung für `CRGB(255, 0, 0)`. Mit `FastLED.setBrightness(60)` dimmst du alle Farben auf 60 von 255.

Vier Farben und einmal aus `081_led_colors_solution.ino`

```
// Eine RGB-LED an D4: Rot, Gruen, Blau, Orange, Aus.
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_LED = 4;  // WS2812-LED
CRGB leds[1];
CRGB colors[] = {CRGB::Red, CRGB::Green, CRGB::Blue,
                 CRGB(255, 120, 0), CRGB::Black};
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void setup() {
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
}
void loop() {
  for (uint8_t i = 0; i < 5; i++) {
    leds[0] = colors[i];
    FastLED.show();
    delay(500);
  }
  delay(1000);
}
```

Zeile 7 legt den Speicher für eine LED an. Zeile 8 und 9 bauen die Farbliste: Rot, Grün, Blau, ein eigenes Orange und Schwarz. Schwarz bedeutet aus. Die Schleife in Zeile 24 bis 28 zeigt jede Farbe 500 Millisekunden lang. `FastLED.show()` schickt die Farbe erst an die LED. Ohne diesen Aufruf bleibt sie dunkel.

> **Probiere**
>
> Ändere `FastLED.setBrightness(60)` auf `20`. Die LED wird deutlich dunkler. Ändere dann `CRGB(255, 120, 0)` auf `CRGB(0, 255, 120)`. Welche Farbe kommt heraus?

Die Fernbedienung auslesen

Jede Taste sendet einen Rahmen. Im Karton liegen zwei Revisionen. Darum liest du zuerst selbst aus, was deine Fernbedienung sendet. Der Empfänger sitzt auf D9.

`IrReceiver.begin(PIN_IR, DISABLE_LED_FEEDBACK)` schaltet die Rückmeldung auf D13 ab. D13 ist der Trigger des Ultraschallsensors. Mit eingeschalteter Rückmeldung blinkt dort eine LED und der Abstand wird falsch. Das zweite Argument ist deshalb wichtig.

`decodedIRData` ist eine kleine Tabelle mit mehreren Feldern:

- `protocol` sagt, welches Übertragungsformat die Taste benutzt.
- `address` ist die Adresse des Geräts.
- `command` ist die eigentliche Tastennummer. Sie passt in ein `uint8_t`, also in eine Zahl von 0 bis 255.
- `decodedRawData` ist der ganze 32-Bit-Rahmen. Bei NEC steht er in IRremote 4.x anders herum als in alten Beispielen. Zum Steuern brauchst du ihn nicht.

Zum Vergleichen nimmst du `command`. Er ist kurz und für alle Tasten einer Fernbedienung eindeutig. `decodedRawData` hilft nur, wenn du einen Wert aus einer alten Anleitung wiedererkennen willst.

Was `0x` bedeutet, steht in Kapitel 6. Kurz: `0x62` ist die Hex-Schreibweise für 98.

Protokoll, Adresse und Befehl auslesen 086_ir_learn_solution.ino

```
// Lern-Sketch: zeigt Protokoll, Adresse, Befehl und Rohwert.
#include <IRremote.hpp>

const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_IR = 9;  // IR-Empfaenger

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  // DISABLE_LED_FEEDBACK laesst D13 frei.
  IrReceiver.begin(PIN_IR, DISABLE_LED_FEEDBACK);
  Serial.println(F("Druecke eine Taste der Fernbedienung."));
}

void loop() {
  if (IrReceiver.decode()) {
    Serial.print(F("Protokoll "));
    Serial.print((uint8_t)IrReceiver.decodedIRData.protocol);
    Serial.print(F(" Adresse 0x"));
    Serial.print(IrReceiver.decodedIRData.address, HEX);
    Serial.print(F(" Befehl 0x"));
    Serial.print(IrReceiver.decodedIRData.command, HEX);
    Serial.print(F(" Roh 0x"));
    Serial.println(IrReceiver.decodedIRData.decodedRawData, HEX);
    IrReceiver.resume();
  }
}
```

Der Sketch gibt pro Tastendruck eine Zeile aus, zum Beispiel `Protokoll 1 Adresse 0x0 Befehl 0x62 Roh 0xB946FF00`. `Protokoll 1` ist NEC. `Befehl 0x62` gehört zur Taste „hoch“. Nach jeder Zeile ruft der Sketch `resume()` auf. Damit ist der Empfänger für die nächste Taste frei.

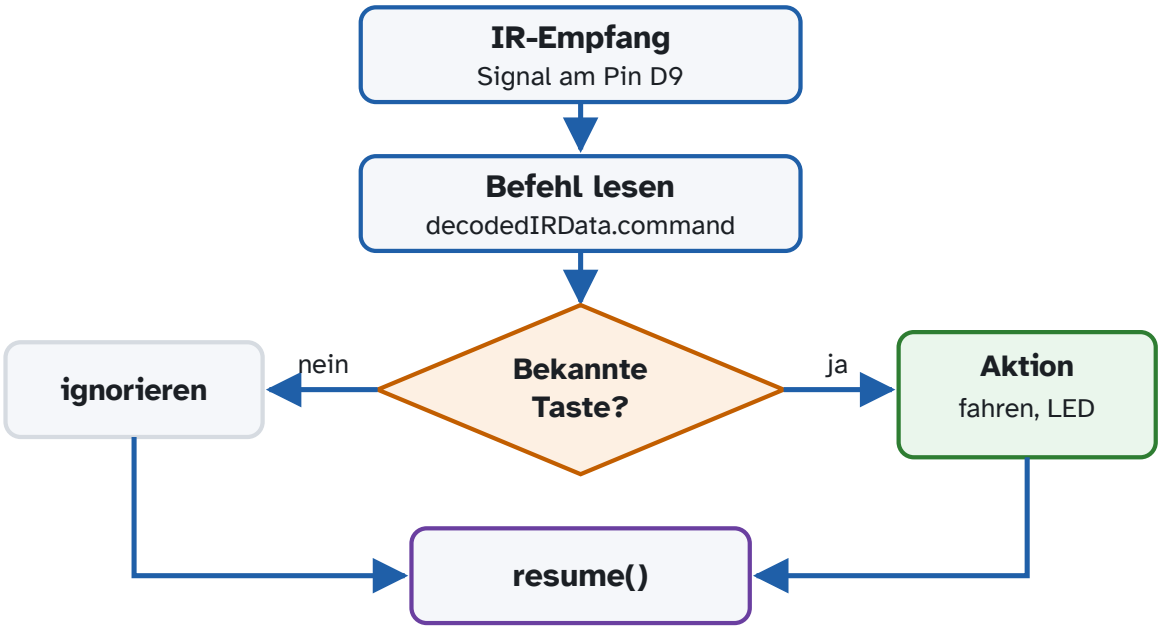
> **Probiere**
>
> Drücke alle 14 Tasten der Reihe nach. Schreibe jede Zeile auf ein Blatt. Manche Tasten senden beim Halten denselben Befehl mehrfach. Notiere nur den ersten Wert.

Mission: Fahren per Fernbedienung

Jetzt verbindest du die Befehle mit den Motoren. Der Sketch prüft `decodedIRData.command` in einer `switch`-Anweisung. Die fünf `CMD_...`-Konstanten stehen oben an einer einzigen Stelle. Für eine andere Fernbedienung änderst du nur dort die Zahlen.

`OK` ruft `stopMotors()` auf. Die anderen Tasten rufen `drive()` mit 160 von 255 auf. Das Start-Gate bleibt Pflicht: Erst drückst du die Taste auf dem Shield, dann läuft der Countdown, dann gehorcht das Auto der Fernbedienung. Vorher bewegt sich nichts.

Der Sketch merkt sich die Zeit des letzten gültigen Befehls. Kommt eine Sekunde lang kein neuer Befehl, hält er an. Das schützt dich, wenn die Batterie leer ist, du hinter ein Hindernis greifst oder die Verbindung abreißt. Das Auto fährt nie endlos weiter.



command ist die Tastennummer (uint8_t).
decodedRawData ist der ganze 32-Bit-Rahmen.

Abbildung 7.1: So wertet der Sketch eine Taste aus. Nur ein bekannter Befehl löst eine Aktion aus. Danach gibt `resume()` den Empfänger wieder frei.

Abbildung 7.1 zeigt den Weg vom Signal bis zur Aktion. Ein unbekannter Befehl wird verworfen. Das ist wichtig, weil manche Fernbedienungen beim Halten denselben Befehl mehrfach senden.

Deine Aufgabe

Öffne die Übungsdatei. Sie ist vollständig und baut sofort. Ergänze in `handleCode` die fünf Fälle. Nimm `drive(SPEED, SPEED)` für vorwärts, negative Werte für rückwärts und `stopMotors()` für `OK`. Die Konstanten `CMD_...` und der Watchdog sind schon fertig.

Dein Code 091_ir_drive_exercise.ino

```
// Vollstaendiger Sketch. Ergaenze in handleCode die fuenf Faelle.
#include <IRremote.hpp>

const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;  // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;  // linker Motor: Richtung
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_IR = 9;  // IR-Empfaenger

const int SPEED = 160;  // festes Tempo, 0..255
const unsigned long REMOTE_TIMEOUT_MS = 1000;  // Stopp ohne Befehl

// --- Fernbedienung: Befehle der Familie A (Elegoo) -------------------
// Andere Fernbedienung (Familie B)? Mit 086 messen und die Zahlen hier
// eintragen. Es gibt nur diese eine Stelle im Sketch.
const uint8_t CMD_UP = 0x62;     // hoch
const uint8_t CMD_DOWN = 0xA8;   // runter
const uint8_t CMD_LEFT = 0x22;   // links
const uint8_t CMD_RIGHT = 0xC2;  // rechts
const uint8_t CMD_OK = 0x02;     // anhalten

bool driving = false;  // laufen die Motoren gerade?
unsigned long lastCommand = 0;  // Zeit des letzten gueltigen Befehls

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
```

Fortsetzung 091_ir_drive_exercise.ino

```
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void handleCode(uint8_t command) {
  switch (command) {
    case CMD_UP: /* TODO: drive(SPEED, SPEED); */ break;
    case CMD_DOWN: /* TODO: drive(-SPEED, -SPEED); */ break;
    case CMD_LEFT: /* TODO: drive(-SPEED, SPEED); */ break;
    case CMD_RIGHT: /* TODO: drive(SPEED, -SPEED); */ break;
    case CMD_OK: /* TODO: stopMotors(); */ break;
    default: return;  // unbekannte Taste ignorieren
  }
  driving = (command != CMD_OK);
  lastCommand = millis();
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  // DISABLE_LED_FEEDBACK laesst D13 frei.
  IrReceiver.begin(PIN_IR, DISABLE_LED_FEEDBACK);
  waitForStart();
}

void loop() {
  if (IrReceiver.decode()) {
    uint8_t command = IrReceiver.decodedIRData.command;
    IrReceiver.resume();
    handleCode(command);
  }
  // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
  if (driving && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
    stopMotors();
    driving = false;
    Serial.println(F("Stopp: kein IR-Befehl."));
```

Fortsetzung 091_ir_drive_exercise.ino

```
  }
  // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
  if (driving && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
    stopMotors();
    driving = false;
    Serial.println(F("Stopp: kein IR-Befehl."));
  }
}
```

Ergänze die fehlenden Zeilen.

Lösung Mission Fernbedienung

Mission Fernbedienung 091_ir_drive_solution.ino

```
// Fahren mit der IR-Fernbedienung der Familie A (Elegoo).
#include <IRremote.hpp>

const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;  // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;  // linker Motor: Richtung
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_IR = 9;  // IR-Empfaenger

const int SPEED = 160;  // festes Tempo, 0..255
const unsigned long REMOTE_TIMEOUT_MS = 1000;  // Stopp ohne Befehl

// --- Fernbedienung: Befehle der Familie A (Elegoo) -------------------
// Andere Fernbedienung (Familie B)? Mit 086 messen und die Zahlen hier
// eintragen. Es gibt nur diese eine Stelle im Sketch.
const uint8_t CMD_UP = 0x62;     // hoch
const uint8_t CMD_DOWN = 0xA8;   // runter
const uint8_t CMD_LEFT = 0x22;   // links
const uint8_t CMD_RIGHT = 0xC2;  // rechts
const uint8_t CMD_OK = 0x02;     // anhalten

bool driving = false;  // laufen die Motoren gerade?
unsigned long lastCommand = 0;  // Zeit des letzten gueltigen Befehls

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
```

Fortsetzung 091_ir_drive_solution.ino

```
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void handleCode(uint8_t command) {
  switch (command) {
    case CMD_UP: drive(SPEED, SPEED); break;
    case CMD_DOWN: drive(-SPEED, -SPEED); break;
    case CMD_LEFT: drive(-SPEED, SPEED); break;
    case CMD_RIGHT: drive(SPEED, -SPEED); break;
    case CMD_OK: stopMotors(); break;
    default: return;  // unbekannte Taste ignorieren
  }
  driving = (command != CMD_OK);
  lastCommand = millis();
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  // DISABLE_LED_FEEDBACK laesst D13 frei.
  IrReceiver.begin(PIN_IR, DISABLE_LED_FEEDBACK);
  waitForStart();
```

Fortsetzung 091_ir_drive_solution.ino

```
}

void loop() {
  if (IrReceiver.decode()) {
    uint8_t command = IrReceiver.decodedIRData.command;
    IrReceiver.resume();
    handleCode(command);
  }
  // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
  if (driving && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
    stopMotors();
    driving = false;
    Serial.println(F("Stopp: kein IR-Befehl."));
  }
}
```

Befehlstabelle der Fernbedienung Familie A

Diese Werte stehen in `decodedIRData.command`. Prüfe sie mit dem Auslese-Sketch. Andere Fernbedienungen (Familie B) senden andere Zahlen. Miss sie mit 086 und trage sie im Sketch an der einen markierten Stelle ein. Das `0x` gehört vor jede Zahl.

Taste	command	Aktion
hoch	0x62	vorwärts
runter	0xA8	rückwärts
links	0x22	links drehen
rechts	0xC2	rechts drehen
OK	0x02	anhalten
1	0x68	noch frei
2	0x98	noch frei
3	0xB0	noch frei
4	0x30	noch frei
5	0x18	noch frei
6	0x7A	noch frei
7	0x10	noch frei
8	0x38	noch frei
9	0x5A	noch frei

Häufiger Fehler

So sieht es aus	Das hilft
Die LED bleibt dunkel.	Prüfe die Farbreihenfolge im `addLeds`-Aufruf. Sie muss `GRB` heißen. Mit `RGB` leuchten die Farben falsch oder gar nicht.
Die Fernbedienung reagiert nicht.	Prüfe den Pin: Der Empfänger hängt an D9. Steht in `begin` eine andere Zahl, kommt nichts an. Außerdem muss dort `DISABLE_LED_FEEDBACK` stehen, sonst stört D13 den Ultraschall.

So sieht es aus	Das hilft
Das Auto fährt bei einem Tastendruck nicht.	Drücke zuerst die Taste auf dem Shield. Ohne das Start-Gate bleibt `waitForStart()` stehen und der Loop liest keine Befehle.
Nur manche Tasten tun etwas.	Deine Fernbedienung nutzt die andere Familie B. Lies die Befehle mit dem Auslese-Sketch 086 neu aus und ändere die `CMD_...`-Zahlen an der markierten Stelle im Sketch.
Das Auto hält nach einer Sekunde an, obwohl ich noch fahre.	Das ist der Watchdog. Halte die Richtungstaste gedrückt oder drücke sie erneut. Ohne neuen Befehl soll das Auto nicht weiterfahren.

Merken

- Eine WS2812 mischt Rot, Grün und Blau. Jede Zahl geht von 0 bis 255.
- `FastLED.show()` schickt die Farbe an die LED.
- Der IR-Empfänger sitzt auf D9. `begin(PIN_IR, DISABLE_LED_FEEDBACK)` hält D13 frei.
- `command` ist die Tastennummer, `decodedRawData` der ganze Rahmen.
- Die `CMD_...`-Zahlen stehen an einer einzigen Stelle im Sketch.
- Ohne neuen Befehl stoppt der Watchdog die Motoren nach einer Sekunde. `resume()` öffnet den Empfänger für die nächste Taste.

- [ ] Ich kann eine Farbe aus Rot, Grün und Blau mischen.
- [ ] Ich kann die Helligkeit der LED ändern.
- [ ] Ich lese Protokoll, Adresse, Befehl und Rohwert im Serial Monitor.
- [ ] Ich fahre das Auto mit der Fernbedienung und stoppe mit `OK`.
- [ ] Ich drücke vor jeder Fahrt das Start-Gate.
- [ ] Ich erkläre, warum das Auto ohne neuen Befehl anhält.

8
Kamera und WLAN

Ziel

Du verbindest dein Handy mit der Kamera des Autos. Du siehst das Bild im Browser und steuerst das Auto über das WLAN. Danach verstehst du, wie Handy, ESP32 und Uno miteinander reden.

Mission

Baue eine Fernsteuerung über das eigene WLAN und fahre das Auto per Tastendruck im Serial Monitor. Die Kamera zeigt dir dabei das Bild nach vorn.

Ein eigenes WLAN im Auto

Auf dem Kamera-Modul sitzt ein ESP32-WROVER. Er ist ein eigener kleiner Computer mit WLAN. Der ESP32 öffnet einen eigenen WLAN-Kreis. Dein Handy verbindet sich direkt damit, ohne Router und ohne Internet.

Der Name des WLANs beginnt mit `ELEGOO-`. Dahinter stehen 12 Hexzeichen aus der MAC-Adresse des Moduls. Jedes Auto hat einen anderen Namen, zum Beispiel `ELEGOO-A1B2C3D4E5F6`. Ein Passwort ist ab Werk nicht gesetzt. Der Kanal ist fest auf 9 gestellt.

Achtung

Das WLAN hat kein Passwort. Jeder in Reichweite kann es sehen und betreten. Nutze es nur in deinem Zimmer und lasse niemanden Fremdes mit dem Auto verbinden.

Mit dem Handy verbinden

Schalte das Auto ein. Warte, bis der ESP32 gestartet ist. Das dauert einige Sekunden. Öffne dann auf dem Handy die WLAN-Einstellungen. Suche den Namen `ELEGOO-...` und tippe darauf. Ein Passwort wird nicht verlangt.

Danach öffnest du die App `ElegooKit`. Sie gibt es für Android und für das iPhone. Alternativ öffnest du den Browser und rufst `http://192.168.4.1/` auf. Dort findest du die Steuerung. Das reine Bild zeigt `http://192.168.4.1/Test`. Das Standardbild ist 800 mal 600 Pixel groß.

Am Kamera-Modul leuchten zwei kleine LEDs. Ihre Bedeutung hilft beim Suchen:

Pin	Bedeutung
Rot blinkt	Die Verbindung ist schlecht. Prüfe das 4-Pin-Kabel und stecke es neu.
Grün blinkt	Der ESP32 läuft, aber das Handy ist noch nicht verbunden.
Grün leuchtet	Das Handy ist verbunden. Das Bild kann kommen.

Der Schalter Upload und Cam

Auf dem Shield sitzt ein kleiner Schalter mit zwei Stellungen: `Upload` und `Cam`. Er entscheidet, wer mit dem USB-Anschluss und der Leitung zum Uno spricht.

In Stellung `Upload` programmiert die Arduino IDE den Uno. Der USB- Anschluss gehört dann dem Uno. Jedes Mal, wenn du ein neues Programm hochlädst, muss der Schalter auf `Upload` stehen. Sonst meldet die IDE einen Fehler oder das Programm landet nicht auf dem Uno.

In Stellung `Cam` gehört die Leitung dem ESP32. Der ESP32 nimmt die Befehle vom Handy an und gibt sie an den Uno weiter. Für die Handy-Steuerung und das Kamerabild ist das die richtige Stellung.

Merke dir die Reihenfolge: erst `Upload` und Programm hochladen, dann `Cam` und Handy verbinden. Wenn du später ein neues Programm aufspielst, schaltest du wieder auf `Upload`.

Achtung

Stecke das Kamera-Modul nie ab, solange der Akku eingeschaltet ist. Schalte das Auto zuerst aus, dann ziehe das 4-Pin-Kabel. Sonst können Kamera und Shield Schaden nehmen.

Wie Uno und ESP32 reden

Der ESP32 und der Uno sind mit einer seriellen Leitung verbunden. Sie läuft mit 9600 Baud. Der ESP32 schickt kurze Textstücke im JSON-Format. Ein Befehl endet immer mit `}`.

`N` wählt die Funktion. `D1`, `D2` und `D3` sind Werte für diese Funktion. `H` ist eine laufende Nummer, die der ESP32 vergibt. Der Uno antwortet mit `{"ID_ok"}` oder einem Messwert.

Achtung

Diese JSON-Befehle gehören zur Werksfirmware auf ESP32 und Uno. Deine eigenen Sketches aus diesem Buch, zum Beispiel 096 und 131, lesen dieses JSON nicht. Die App steuert sie deshalb nicht automatisch. Für eigene Programme nutzt du den Serial Monitor oder die eigene ESP32-Firmware aus Kapitel 10.

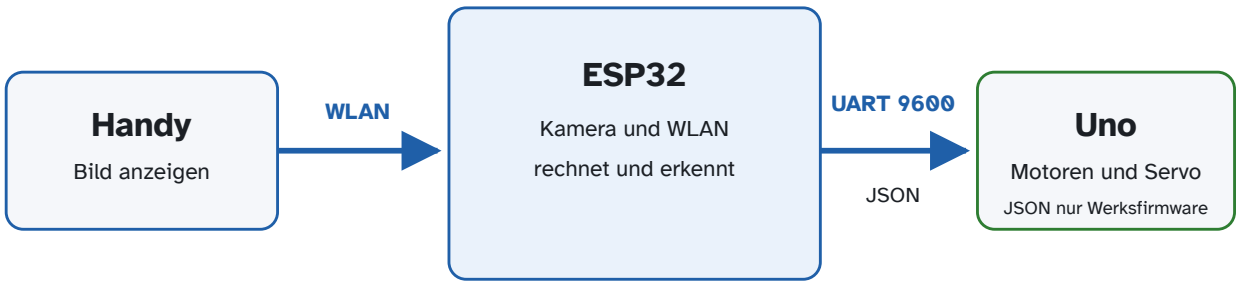
Drei Befehle aus der App

Befehl	Bedeutung
`{"N":100}`	Alles anhalten
`{"H":1,"N":1,"D1":0,"D2":128,"D3":1}`	Motor: `D1` 0 = beide, `D2` = 128 von 255, `D3` 1 = vorwärts
`{"H":3,"N":5,"D1":1,"D2":90}`	Servo: `D1` 1 = links/rechts, `D2` = 90 Grad

Die App schickt diese Befehle an die Werksfirmware. Du musst sie nicht selbst tippen. Für das Verständnis reicht: Der ESP32 übersetzt deinen Fingertipp in JSON, der Uno der Werksfirmware führt ihn aus. Ein eigener Sketch wie 096 ignoriert diese Zeilen.

Zwei Gehirne im Auto

Im Auto arbeiten zwei Computer. Der ESP32 sitzt an der Kamera. Er nimmt das Bild auf, komprimiert es und schickt es über WLAN an dein Handy. Auf Wunsch sucht er auch Gesichter. Der Uno steuert die Motoren und den Servo. Vom ESP32 bekommt er nur kurze Befehle.



Die App steuert nur die Werksfirmware.
Eigene Sketches 096 und 131 lesen kein JSON.

Der Weg vom Bild zum Befehl.

Der ESP32 rechnet, der Uno fährt. Sie tauschen kurze Textbefehle aus. Mehr braucht der Uno nicht. Er muss nicht wissen, wie das Bild aussieht. Er muss nur wissen, wohin er fahren soll.

> **Achtung**
>
> Kapitel 10 ersetzt die Werksfirmware des ESP32. Danach ist die Kamera-App aus diesem Kapitel nicht mehr unverändert nutzbar. Der Rückweg mit `esptool` ist im Buch beschrieben, aber noch nicht am Gerät getestet. Probiere ihn vorher mit einer erwachsenen Person aus und halte die Originaldatei bereit. Ohne getesteten Rückweg behandle Kapitel 10 als betreutes Zusatzprojekt.

Warum der Uno kein Bild liest

Der Uno hat keinen Anschluss für die Kamera. Das Kamerakabel endet am ESP32. Selbst mit einem Adapter würde es nicht reichen:

- Der Uno hat 2 KB RAM. Ein Bild passt nicht hinein.
- Sein Takt liegt bei 16 MHz. Für Bildverarbeitung ist das zu langsam.
- Die Leitung schafft 9600 Baud, etwa 1 KB pro Sekunde. Ein JPEG ist 10 bis 20 KB groß.

Ein Bild bräuchte also 10 bis 20 Sekunden. So lange kann das Auto nicht warten. Der ESP32 erledigt die Bildarbeit und schickt nur das Ergebnis.

Gesichtserkennung ausprobieren

Die Kamera-Firmware kann Gesichter finden. Du lädst dafür nichts nach. So probierst du es aus:

1. Verbinde dein Handy mit dem WLAN `ELEGOO-....`
2. Stelle die Bildgröße auf QVGA, also 320 mal 240 Pixel. Die Erkennung läuft nur bei einer Bildbreite bis 400 Pixel.
3. Öffne im Browser `http://192.168.4.1/control?var=face_detect&val=1`.
4. Öffne danach `http://192.168.4.1/Test`. Gesichter bekommen einen Rahmen im Bild.

Mit `val=0` schaltest du die Erkennung wieder aus. Willst du eine Person wiedererkennen, ersetze in der Adresse `face_detect` durch `face_recognize`. Danach rufst du die Adresse mit `face_enroll` auf. Halte das Gesicht fünf Mal in die Kamera. Mit `face_enroll` und `val=0` beendest du das Anlernen. Die gelernten Gesichter liegen nur im Arbeitsspeicher. Nach dem Ausschalten lernst du neu an.

Messaufgabe

Halte dein Gesicht in die Kamera und fülle die Tabelle aus. Probiere jede Entfernung bei hellem Licht und im Dunkeln.

Entfernung	Hell	Dunkel	Notizen
0,5 m	ja / nein	ja / nein	
1 m	ja / nein	ja / nein	
2 m	ja / nein	ja / nein	
3 m	ja / nein	ja / nein	

Follow nutzt den Ultraschall

In der App gibt es einen Modus `Follow`. Der Name klingt nach Bildverfolgung. Gemeint ist Abstand halten.

Im Uno-Code heißt die Funktion `ApplicationFunctionSet_Follow`. Sie liest den Ultraschallsensor, schwenkt den Servo nach links und rechts und fährt in die Richtung, in der es frei ist. Die Kamera kommt darin nicht vor. Die App schickt nur den Modusbefehl `N=101` mit `D1=3` und wählt damit den Modus aus.

Ausblick

Eine eigene ESP32-Firmware könnte den Mittelpunkt einer Farbe berechnen. Sie schickt dann `{"x":120,"w":40}` an den Uno. `x` ist die Mitte in Pixeln, `w` die Breite. Bei QVGA liegt die Bildmitte bei 160. Der Uno vergleicht `x` mit 160 und lenkt nach. In Kapitel 10 bauen wir genau das. In diesem Kapitel bleibt der Uno bei den Buchstaben.

Mission: Buchstaben statt JSON

Im Serial Monitor kannst du den Uno auch ohne ESP32 steuern. Öffne den Serial Monitor mit 9600 Baud. Tippe einen Buchstaben und drücke Enter:

- `v` fährt vorwärts
- `z` fährt rückwärts
- `l` dreht links
- `r` dreht rechts
- `s` hält an
- `q` macht langsamer, `e` schneller

Der Sketch liest mit `Serial.read()` genau ein Zeichen. Das Start-Gate bleibt Pflicht. Erst Taste auf dem Shield, dann Countdown, dann nimmt der Loop Buchstaben an.

`q` und `e` wirken sofort, auch während der Fahrt. Der Sketch merkt sich die aktuelle Richtung in `dirLeft` und `dirRight`. Nach einer Tempoänderung gibt er diese Richtung mit dem neuen Tempo erneut aus. Du musst also keine Richtungstaste nachdrücken.

Nach dem letzten gültigen Befehl läuft eine Uhr. Kommt eine Sekunde lang nichts, hält der Sketch an. So stoppt das Auto, wenn du das Kabel ziehst, und es fährt nicht endlos weiter, wenn die Verbindung abreist.

Fahren mit Buchstaben 096_serial_remote_solution.ino

```
// Fahre mit Buchstaben. q und e aendern das Tempo sofort.
const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;  // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;  // linker Motor: Richtung
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield

const int SPEED_STEP = 20;  // Schrittweite pro Tastendruck
const int SPEED_MIN = 60;  // kleinstes Tempo
const int SPEED_MAX = 220;  // groesstes Tempo
const unsigned long REMOTE_TIMEOUT_MS = 1000;  // Stopp ohne Befehl

int speed = 140;  // aktuelles Tempo
int8_t dirLeft = 0;   // -1 zurueck, 0 halt, 1 vor
int8_t dirRight = 0;  // -1 zurueck, 0 halt, 1 vor
unsigned long lastCommand = 0;  // Zeit des letzten gueltigen Befehls

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
// Gespeicherte Richtung mit dem aktuellen Tempo erneut ausgeben.
void applyDrive() {
  if (dirLeft == 0 && dirRight == 0) {
    stopMotors();
  } else {
    drive(dirLeft * speed, dirRight * speed);
  }
}
bool moving() {
  return dirLeft != 0 || dirRight != 0;
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
```

Fortsetzung 096_serial_remote_solution.ino

```
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void handleKey(char c) {
  if (c == 'v') {
    dirLeft = 1; dirRight = 1;
  } else if (c == 'z') {
    dirLeft = -1; dirRight = -1;
  } else if (c == 'l') {
    dirLeft = -1; dirRight = 1;
  } else if (c == 'r') {
    dirLeft = 1; dirRight = -1;
  } else if (c == 's') {
    dirLeft = 0; dirRight = 0;
  } else if (c == 'q') {
    if (speed > SPEED_MIN) speed -= SPEED_STEP;
  } else if (c == 'e') {
    if (speed < SPEED_MAX) speed += SPEED_STEP;
  } else {
    return;
  }
  // q und e wirken auch waehrend der Fahrt sofort.
  applyDrive();
  lastCommand = millis();
  Serial.print(F("Tempo: "));
  Serial.println(speed);
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_BUTTON, INPUT_PULLUP); pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  Serial.println(F("v vor, z zurueck, l links, r rechts, s stop"));
  Serial.println(F("q langsamer, e schneller"));
  waitForStart();
}
void loop() {
  int c = Serial.read();
  if (c > 0) handleKey((char)c);
  // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
  if (moving() && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
```

Fortsetzung 096_serial_remote_solution.ino

```
102    if (c > 0) handleKey((char)c);
103    // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
104    if (moving() && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
105      dirLeft = 0; dirRight = 0;
106      stopMotors();
107      Serial.println(F("Stopp: kein Befehl."));
108    }
109  }
```

`setup` richtet den Serial Monitor ein und wartet auf das Gate. `loop` liest ein Zeichen, ruft `handleKey` auf und prüft den Watchdog. `handleKey` speichert die Richtung und ruft `applyDrive` auf. Bei `l` bekommt der linke Motor einen negativen Wert und der rechte einen positiven. So dreht das Auto auf der Stelle. Ändert `q` oder `e` das Tempo, gibt `applyDrive` die gespeicherte Richtung sofort neu aus.

Probiere

Tippe `v` und warte zwei Sekunden. Tippe dann `s`. Das Auto hält an. Tippe `l` und danach `s`. Beobachte, in welche Richtung es sich dreht.

Aufgabe: Schneller und langsamer

Jetzt bekommt die Steuerung zwei neue Tasten. `q` macht das Auto um 20 Stufen langsamer, `e` um 20 Stufen schneller. Das Tempo bleibt zwischen 60 und 220. Außerhalb dieser Grenzen passiert nichts.

Deine Aufgabe

Öffne die Übungsdatei. Sie ist vollständig und baut sofort. Ergänze in `handleKey` die beiden Fälle für `q` und `e`. Prüfe mit `speed > SPEED_MIN` und `speed < SPEED_MAX`, ob die Änderung erlaubt ist. Der Watchdog und die gespeicherte Richtung sind schon fertig.

Dein Code 096_serial_remote_exercise.ino

```
// Vollstaendiger Sketch. Ergaenze in handleKey die Faelle q und e.
const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;  // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;  // linker Motor: Richtung
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield

const int SPEED_STEP = 20;  // Schrittweite pro Tastendruck
const int SPEED_MIN = 60;  // kleinstes Tempo
const int SPEED_MAX = 220;  // groesstes Tempo
const unsigned long REMOTE_TIMEOUT_MS = 1000;  // Stopp ohne Befehl

int speed = 140;  // aktuelles Tempo
int8_t dirLeft = 0;   // -1 zurueck, 0 halt, 1 vor
int8_t dirRight = 0;  // -1 zurueck, 0 halt, 1 vor
unsigned long lastCommand = 0;  // Zeit des letzten gueltigen Befehls

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
// Gespeicherte Richtung mit dem aktuellen Tempo erneut ausgeben.
void applyDrive() {
  if (dirLeft == 0 && dirRight == 0) {
    stopMotors();
  } else {
    drive(dirLeft * speed, dirRight * speed);
  }
}
bool moving() {
  return dirLeft != 0 || dirRight != 0;
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
```

Fortsetzung 096_serial_remote_exercise.ino

```
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void handleKey(char c) {
  if (c == 'v') {
    dirLeft = 1; dirRight = 1;
  } else if (c == 'z') {
    dirLeft = -1; dirRight = -1;
  } else if (c == 'l') {
    dirLeft = -1; dirRight = 1;
  } else if (c == 'r') {
    dirLeft = 1; dirRight = -1;
  } else if (c == 's') {
    dirLeft = 0; dirRight = 0;
  } else if (c == 'q') {
    // TODO: speed um SPEED_STEP verkleinern, aber nicht unter SPEED_MIN.
  } else if (c == 'e') {
    // TODO: speed um SPEED_STEP vergroessern, aber nicht ueber SPEED_MAX.
  } else {
    return;
  }
  // q und e wirken auch waehrend der Fahrt sofort.
  applyDrive();
  lastCommand = millis();
  Serial.print(F("Tempo: "));
  Serial.println(speed);
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_BUTTON, INPUT_PULLUP); pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  Serial.println(F("v vor, z zurueck, l links, r rechts, s stop"));
  Serial.println(F("q langsamer, e schneller"));
  waitForStart();
}
void loop() {
  int c = Serial.read();
  if (c > 0) handleKey((char)c);
  // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
  if (moving() && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
```

Fortsetzung 096_serial_remote_exercise.ino

```
102    if (c > 0) handleKey((char)c);
103    // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
104    if (moving() && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
105      dirLeft = 0; dirRight = 0;
106      stopMotors();
107      Serial.println(F("Stopp: kein Befehl."));
108    }
109  }
```

Ergänze die fehlenden Zeilen.

Lösung Mission Tempo

Mission Tempo 096_serial_remote_solution.ino

```ino
// Fahre mit Buchstaben. q und e aendern das Tempo sofort.
const uint8_t PIN_PWMA = 5;  // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;  // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;  // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;  // linker Motor: Richtung
const uint8_t PIN_STBY = 3;  // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield

const int SPEED_STEP = 20;  // Schrittweite pro Tastendruck
const int SPEED_MIN = 60;  // kleinstes Tempo
const int SPEED_MAX = 220;  // groesstes Tempo
const unsigned long REMOTE_TIMEOUT_MS = 1000;  // Stopp ohne Befehl

int speed = 140;  // aktuelles Tempo
int8_t dirLeft = 0;   // -1 zurueck, 0 halt, 1 vor
int8_t dirRight = 0;  // -1 zurueck, 0 halt, 1 vor
unsigned long lastCommand = 0;  // Zeit des letzten gueltigen Befehls

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
// Gespeicherte Richtung mit dem aktuellen Tempo erneut ausgeben.
void applyDrive() {
  if (dirLeft == 0 && dirRight == 0) {
    stopMotors();
  } else {
    drive(dirLeft * speed, dirRight * speed);
  }
```

Fortsetzung 096_serial_remote_solution.ino

```
}
bool moving() {
  return dirLeft != 0 || dirRight != 0;
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void handleKey(char c) {
  if (c == 'v') {
    dirLeft = 1; dirRight = 1;
  } else if (c == 'z') {
    dirLeft = -1; dirRight = -1;
  } else if (c == 'l') {
    dirLeft = -1; dirRight = 1;
  } else if (c == 'r') {
    dirLeft = 1; dirRight = -1;
  } else if (c == 's') {
    dirLeft = 0; dirRight = 0;
  } else if (c == 'q') {
    if (speed > SPEED_MIN) speed -= SPEED_STEP;
  } else if (c == 'e') {
    if (speed < SPEED_MAX) speed += SPEED_STEP;
  } else {
    return;
  }
  // q und e wirken auch waehrend der Fahrt sofort.
  applyDrive();
  lastCommand = millis();
  Serial.print(F("Tempo: "));
  Serial.println(speed);
}
void setup() {
  Serial.begin(9600);
```

Fortsetzung 096_serial_remote_solution.ino

```
92    pinMode(PIN_BUTTON, INPUT_PULLUP); pinMode(PIN_STBY, OUTPUT);
93    pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
94    pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
95    stopMotors();
96    Serial.println(F("v vor, z zurueck, l links, r rechts, s stop"));
97    Serial.println(F("q langsamer, e schneller"));
98    waitForStart();
99  }
100 void loop() {
101   int c = Serial.read();
102   if (c > 0) handleKey((char)c);
103   // Watchdog: ohne neuen Befehl nach kurzer Zeit anhalten.
104   if (moving() && millis() - lastCommand > REMOTE_TIMEOUT_MS) {
105     dirLeft = 0; dirRight = 0;
106     stopMotors();
107     Serial.println(F("Stopp: kein Befehl."));
108   }
109 }
```

Achtung

Datenschutz: Das WLAN ist offen und reicht bis in den Garten. Lass das Auto nicht unbeaufsichtigt mit eingeschaltetem WLAN stehen. Verbinde nur dein eigenes Handy. Schalte das Auto aus, wenn du nicht fährst.

Häufiger Fehler

So sieht es aus	Das hilft
Kein WLAN `ELEGOO-…` zu sehen.	Warte 10 Sekunden nach dem Einschalten. Prüfe den Akku und das 4-Pin-Kabel. Leuchtet keine LED am Modul, startet der ESP32 nicht.
Das Bild bleibt schwarz.	Prüfe den Schalter: Für die Handy-Steuerung muss er auf `Cam` stehen. Stecke das 4-Pin-Kabel bei ausgeschaltetem Auto neu ein.
Die App findet das Auto, aber es fährt nicht.	Die App steuert nur die Werksfirmware. Mit 096 oder 131 im Uno ignoriert der Sketch das JSON; nutze dann den Serial Monitor. Mit Werksfirmware: Drücke die Taste auf dem Shield und prüfe, ob der Schalter auf `Cam` steht.
Der Stream öffnet sich nicht.	Tippe die Adresse genau ein: `http://192.168.4.1/Test`. Achte auf `http` und die Punkte. Das Handy muss im WLAN `ELEGOO-…` sein.

Merken

- Der ESP32 öffnet ein eigenes WLAN `ELEGOO-` plus 12 Hexzeichen.
- `Upload` ist für Uno-Programme, `Cam` für Handy und Kamera.

> - Der ESP32 rechnet und erkennt, der Uno fährt.
> - Der Uno kann kein Bild lesen: 2 KB RAM und 9600 Baud reichen nicht.
> - `face_detect=1` schaltet die Gesichtserkennung ein.
> - `Follow` nutzt Ultraschall und Servo, nicht die Kamera.
> - Die App steuert nur die Werksfirmware, nicht eigene Sketches.
> - Ohne neuen Befehl stoppt 096 die Motoren nach einer Sekunde.
> - `q` und `e` wirken sofort, weil die Richtung gespeichert ist.

- [ ] Ich verbinde mein Handy mit dem WLAN des Autos.
- [ ] Ich öffne das Kamerabild im Browser oder in der App.
- [ ] Ich erkläre den Unterschied zwischen `Upload` und `Cam`.
- [ ] Ich fahre das Auto mit `v`, `z`, `l`, `r` und `s`.
- [ ] Ich ändere das Tempo mit `q` und `e`, auch während der Fahrt.
- [ ] Ich erkläre, dass die App nur die Werksfirmware steuert.
- [ ] Ich erkläre den Watchdog nach dem letzten Befehl.
- [ ] Ich erkläre, warum der Uno kein Kamerabild verarbeitet.
- [ ] Ich schalte die Gesichtserkennung im Browser ein.
- [ ] Ich schalte den Akku aus, bevor ich das Kamerakabel anfasse.

9 Grossprojekt Rettungsrover

Ziel

Du verbindest Linienfolger, Ultraschall, Servo und LED in einem Programm. Du baust eine Zustandsmaschine mit fünf Zuständen. Du lässt den Rover Hindernissen ausweichen und nach drei Querlinien anhalten.

Mission

Ein Rover, der einer Linie folgt, Hindernissen ausweicht und nach drei Querlinien anhält.

Jedes Kapitel bis hierher hatte eine Aufgabe. Dieses verbindet sie. Der Rover fährt, sieht, dreht den Kopf und leuchtet. Du schreibst dafür ein einziges Programm.

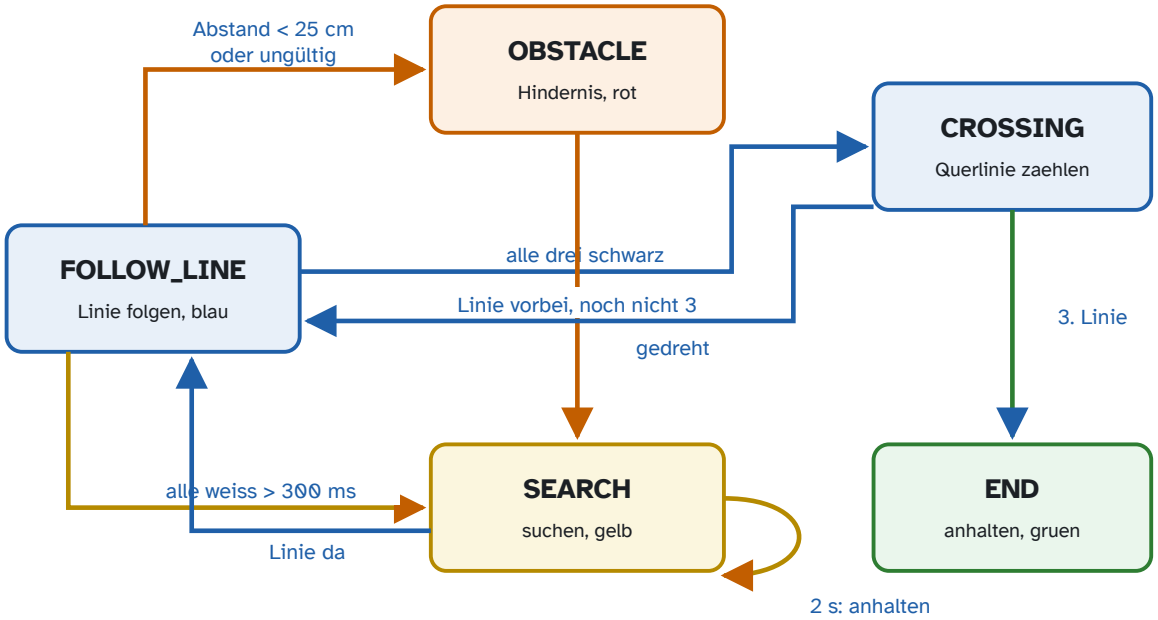


Abbildung 9.1: Fünf Zustände. Die Pfeile tragen die Bedingung für den Wechsel.

Fünf Zustände statt vieler if-Blöcke

Bei einer Kreuzung muss das Auto wissen, was es gerade tut. Ein Linienfolger fährt, ein Hindernis warnt, eine Querlinie zählt. Mit vielen `if`-Blöcken verliert man den Überblick. Eine Zustandsmaschine merkt sich einen Zustand in einer Variablen. `loop` fragt nur: Welcher Zustand ist aktiv? Dann ruft sie genau die passende Funktion auf. Jede Funktion darf den Zustand wechseln. Der Zustand ist wie ein Gang beim Fahrrad: Du fährst immer nur in einem, und du schaltest bewusst.

Die fünf Zustände

Zustand	LED	Aufgabe
`FOLLOW_LINE`	blau	Linie folgen, Abstand messen
`OBSTACLE`	rot	anhalten, zurücksetzen, Kopf scannen

Zustand	LED	Aufgabe
`SEARCH`	gelb	im Bogen nach der Linie suchen
`CROSSING`	blau	über die Querlinie fahren und zählen
`END`	grün	anhalten, Kopf auf 90 Grad

Das vollständige Programm

Rettungsrover 131_rescue_rover_solution.ino

```
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA=5, PIN_PWMB=6, PIN_AIN1=7, PIN_BIN1=8; // Motor
const uint8_t PIN_STBY=3, PIN_BUTTON=2;  // Treiber, Taste
const uint8_t PIN_TRIG=13, PIN_ECHO=12;  // Ultraschall
const uint8_t PIN_SERVO=10, PIN_LED=4;   // Kopf, LED
const uint8_t PIN_LINE_LEFT=A2, PIN_LINE_MIDDLE=A1;  // Linien-Sensoren
const uint8_t PIN_LINE_RIGHT=A0;  // Linien-Sensor rechts
enum State { FOLLOW_LINE, OBSTACLE, SEARCH, CROSSING, END };
State state = FOLLOW_LINE;
unsigned long stateSince = 0, whiteSince = 0;  // Zustandszeiten
uint8_t crossings = 0;  // Anzahl Querlinien
unsigned long lastProgress = 0;  // letzter Fortschritt
const char* const STATE_NAMES[] = {"FOLLOW", "OBSTACLE", "SEARCH",
                                   "CROSSING", "END"};
const CRGB STATE_LED[] = {CRGB::Blue, CRGB::Red, CRGB::Yellow,
                          CRGB::Blue, CRGB::Green};
const int LINE_THRESHOLD=600, BASE_SPEED=110, STEER_GAIN=90;  // Regelwerte
const int OBSTACLE_CM=25, SEARCH_MS=2000, WHITE_MS=300;  // Abstand/Zeiten
const int SCAN_LEFT=20, SCAN_RIGHT=150, SCAN_WAIT=400;  // Scan
const unsigned long RESCUE_MAX_MS = 30000;  // ohne Fortschritt: Stopp
CRGB leds[1]; Servo headServo;
void stopMotors() {
  analogWrite(PIN_PWMA,0); analogWrite(PIN_PWMB,0);
  digitalWrite(PIN_STBY,LOW);
}
void drive(int l, int r) {
  l = constrain(l, -255, 255); r = constrain(r, -255, 255);
  digitalWrite(PIN_BIN1, l >= 0); digitalWrite(PIN_AIN1, r >= 0);
  analogWrite(PIN_PWMB, abs(l)); analogWrite(PIN_PWMA, abs(r));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) != LOW) return false;
  delay(30); return digitalRead(PIN_BUTTON) == LOW;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste."));
  while (!buttonPressed()) delay(20);
  while (buttonPressed()) delay(20);
  for (int i = 3; i > 0; i--) { Serial.print(i); delay(1000); }
  Serial.println(F("Los!"));
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long t = pulseIn(PIN_ECHO, HIGH, 25000);
  if (t == 0) return -1;  // kein Echo: ungueltig, nicht "frei"
  return (int)(t / 58UL);
}
bool onBlack(uint8_t pin) { return analogRead(pin) < LINE_THRESHOLD; }
```

Fortsetzung 131_rescue_rover_solution.ino

```
53  void lookAt(int angle) {
54    headServo.write(angle); delay(SCAN_WAIT);
55  }
56  void setState(State next) {
57    state = next; stateSince = millis();
58    if (next == CROSSING) { crossings++; lastProgress = millis(); }
59    if (next == FOLLOW_LINE) whiteSince = 0;
60    leds[0] = STATE_LED[next]; FastLED.show();
61    Serial.println(STATE_NAMES[next]);
62  }
63  void followLine(unsigned long now) {
64    bool l=onBlack(PIN_LINE_LEFT), m=onBlack(PIN_LINE_MIDDLE);
65    bool r=onBlack(PIN_LINE_RIGHT);
66    if (l && m && r) { setState(CROSSING); return; }
67    if (!l && !m && !r) {
68      if (whiteSince == 0) whiteSince = now;
69      if (now - whiteSince > WHITE_MS) { setState(SEARCH); return; }
70    } else whiteSince = 0;
71    int cm = readDistanceCm();
72    if (cm < 0) { stopMotors(); setState(OBSTACLE); return; }
73    if (cm < OBSTACLE_CM) { setState(OBSTACLE); return; }
74    int e = l ? -1 : (r ? 1 : 0);
75    drive(BASE_SPEED + e * STEER_GAIN, BASE_SPEED - e * STEER_GAIN);
76  }
77  void avoidObstacle() {
78    stopMotors();
79    if (readDistanceCm() < 0) { setState(END); return; }
80    drive(-110, -110); delay(400); stopMotors();
81    lookAt(SCAN_LEFT); int l = readDistanceCm();
82    lookAt(90); int m = readDistanceCm();
83    lookAt(SCAN_RIGHT); int r = readDistanceCm(); lookAt(90);
84    if (l < 0 || m < 0 || r < 0) { setState(END); return; }
85    if (l > m && l ⩾ r) drive(-120, 120); else if (r > m) drive(120, -120);
86    delay(500); stopMotors(); setState(SEARCH);
87  }
88  void searchLine(unsigned long now) {
89    if (onBlack(PIN_LINE_LEFT)||onBlack(PIN_LINE_MIDDLE)||
90        onBlack(PIN_LINE_RIGHT)) { setState(FOLLOW_LINE); return; }
91    if (now - stateSince > SEARCH_MS) { stopMotors(); return; }
92    drive(70, 30);
93  }
94  void crossLine(unsigned long now) {
95    if (now - stateSince < 250) { drive(90, 90); return; }
96    if (onBlack(PIN_LINE_LEFT)&&onBlack(PIN_LINE_MIDDLE)&&
97        onBlack(PIN_LINE_RIGHT)) return;
98    if (crossings ⩾ 3) { stopMotors(); lookAt(90); setState(END); }
99    else setState(FOLLOW_LINE);
100 }
101 void setup() {
102   Serial.begin(9600); pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
103   pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
104   pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
```

Fortsetzung 131_rescue_rover_solution.ino

```
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_LINE_LEFT, INPUT); pinMode(PIN_LINE_MIDDLE, INPUT);
  pinMode(PIN_LINE_RIGHT, INPUT);
  headServo.attach(PIN_SERVO); stopMotors(); lookAt(90);
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60); leds[0]=CRGB::Black; FastLED.show();
  lastProgress = millis(); waitForStart(); setState(FOLLOW_LINE);
}
void loop() {
  unsigned long now = millis();
  if (state != END && now - lastProgress > RESCUE_MAX_MS) {
    stopMotors(); setState(END); return;
  }
  switch (state) {
    case FOLLOW_LINE: followLine(now); break;
    case OBSTACLE: avoidObstacle(); break;
    case SEARCH: searchLine(now); break;
    case CROSSING: crossLine(now); break;
    case END: stopMotors(); break;
  }
}
```

Das `enum` legt die fünf Zustände als Namen für feste Zahlen an. `FOLLOW_LINE` ist damit `0`, `OBSTACLE` ist 1, und so weiter. Die Variable `state` merkt sich den aktuellen Zustand. Das `switch` in `loop` wählt danach die passende Funktion.

`setState` macht drei Dinge: Es speichert den neuen Zustand, es merkt sich die Startzeit in `stateSince`, und es setzt die LED-Farbe aus der Liste `STATE_LED`. Die Namen für den Serial Monitor stehen in `STATE_NAMES`. Jeder Wechsel wird ausgegeben, zum Beispiel `OBSTACLE` oder `END`.

In `followLine` liest der Rover die drei Liniensensoren. Der Fehler ist –1, wenn links schwarz ist, +1 bei rechts und `0` sonst. Mit `BASE_SPEED = 110` und `STEER_GAIN = 90` bekommt der linke Motor `110 + 90`, der rechte `110 - 90`. So zieht das Auto zur Linie zurück. Sehen alle drei Sensoren schwarz, wechselt der Zustand nach `CROSSING`. Sehen alle drei länger als `300` Millisekunden weiß, geht es nach `SEARCH`. Ist ein Hindernis näher als 25 cm oder ist die Messung ungültig, geht es nach `OBSTACLE`.

`avoidObstacle` hält an, fährt `400` Millisekunden rückwärts und stellt den Kopf auf `20`, `90` und `150` Grad. `lookAt` schreibt den Winkel mit `headServo.write` und wartet `400` Millisekunden. Danach misst der Rover den Abstand. Er dreht `500` Millisekunden in die Seite mit mehr Platz und sucht dann die Linie.

`searchLine` fährt langsam im Bogen. Sieht ein Sensor wieder schwarz, geht es zurück nach `FOLLOW_LINE`. Nach 2 Sekunden ohne Linie hält der Rover an und bleibt in `SEARCH`.

`crossLine` zählt beim Betreten die Querlinie. Dafür sorgt `crossings += (next == CROSSING)` in `setState`. Der Rover fährt `250` Millisekunden gerade über die Linie. Ist die Linie vorbei und der Zähler noch unter 3, folgt er wieder der Linie. Bei der dritten Querlinie wechselt er nach `END`. Dort stoppt er, die LED wird grün, der Kopf steht auf `90` Grad.

Sicherheit vor Tempo

Drei Fälle dürfen nicht zum Blindflug werden:

- **Kein Echo:** Liefert der Ultraschall keine gültige Messung, weil die Leitung ab ist oder eine weiche Fläche schluckt, zählt das wie ein Hindernis. Der Rover hält an und wechselt nach `OBSTACLE`.
- **Linie verloren:** `SEARCH` sucht höchstens 2 Sekunden. Danach stoppt er und bleibt stehen.
- **Fahrt dauert zu lange:** Nach `RESCUE_MAX_MS` (30 Sekunden) ohne neue Querlinie wechselt der Rover nach `END` und stoppt. Auch eine hängende Querlinie beendet die Fahrt.

Bibliotheksversionen

FastLED 3.9.15 und Servo 1.2.2 vertragen sich. Mit FastLED 3.10.x bricht der Linker mit `multiple definition of __vector_11` ab. Wer die neuere Version installiert hat, wählt im Bibliotheksmanager 3.9.15.

Mission: zwei Funktionen fehlen

Im Sketch 131 sind `avoidObstacle()` und `searchLine()` leer. Alles andere ist vollständig. Ergänze die beiden Funktionen so, dass der Rover Hindernissen ausweicht und die Linie wiederfindet.

Deine Aufgabe

In `avoidObstacle()`: anhalten, 400 Millisekunden rückwärts fahren, den Kopf auf 20, 90 und 150 Grad stellen und jeweils den Abstand messen. Drehe danach 500 Millisekunden zu der Seite mit mehr Platz und starte `SEARCH`.

In `searchLine()`: Fahre mit `drive(70, 30)` im Bogen. Sieht ein Sensor schwarz, starte `FOLLOW_LINE`. Sind 2 Sekunden ohne Linie vergangen, halte an. `now - stateSince` liefert die Zeit seit dem Zustandswechsel.

Dein Code 131_rescue_rover_exercise.ino

```
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA=5, PIN_PWMB=6, PIN_AIN1=7, PIN_BIN1=8; // Motor
const uint8_t PIN_STBY=3, PIN_BUTTON=2;  // Treiber, Taste
const uint8_t PIN_TRIG=13, PIN_ECHO=12;  // Ultraschall
const uint8_t PIN_SERVO=10, PIN_LED=4;   // Kopf, LED
const uint8_t PIN_LINE_LEFT=A2, PIN_LINE_MIDDLE=A1;  // Linien-Sensoren
const uint8_t PIN_LINE_RIGHT=A0;  // Linien-Sensor rechts
enum State { FOLLOW_LINE, OBSTACLE, SEARCH, CROSSING, END };
State state = FOLLOW_LINE;
unsigned long stateSince = 0, whiteSince = 0;  // Zustandszeiten
uint8_t crossings = 0;  // Anzahl Querlinien
unsigned long lastProgress = 0;  // letzter Fortschritt
const char* const STATE_NAMES[] = {"FOLLOW", "OBSTACLE", "SEARCH",
                                   "CROSSING", "END"};
const CRGB STATE_LED[] = {CRGB::Blue, CRGB::Red, CRGB::Yellow,
                          CRGB::Blue, CRGB::Green};
const int LINE_THRESHOLD=600, BASE_SPEED=110, STEER_GAIN=90;  // Regelwerte
const int OBSTACLE_CM=25, SEARCH_MS=2000, WHITE_MS=300;  // Abstand/Zeiten
const int SCAN_LEFT=20, SCAN_RIGHT=150, SCAN_WAIT=400;  // Scan
const unsigned long RESCUE_MAX_MS = 30000;  // ohne Fortschritt: Stopp
CRGB leds[1]; Servo headServo;
void stopMotors() {
  analogWrite(PIN_PWMA,0); analogWrite(PIN_PWMB,0);
  digitalWrite(PIN_STBY,LOW);
}
void drive(int l, int r) {
  l = constrain(l, -255, 255); r = constrain(r, -255, 255);
  digitalWrite(PIN_BIN1, l >= 0); digitalWrite(PIN_AIN1, r >= 0);
  analogWrite(PIN_PWMB, abs(l)); analogWrite(PIN_PWMA, abs(r));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) != LOW) return false;
  delay(30); return digitalRead(PIN_BUTTON) == LOW;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste."));
  while (!buttonPressed()) delay(20);
  while (buttonPressed()) delay(20);
  for (int i = 3; i > 0; i--) { Serial.print(i); delay(1000); }
  Serial.println(F("Los!"));
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long t = pulseIn(PIN_ECHO, HIGH, 25000);
  if (t == 0) return -1;  // kein Echo: ungueltig, nicht "frei"
  return (int)(t / 58UL);
}
bool onBlack(uint8_t pin) { return analogRead(pin) < LINE_THRESHOLD; }
```

Fortsetzung 131_rescue_rover_exercise.ino

```
53  void lookAt(int angle) {
54    headServo.write(angle); delay(SCAN_WAIT);
55  }
56  void setState(State next) {
57    state = next; stateSince = millis();
58    if (next == CROSSING) { crossings++; lastProgress = millis(); }
59    if (next == FOLLOW_LINE) whiteSince = 0;
60    leds[0] = STATE_LED[next]; FastLED.show();
61    Serial.println(STATE_NAMES[next]);
62  }
63  void followLine(unsigned long now) {
64    bool l=onBlack(PIN_LINE_LEFT), m=onBlack(PIN_LINE_MIDDLE);
65    bool r=onBlack(PIN_LINE_RIGHT);
66    if (l && m && r) { setState(CROSSING); return; }
67    if (!l && !m && !r) {
68      if (whiteSince == 0) whiteSince = now;
69      if (now - whiteSince > WHITE_MS) { setState(SEARCH); return; }
70    } else whiteSince = 0;
71    int cm = readDistanceCm();
72    if (cm < 0) { stopMotors(); setState(OBSTACLE); return; }
73    if (cm < OBSTACLE_CM) { setState(OBSTACLE); return; }
74    int e = l ? -1 : (r ? 1 : 0);
75    drive(BASE_SPEED + e * STEER_GAIN, BASE_SPEED - e * STEER_GAIN);
76  }
77  void avoidObstacle() {
78    // HIER FEHLT: anhalten, 400 ms rueckwaerts, Servo 20/90/150 Grad
79    // scannen, in die freie Richtung drehen, dann SEARCH starten.
80    // Bei ungueltigem Echo sofort anhalten, nicht blind fahren.
81  }
82  void searchLine(unsigned long now) {
83    (void)now;
84    // HIER FEHLT: im Bogen fahren, bei Linie FOLLOW_LINE, nach 2 s
85    // anhalten.
86  }
87  void crossLine(unsigned long now) {
88    if (now - stateSince < 250) { drive(90, 90); return; }
89    if (onBlack(PIN_LINE_LEFT)&&onBlack(PIN_LINE_MIDDLE)&&
90        onBlack(PIN_LINE_RIGHT)) return;
91    if (crossings >= 3) { stopMotors(); lookAt(90); setState(END); }
92    else setState(FOLLOW_LINE);
93  }
94  void setup() {
95    Serial.begin(9600); pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
96    pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
97    pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
98    pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
99    pinMode(PIN_LINE_LEFT, INPUT); pinMode(PIN_LINE_MIDDLE, INPUT);
100   pinMode(PIN_LINE_RIGHT, INPUT);
101   headServo.attach(PIN_SERVO); stopMotors(); lookAt(90);
102   FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
103   FastLED.setBrightness(60); leds[0]=CRGB::Black; FastLED.show();
104   lastProgress = millis(); waitForStart(); setState(FOLLOW_LINE);
```

Fortsetzung 131_rescue_rover_exercise.ino

```
}
void loop() {
  unsigned long now = millis();
  if (state != END && now - lastProgress > RESCUE_MAX_MS) {
    stopMotors(); setState(END); return;
  }
  switch (state) {
    case FOLLOW_LINE: followLine(now); break;
    case OBSTACLE: avoidObstacle(); break;
    case SEARCH: searchLine(now); break;
    case CROSSING: crossLine(now); break;
    case END: stopMotors(); break;
  }
}
```

Ergänze die fehlenden Zeilen.

Lösung Mission Rettungsrover

Mission Rettungsrover 131_rescue_rover_solution.ino

```
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA=5, PIN_PWMB=6, PIN_AIN1=7, PIN_BIN1=8; // Motor
const uint8_t PIN_STBY=3, PIN_BUTTON=2;  // Treiber, Taste
const uint8_t PIN_TRIG=13, PIN_ECHO=12;  // Ultraschall
const uint8_t PIN_SERVO=10, PIN_LED=4;   // Kopf, LED
const uint8_t PIN_LINE_LEFT=A2, PIN_LINE_MIDDLE=A1;  // Linien-Sensoren
const uint8_t PIN_LINE_RIGHT=A0;  // Linien-Sensor rechts
enum State { FOLLOW_LINE, OBSTACLE, SEARCH, CROSSING, END };
State state = FOLLOW_LINE;
unsigned long stateSince = 0, whiteSince = 0;  // Zustandszeiten
uint8_t crossings = 0;  // Anzahl Querlinien
unsigned long lastProgress = 0;  // letzter Fortschritt
const char* const STATE_NAMES[] = {"FOLLOW", "OBSTACLE", "SEARCH",
                                   "CROSSING", "END"};
const CRGB STATE_LED[] = {CRGB::Blue, CRGB::Red, CRGB::Yellow,
                          CRGB::Blue, CRGB::Green};
const int LINE_THRESHOLD=600, BASE_SPEED=110, STEER_GAIN=90;  // Regelwerte
const int OBSTACLE_CM=25, SEARCH_MS=2000, WHITE_MS=300;  // Abstand/Zeiten
const int SCAN_LEFT=20, SCAN_RIGHT=150, SCAN_WAIT=400;  // Scan
const unsigned long RESCUE_MAX_MS = 30000;  // ohne Fortschritt: Stopp
CRGB leds[1]; Servo headServo;
void stopMotors() {
  analogWrite(PIN_PWMA,0); analogWrite(PIN_PWMB,0);
  digitalWrite(PIN_STBY,LOW);
}
void drive(int l, int r) {
  l = constrain(l, -255, 255); r = constrain(r, -255, 255);
  digitalWrite(PIN_BIN1, l >= 0); digitalWrite(PIN_AIN1, r >= 0);
  analogWrite(PIN_PWMB, abs(l)); analogWrite(PIN_PWMA, abs(r));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) != LOW) return false;
  delay(30); return digitalRead(PIN_BUTTON) == LOW;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste."));
  while (!buttonPressed()) delay(20);
```

Fortsetzung 131_rescue_rover_solution.ino

```
  while (buttonPressed()) delay(20);
  for (int i = 3; i > 0; i--) { Serial.print(i); delay(1000); }
  Serial.println(F("Los!"));
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long t = pulseIn(PIN_ECHO, HIGH, 25000);
  if (t == 0) return -1;  // kein Echo: ungueltig, nicht "frei"
  return (int)(t / 58UL);
}
bool onBlack(uint8_t pin) { return analogRead(pin) < LINE_THRESHOLD; }
void lookAt(int angle) {
  headServo.write(angle); delay(SCAN_WAIT);
}
void setState(State next) {
  state = next; stateSince = millis();
  if (next == CROSSING) { crossings++; lastProgress = millis(); }
  if (next == FOLLOW_LINE) whiteSince = 0;
  leds[0] = STATE_LED[next]; FastLED.show();
  Serial.println(STATE_NAMES[next]);
}
void followLine(unsigned long now) {
  bool l=onBlack(PIN_LINE_LEFT), m=onBlack(PIN_LINE_MIDDLE);
  bool r=onBlack(PIN_LINE_RIGHT);
  if (l && m && r) { setState(CROSSING); return; }
  if (!l && !m && !r) {
    if (whiteSince == 0) whiteSince = now;
    if (now - whiteSince > WHITE_MS) { setState(SEARCH); return; }
  } else whiteSince = 0;
  int cm = readDistanceCm();
  if (cm < 0) { stopMotors(); setState(OBSTACLE); return; }
  if (cm < OBSTACLE_CM) { setState(OBSTACLE); return; }
  int e = l ? -1 : (r ? 1 : 0);
  drive(BASE_SPEED + e * STEER_GAIN, BASE_SPEED - e * STEER_GAIN);
}
void avoidObstacle() {
  stopMotors();
  if (readDistanceCm() < 0) { setState(END); return; }
  drive(-110, -110); delay(400); stopMotors();
  lookAt(SCAN_LEFT); int l = readDistanceCm();
  lookAt(90); int m = readDistanceCm();
  lookAt(SCAN_RIGHT); int r = readDistanceCm(); lookAt(90);
  if (l < 0 || m < 0 || r < 0) { setState(END); return; }
  if (l > m && l ≥ r) drive(-120, 120); else if (r > m) drive(120, -120);
  delay(500); stopMotors(); setState(SEARCH);
}
void searchLine(unsigned long now) {
  if (onBlack(PIN_LINE_LEFT)||onBlack(PIN_LINE_MIDDLE)||
      onBlack(PIN_LINE_RIGHT)) { setState(FOLLOW_LINE); return; }
  if (now - stateSince > SEARCH_MS) { stopMotors(); return; }
```

Fortsetzung 131_rescue_rover_solution.ino

```
  drive(70, 30);
}
void crossLine(unsigned long now) {
  if (now - stateSince < 250) { drive(90, 90); return; }
  if (onBlack(PIN_LINE_LEFT)&&onBlack(PIN_LINE_MIDDLE)&&
      onBlack(PIN_LINE_RIGHT)) return;
  if (crossings ⩾ 3) { stopMotors(); lookAt(90); setState(END); }
  else setState(FOLLOW_LINE);
}
void setup() {
  Serial.begin(9600); pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_LINE_LEFT, INPUT); pinMode(PIN_LINE_MIDDLE, INPUT);
  pinMode(PIN_LINE_RIGHT, INPUT);
  headServo.attach(PIN_SERVO); stopMotors(); lookAt(90);
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60); leds[0]=CRGB::Black; FastLED.show();
  lastProgress = millis(); waitForStart(); setState(FOLLOW_LINE);
}
void loop() {
  unsigned long now = millis();
  if (state ≠ END && now - lastProgress > RESCUE_MAX_MS) {
    stopMotors(); setState(END); return;
  }
  switch (state) {
    case FOLLOW_LINE: followLine(now); break;
    case OBSTACLE: avoidObstacle(); break;
    case SEARCH: searchLine(now); break;
    case CROSSING: crossLine(now); break;
    case END: stopMotors(); break;
  }
}
```

In `avoidObstacle()` stehen die drei Messungen hintereinander. `l`, `m` und `r` sind die Abstände links, in der Mitte und rechts. Die Bedingung `l > m && l ⩾ r` wählt links, wenn dort mehr Platz ist als in der Mitte und mindestens so viel wie rechts. Sonst prüft `r > m` die rechte Seite. Ohne freie Seite fährt der Rover nicht los, sondern geht direkt in `SEARCH`.

In `searchLine()` prüft die erste Abfrage alle drei Sensoren. Ein einziger schwarzer Sensor reicht für `FOLLOW_LINE`. Danach entscheidet die Zeit: `now - stateSince > SEARCH_MS` bedeutet 2 Sekunden ohne Linie. Dann stoppt `stopMotors()` den Rover.

Erweitere selbst

Probiere

- **Kamerabild während der Fahrt:** Schalte den Schalter auf `Cam`, verbinde dein Handy mit dem WLAN und öffne `http://192.168.4.1/Test`. Der Uno fährt weiter, während das Handy das Bild zeigt.
- **Summen statt LED:** Ersetze die LED-Farbe durch einen kleinen Summer an einem freien Pin. Jeder Zustand bekommt einen anderen Ton. Welchen Pin dein Summer braucht, steht in seiner Anleitung.

- **Vierter Zustand:** Ergänze `MANUAL`. Dort liest der Rover die Fernbedienung aus Kapitel 7 und fährt mit den Tasten. Ein Tastendruck schaltet zwischen `FOLLOW_LINE` und `MANUAL` um.

Häufiger Fehler

So sieht es aus	Das hilft
Der Rover reagiert nicht auf die Taste.	Prüfe `pinMode(PIN_BUTTON, INPUT_PULLUP)`. Ohne den internen Widerstand liest der Pin keinen Tastendruck.
Der Rover zählt die Querlinien nicht.	Die Schwelle passt nicht. Lies die Rohwerte neu und setze `LINE_THRESHOLD` zwischen Schwarz und Weiß.
Der Linker meldet `multiple definition of __vector_11`.	Deine FastLED-Version ist zu neu. FastLED 3.10.x und Servo 1.2.2 belegen denselben Timer. Wähle im Bibliotheksmanager FastLED 3.9.15.
Der Rover fährt trotz fehlender Ultraschallmessung weiter.	Behandle den Timeout als Hindernis. `readDistanceCm()` muss einen ungültigen Wert liefern und `followLine` muss dann nach `OBSTACLE` wechseln. Ohne Echo gilt nie „freie Fahrt“.

Merken

- Ein Zustand ist eine Zahl mit einem Namen. `enum` legt die Namen fest.
- `switch` wählt die Funktion für den aktuellen Zustand.
- `millis()` liefert die Zeit seit dem Einschalten. Die Differenz zu `stateSince` misst, wie lange ein Zustand schon läuft.
- Jeder Zustand hat eine LED-Farbe und eine Aufgabe.
- FastLED 3.9.15 und Servo 1.2.2 passen zusammen. FastLED 3.10.x bricht beim Linken ab.
- Eine ungültige Messung zählt wie ein Hindernis, nicht als freie Fahrt.
- Nach 2 Sekunden ohne Linie und nach 30 Sekunden ohne neue Querlinie stoppt der Rover.

- [ ] Ich erkläre die fünf Zustände und ihre LED-Farben.
- [ ] Ich finde im Listing die Stelle, die den Zustand wechselt.
- [ ] Mein Rover folgt der Linie und weicht einem Hindernis aus.
- [ ] Mein Rover zählt drei Querlinien und hält danach an.
- [ ] Mein Rover hält bei fehlendem Echo und verlorener Linie an.
- [ ] Ich lese jeden Zustandswechsel im Serial Monitor.
- [ ] Ich habe eine eigene Erweiterung ausprobiert.

10
ESP32 selbst programmieren

Ziel

Du spielst ein eigenes Programm auf den ESP32. Es sucht eine Farbe im Kamerabild und schickt die Mitte an den Uno. Der Uno fährt dem farbigen Punkt nach. Du änderst die Farbe und baust eine ruhige Lenkung ein.

Mission

Baue ein Auto, das einem grünen Gegenstand folgt. Der ESP32 rechnet, der Uno lenkt. Zuerst testest du die Erkennung mit einem roten Gegenstand, am Ende sucht der Sketch Grün.

Warum eigene Firmware

Auf dem ESP32 läuft ab Werk eine fertige Firmware. Sie macht das Bild für die App und nimmt Befehle vom Handy an. Eine Position schickt sie nicht. Für die Farbverfolgung fehlt ihr genau diese Zahl.

Darum schreibst du ein eigenes Programm. Der ESP32 holt ein Bild, sucht die Farbe und schickt zwei Zahlen an den Uno: die Mitte `x` und die Breite `w` des Farbflecks. Der Uno muss das Bild nie sehen.

Achtung

Achtung, dieser Schritt überschreibt die Werksfirmware. Danach gehen App und Kamerabild über WLAN nicht mehr. Ohne die alte Datei gibt es keinen einfachen Rückweg. Lass dir dabei helfen und lies zuerst den Abschnitt „Werksfirmware zurückholen“ unten.

Firmware aufspielen

1. Öffne die Arduino IDE. Wähle als Board `ESP32 Dev Module`.
2. Stelle die Partition auf `Huge APP (3MB No OTA/1MB SPIFFS)`.
3. Schalte `PSRAM` auf `enabled`.
4. Stelle die Upload-Geschwindigkeit auf `115200`.
5. Schiebe den Schalter am Shield auf `Cam`.
6. Stecke das USB-Kabel am Uno ein. Der Schalter führt die Leitung zum ESP32.
7. Drücke auf Hochladen. Der ESP32 startet nach dem Flashen neu.

Werksfirmware zurückholen

Diesen Rückweg **musst du am Gerät testen**, bevor du ihn im Kurs einsetzt. Er ist hier nicht am Auto geprüft.

1. Schalter am Shield auf `Cam` schieben. Nur so hängt der USB-Anschluss am ESP32.
2. USB-Kabel am Uno lassen und den Port im Arduino IDE unter **Werkzeuge > Port** notieren, zum Beispiel `COM3`.
3. Tool: Arduino IDE mit dem Kern `esp32` 2.0.17. Board `ESP32 Dev Module`, Partition `Huge APP`, `PSRAM` auf `enabled`, 115200 Baud.
4. Binary im Kit: `02 Manual & Main Code & APP/04 Code of Carmer (ESP32)/ESP32-WROVER-Camera/ESP32_CameraServer_AP_20220120/ESP32_CameraServer_AP_20220120.ino.bin`. Einfacher ist der Quellordner daneben: Öffne dort `ESP32_CameraServer_AP_20220120.ino` und lade ihn hoch.
5. Mit `esptool` geht es auch: `esptool.py --chip esp32 --port COM3 --baud 115200 write_flash 0x10000 ESP32_CameraServer_AP_20220120.ino.bin`. Das setzt die passende Partitionstabelle voraus.

6. Prüfschritt: Schalter auf `Upload`, Serieller Monitor auf 115200. Nach dem Neustart erscheint wieder die Kamera-Ausgabe. Ohne diese Meldung ist die Firmware nicht zurückgespielt.

In Stellung `Cam` gehört der USB-Anschluss dem ESP32. Der Serial Monitor zeigt dann den ESP32. Der Uno bekommt in dieser Stellung keine Zeichen vom USB-Anschluss. Für den Uno-Upload schaltest du wieder auf `Upload`.

Die Kamera-Pins

Das Modul ist als `M5STACK_WIDE` verdrahtet. Diese Werte stehen im Kit-Sketch in der Datei `camera_pins.h`. Du kopierst sie in deinen Sketch. Eine eigene Header-Datei brauchst du nicht.

Pin	Bedeutung
PWDN	nicht benutzt, Wert -1
RESET	15
XCLK	27
SIOD / SIOC	22 / 23
D0 bis D7	32, 35, 34, 5, 39, 18, 36, 19
VSYNC / HREF	25 / 26
PCLK	21

Der Algorithmus in fünf Schritten

1. **Bild holen.** `esp_camera_fb_get()` gibt einen Zeiger auf die Bilddaten.
2. **Jedes vierte Pixel prüfen.** Bei 320 mal 240 Pixeln sind das 80 mal 60 Stichproben. Das reicht und spart Zeit.
3. **Farbregel anwenden.** Jedes Pixel wird in Rot, Grün und Blau zerlegt. Ein Treffer liegt vor, wenn Rot deutlich stärker ist als Grün und Blau.
4. **Mittelpunkt und Breite rechnen.** `x` ist der mittlere Wert aller Treffer, `w` der Abstand vom linken zum rechten Treffer.
5. **Senden.** Alle 100 Millisekunden geht `{"x":160,"w":42}` über `Serial2` an den Uno. Denselben Wert schreibt der ESP32 als `x=160 w=42` auf `Serial`. Das ist die Ausgabe für den Monitor.

Das Bild bleibt als Rohwert auf dem Chip. Es wird nicht als JPEG gepackt und nicht auf dem Uno dekodiert. Der ESP32 erledigt alles allein.

Farbe finden auf dem ESP32 101_color_track_esp32_exercise.ino

```
#include "esp_camera.h"
// Kamera-Pins des Moduls M5STACK_WIDE
const int PWDN=-1, RESET=15, XCLK=27, SIOD=22, SIOC=23;  // Kamera-Pins
const int D0=32, D1=35, D2=34, D3=5, D4=39, D5=18, D6=36, D7=19;  // Daten
const int VSYNC=25, HREF=26, PCLK=21, RXD2=33, TXD2=4;  // Kamera-/UART-Pins
const int W=320, H=240, STEP=4, SEND_MS=100, MIN_COUNT=4;  // Bild/Timing
const int R_MIN=120, R_OVER_G=50, R_OVER_B=50;  // Rot
void sendPosition(int x, int w) {
  // Lesbarer Wert fuer den Serial Monitor des ESP32.
  Serial.print(F("x=")); Serial.print(x);
  Serial.print(F(" w=")); Serial.println(w);
  // Dasselbe Telegramm an den Uno.
  Serial2.print(F("{\"x\":")); Serial2.print(x);
  Serial2.print(F(",\"w\":")); Serial2.print(w);
  Serial2.println(F("}"));
}
bool isTarget(uint8_t r, uint8_t g, uint8_t b) {
  return r>R_MIN && (r-g)>R_OVER_G && (r-b)>R_OVER_B;
}
void setup() {
  Serial.begin(9600); Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  camera_config_t c = {
    PWDN, RESET, XCLK, SIOD, SIOC,
    D7, D6, D5, D4, D3, D2, D1, D0,
    VSYNC, HREF, PCLK, 10000000, LEDC_TIMER_0, LEDC_CHANNEL_0,
    PIXFORMAT_RGB565, FRAMESIZE_QVGA,
    12, 1, CAMERA_FB_IN_PSRAM, CAMERA_GRAB_LATEST, 0,
  };
  if (esp_camera_init(&c) != ESP_OK) {
    Serial.println(F("Kamera startet nicht."));
    return;
  }
  Serial.println(F("Kamera bereit."));
}
void loop() {
  camera_fb_t *fb = esp_camera_fb_get();
  if (fb == NULL) { delay(200); return; }
  long sumX=0; int count=0; int minX=W; int maxX=-1;
  for (int y=0; y<H; y+=STEP) {
    for (int x=0; x<W; x+=STEP) {
      long i = 2L*(y*W + x);
      uint16_t p = fb->buf[i] | (fb->buf[i+1] << 8);
      uint8_t r=((p>>11)&0x1F)*255/31, g=((p>>5)&0x3F)*255/63;
      uint8_t b=(p&0x1F)*255/31;
      if (isTarget(r,g,b)) {
        sumX+=x; count++;
        minX = min(minX, x); maxX = max(maxX, x);
      }
```

Fortsetzung 101_color_track_esp32_exercise.ino

```
49      }
50    }
51    esp_camera_fb_return(fb);
52    int cx=W/2, w=0;
53    if (count ⩾ MIN_COUNT) { cx=sumX/count; w=maxX-minX; }
54    sendPosition(cx, w);
55    delay(SEND_MS);
56  }
```

Zeile 3 bis 8 legen Kamera-Pins und Schwellen fest. Zeile 9 bis 17 senden die Position auf `Serial` und `Serial2`. Zeile 18 bis 20 prüft ein Pixel. Zeile 21 bis 35 baut die Kamera auf. Zeile 36 bis 51 laufen über das Bild. Zeile 52 bis 56 rechnen die Mitte und senden.

Probiere

Schalter auf `Cam`, Arduino IDE unter **Werkzeuge > Port** den ESP32 wählen, Seriellen Monitor mit 9600 Baud öffnen. Halte einen roten Gegenstand vor die Kamera. Erfolg: `w=` ist größer als 0, und der Wert wandert mit dem Gegenstand. Ein grüner Gegenstand lässt `w=0`. Beim grünen Lösungssketch ist es umgekehrt. So prüfst du Rot und Grün mit derselben Anzeige.

Aufgabe: Grün statt Rot

Dein Auto soll einem grünen Gegenstand folgen. Ändere nur die Farbregel. Ein grüner Treffer liegt vor, wenn der Grünkanal über 100 liegt und Grün mindestens 40 über Rot und über Blau. Ersetze die drei roten Schwellen durch passende grüne Werte und passe `isTarget` an.

Deine Aufgabe

Nenne die Schwellen `G_MIN`, `G_OVER_R` und `G_OVER_B`. Prüfe in `isTarget`: `g > G_MIN`, `(g - r) > G_OVER_R` und `(g - b) > G_OVER_B`. Der Rest des Sketches bleibt gleich.

Lösung Mission Grün

Mission Grün 101_color_track_esp32_solution.ino

```
#include "esp_camera.h"
// Kamera-Pins des Moduls M5STACK_WIDE
const int PWDN=-1, RESET=15, XCLK=27, SIOD=22, SIOC=23;  // Kamera-Pins
const int D0=32, D1=35, D2=34, D3=5, D4=39, D5=18, D6=36, D7=19;  // Daten
const int VSYNC=25, HREF=26, PCLK=21, RXD2=33, TXD2=4;  // Kamera-/UART-Pins
const int W=320, H=240, STEP=4, SEND_MS=100, MIN_COUNT=4;  // Bild/Timing
const int G_MIN=100, G_OVER_R=40, G_OVER_B=40;  // Gruen
void sendPosition(int x, int w) {
  // Lesbarer Wert fuer den Serial Monitor des ESP32.
  Serial.print(F("x=")); Serial.print(x);
  Serial.print(F(" w=")); Serial.println(w);
  // Dasselbe Telegramm an den Uno.
  Serial2.print(F("{\"x\":")); Serial2.print(x);
  Serial2.print(F(",\"w\":")); Serial2.print(w);
  Serial2.println(F("}"));
}
bool isTarget(uint8_t r, uint8_t g, uint8_t b) {
  return g>G_MIN && (g-r)>G_OVER_R && (g-b)>G_OVER_B;
}
void setup() {
  Serial.begin(9600); Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  camera_config_t c = {
    PWDN, RESET, XCLK, SIOD, SIOC,
    D7, D6, D5, D4, D3, D2, D1, D0,
    VSYNC, HREF, PCLK, 10000000, LEDC_TIMER_0, LEDC_CHANNEL_0,
    PIXFORMAT_RGB565, FRAMESIZE_QVGA,
    12, 1, CAMERA_FB_IN_PSRAM, CAMERA_GRAB_LATEST, 0,
  };
  if (esp_camera_init(&c) != ESP_OK) {
    Serial.println(F("Kamera startet nicht."));
    return;
  }
  Serial.println(F("Kamera bereit."));
}
void loop() {
  camera_fb_t *fb = esp_camera_fb_get();
  if (fb == NULL) { delay(200); return; }
  long sumX=0; int count=0; int minX=W; int maxX=-1;
  for (int y=0; y<H; y+=STEP) {
```

Fortsetzung 101_color_track_esp32_solution.ino

```
40      for (int x=0; x<W; x+=STEP) {
41        long i = 2L*(y*W + x);
42        uint16_t p = fb->buf[i] | (fb->buf[i+1] << 8);
43        uint8_t r=((p>>11)&0x1F)*255/31, g=((p>>5)&0x3F)*255/63;
44        uint8_t b=(p&0x1F)*255/31;
45        if (isTarget(r,g,b)) {
46          sumX+=x; count++;
47          minX = min(minX, x); maxX = max(maxX, x);
48        }
49      }
50    }
51    esp_camera_fb_return(fb);
52    int cx=W/2, w=0;
53    if (count >= MIN_COUNT) { cx=sumX/count; w=maxX-minX; }
54    sendPosition(cx, w);
55    delay(SEND_MS);
56  }
```

Der Uno empfängt

Der Uno liest Zeichen von `Serial`, bis ein `}` kommt. Dann steht ein ganzes Telegramm im Puffer. Mit `strchr` sucht er den Buchstaben `x`, springt zum Doppelpunkt und wandelt die Zahl mit `atoi` um. Dafür braucht er keine JSON-Bibliothek.

Aus `x` und der Bildmitte `160` wird der Fehler. Geteilt durch 4 ergibt er den Lenkanteil. Liegt die Farbe rechts, bekommt der linke Motor mehr Tempo und das Auto dreht nach rechts.

Ist der Fleck schmaler als 12 Pixel, ist der Gegenstand zu weit weg. Dann hält das Auto an. Kommt `600` Millisekunden kein Telegramm, sucht es im langsamen Bogen. Bleibt das Telegramm 3 Sekunden aus, hält es ganz an. Die LED zeigt den Zustand: grün fährt, blau sucht, rot steht.

Dem Farbpunkt folgen 106_follow_color_uno_exercise.ino

```
#include <FastLED.h>
const uint8_t PIN_PWMA=5, PIN_AIN1=7;   // rechter Motor
const uint8_t PIN_PWMB=6, PIN_BIN1=8;   // linker Motor
const uint8_t PIN_STBY=3, PIN_BUTTON=2, PIN_LED=4;  // Treiber, Taste, LED
const int IMG_CENTER=160, MIN_WIDTH=12, STEER_DIV=4;  // Bild/Breite/Lenkung
const int BASE_SPEED=120, SEARCH_SPEED=70, SEARCH_FLIP_MS=1200;  // Tempo
const unsigned long LOST_MS=600;  // Ziel kurz weg: suchen
const unsigned long CAM_LOST_MS=3000;  // keine Kamera: anhalten
CRGB leds[1]; char buf[48]; int bufLen=0, searchDir=1;  // Puffer/Laenge
unsigned long lastSeen=0, lastFlip=0;  // letzte Sicht/Drehung

void stopMotors() {
  analogWrite(PIN_PWMA, 0); analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void setStatus(CRGB color) {
  leds[0] = color;
  FastLED.show();
}
bool readTelegram() {
  while (Serial.available()) {
```

Fortsetzung 106_follow_color_uno_exercise.ino

```
    char c = Serial.read();
    if (c == '{') { bufLen = 0; }
    else if (c == '}') { buf[bufLen] = '\0'; return true; }
    else if (bufLen < (int)sizeof(buf) - 1) { buf[bufLen++] = c; }
  }
  return false;
}
// Sucht "x":160 in text und gibt 160 zurueck. Sonst -1.
int readValue(const char *text, char key) {
  const char *p = strchr(text, key);
  if (p == NULL) return -1;
  p = strchr(p, ':');
  if (p == NULL) return -1;
  return atoi(p + 1);
}
void follow(int x, int w) {
  if (w < MIN_WIDTH) {  // zu weit weg
    stopMotors(); setStatus(CRGB::Red);
    return;
  }
  int error = x - IMG_CENTER;
  int steer = error / STEER_DIV;
  drive(BASE_SPEED + steer, BASE_SPEED - steer);
  setStatus(CRGB::Green);
}
void search() {
  drive(SEARCH_SPEED * searchDir, -SEARCH_SPEED * searchDir);
  setStatus(CRGB::Blue);
  if (millis() - lastFlip > SEARCH_FLIP_MS) {
    searchDir = -searchDir; lastFlip = millis();
  }
}
void setup() {
  Serial.begin(9600); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT); pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  setStatus(CRGB::Black);
  waitForStart();
  lastSeen = millis(); lastFlip = millis();
}
void loop() {
  if (readTelegram()) {
    int x = readValue(buf, 'x'), w = readValue(buf, 'w');
    if (x >= 0 && w >= 0) {
      lastSeen = millis(); follow(x, w);
      return;
    }
  }
  if (millis() - lastSeen > CAM_LOST_MS) {
```

Fortsetzung 106_follow_color_uno_exercise.ino

```
    }
  }
  if (millis() - lastSeen > CAM_LOST_MS) {
    stopMotors(); setStatus(CRGB::Red);
  } else if (millis() - lastSeen > LOST_MS) {
    search();
  }
}
```

Zeile 53 bis 61 lesen ein Telegramm. Zeile 63 bis 69 holen eine Zahl aus dem Text. Zeile 70 bis 79 lenken. Zeile 80 bis 86 suchen. Zeile 98 bis 111 wählen zwischen Folgen, Suchen und Anhalten.

Achtung

Das Start-Gate bleibt Pflicht. Ohne Tastendruck fährt nichts. Lege das Auto vor dem Einschalten ruhig hin, damit der ESP32 startet, bevor der Uno losfährt.

Aufgabe: Ruhiger lenken

Bei jeder kleinen Bewegung zappelt das Auto um die Mitte. Baue eine Totzone ein: Liegt x höchstens 18 Pixel neben der Bildmitte, fährt das Auto geradeaus. Außerdem soll die Suche langsamer werden. Setze `SEARCH_SPEED` von 70 auf 45.

Deine Aufgabe

Ergänze die Konstante `DEAD_ZONE = 18`. Setze `steer` zuerst auf 0. Nur wenn `abs(error) > DEAD_ZONE` ist, rechnest du `steer = error / STEER_DIV`. Ändere danach `SEARCH_SPEED`.

Lösung Mission Ruhige Lenkung

Mission Ruhige Lenkung 106_follow_color_uno_solution.ino

```
#include <FastLED.h>
const uint8_t PIN_PWMA=5, PIN_AIN1=7;   // rechter Motor
const uint8_t PIN_PWMB=6, PIN_BIN1=8;   // linker Motor
const uint8_t PIN_STBY=3, PIN_BUTTON=2, PIN_LED=4;  // Treiber, Taste, LED
const int IMG_CENTER=160, MIN_WIDTH=12, STEER_DIV=4;  // Bild/Breite/Lenkung
const int BASE_SPEED=120, SEARCH_SPEED=45, SEARCH_FLIP_MS=1200;  // Tempo
const unsigned long LOST_MS=600;  // Ziel kurz weg: suchen
const unsigned long CAM_LOST_MS=3000;  // keine Kamera: anhalten
const int DEAD_ZONE=18;  // Toleranz um die Bildmitte
CRGB leds[1]; char buf[48]; int bufLen=0, searchDir=1;  // Puffer/Laenge
unsigned long lastSeen=0, lastFlip=0;  // letzte Sicht/Drehung

void stopMotors() {
  analogWrite(PIN_PWMA, 0); analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
```

Fortsetzung 106_follow_color_uno_solution.ino

```
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void setStatus(CRGB color) {
  leds[0] = color;
  FastLED.show();
}
bool readTelegram() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '{') { bufLen = 0; }
    else if (c == '}') { buf[bufLen] = '\0'; return true; }
    else if (bufLen < (int)sizeof(buf) - 1) { buf[bufLen++] = c; }
  }
  return false;
}
// Sucht "x":160 in text und gibt 160 zurueck. Sonst -1.
int readValue(const char *text, char key) {
  const char *p = strchr(text, key);
  if (p == NULL) return -1;
  p = strchr(p, ':');
  if (p == NULL) return -1;
  return atoi(p + 1);
}
void follow(int x, int w) {
  if (w < MIN_WIDTH) {  // zu weit weg
    stopMotors(); setStatus(CRGB::Red);
    return;
  }
  int error = x - IMG_CENTER, steer = 0;
  if (abs(error) > DEAD_ZONE) steer = error / STEER_DIV;
  drive(BASE_SPEED + steer, BASE_SPEED - steer);
  setStatus(CRGB::Green);
}
void search() {
  drive(SEARCH_SPEED * searchDir, -SEARCH_SPEED * searchDir);
  setStatus(CRGB::Blue);
  if (millis() - lastFlip > SEARCH_FLIP_MS) {
    searchDir = -searchDir; lastFlip = millis();
  }
}
void setup() {
  Serial.begin(9600); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT); pinMode(PIN_PWMA, OUTPUT);
  pinMode(PIN_PWMB, OUTPUT);
  stopMotors();
```

Fortsetzung 106_follow_color_uno_solution.ino

```
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  setStatus(CRGB::Black);
  waitForStart();
  lastSeen = millis(); lastFlip = millis();
}
void loop() {
  if (readTelegram()) {
    int x = readValue(buf, 'x'), w = readValue(buf, 'w');
    if (x ⩾ 0 && w ⩾ 0) {
      lastSeen = millis(); follow(x, w);
      return;
    }
  }
  if (millis() - lastSeen > CAM_LOST_MS) {
    stopMotors(); setStatus(CRGB::Red);
  } else if (millis() - lastSeen > LOST_MS) {
    search();
  }
}
```

Grenzen

- Das Bild kommt mit 10 bis 15 Bildern pro Sekunde. Für ein fahrendes Auto reicht das, für schnelle Bewegungen nicht.
- Die Farberkennung hängt vom Licht ab. Am Fenster wirkt Rot anders als unter der Zimmerlampe. Probiere die Schwellen bei deinem Licht.
- Ähnliche Farben (zum Beispiel grüne Kleidung oder grüne Möbel) stören. Der ESP32 kennt nur Farbe, keine Form.
- 9600 Baud reichen für zwei Zahlen. Ein Bild bräuchte bei dieser Leitung mehrere Sekunden.

Häufiger Fehler

So sieht es aus	Das hilft
Der Serial Monitor bleibt leer.	In Stellung `Cam` zeigt der Monitor den ESP32, nicht den Uno. Schalte für den Uno-Upload auf `Upload`.
Die Kamera startet nicht.	Prüfe `PSRAM = enabled` und die Partition `Huge APP`. Stecke das 4-Pin-Kabel bei ausgeschaltetem Auto neu ein.
Das Auto dreht immer zur selben Seite.	Prüfe die Richtung: `x` über 160 heißt rechts. Tausche sonst `BASE_SPEED + steer` und `BASE_SPEED - steer`.
Das Auto hält ständig an.	Der Fleck ist zu schmal. Gehe näher heran oder senke `MIN_WIDTH` auf 8. Prüfe auch die Schwellen der Farbe.

Merken

- Eigene Firmware ersetzt die Werksfirmware. Der Rückweg steht oben und muss am Gerät getestet werden.
- Der ESP32 rechnet die Position, der Uno lenkt.
- `PIXFORMAT_RGB565` und `FRAMESIZE_QVGA` geben 320 mal 240 Pixel.
- Jedes vierte Pixel reicht für die Farbsuche.

- `{"x":160,"w":42}` sind Mitte und Breite in Pixeln.

- [ ] Ich spiele ein Programm auf den ESP32 und stelle den Schalter auf `Cam`.
- [ ] Ich nenne Board, Partition und PSRAM-Einstellung.
- [ ] Ich erkläre die fünf Schritte der Farbsuche.
- [ ] Ich ändere die Schwellen von Rot auf Grün.
- [ ] Ich lese `x` und `w` ohne Bibliothek aus dem Telegramm.
- [ ] Ich baue eine Totzone ein und mache die Suche langsamer.

11
Projekte zum Weiterbauen

Ziel

Du baust vier kleine Projekte, die du nach Kapitel 9 allein schaffst. Zwei Projekte haben eine Aufgabe mit Lösung auf der nächsten Seite. Jedes Projekt hat einen eigenen Sketch und eine Idee zum Verändern.

Mission

Baue einen Parkwächter, einen Tanzroboter und ein Schloss mit Geheimcode. Danach schickst du das Auto mit der Kamera auf eine Mission.

Vier Ideen für dein Auto

Die drei Lösungen sind vollständig und lassen sich direkt übersetzen; die Übungen enthalten TODO-Stellen. Öffne den passenden Ordner und darin genau diese Datei:

- Parkwächter:
  `code/111_parking_guard/111_parking_guard_exercise/ 111_parking_guard_exercise.ino`
- Tanzroboter: `code/116_dance/116_dance_exercise/116_dance_exercise.ino`
- Geheimcode:
  `code/121_secret_code/121_secret_code_solution/ 121_secret_code_solution.ino`

Jeder Sketch hat einen Pin-Block und ein Start-Gate.

Parkwächter

Ziel: Das Auto steht und beobachtet den Weg. Der Kopf schwenkt langsam nach links, in die Mitte und nach rechts. Unter 40 cm leuchtet die LED rot, und im Serial Monitor steht `Alarm`. Sonst leuchtet sie grün.

Bibliotheksversionen

FastLED 3.9.15 und Servo 1.2.2 vertragen sich. Bei FastLED 3.10.x bricht der Linker mit `multiple definition of __vector_11` ab. Wähle im Bibliotheksmanager dann 3.9.15.

`lookAt` stellt den Winkel mit `write` ein und wartet 450 Millisekunden. Danach steht der Kopf an der neuen Stellung.

Deine Aufgabe

Wähle eine Alarmgrenze zwischen 20 und 60 cm. Wähle einen Schwenkbereich zwischen 20 und 160 Grad. Trage die drei Werte ein und lade den Sketch hoch.

Dein Code 111_parking_guard_exercise.ino

```
// Setze die drei TODO-Werte unten und lade den Sketch hoch.
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;   // Ultraschall Senden
const uint8_t PIN_ECHO = 12;   // Ultraschall Empfangen
const uint8_t PIN_SERVO = 10;  // Servo
const uint8_t PIN_LED = 4;     // WS2812
const int ALARM_CM = 0;         // TODO: Alarmgrenze 20 bis 60
const int SWEEP_LEFT = 0;       // TODO: linker Winkel 20 bis 80
const int SWEEP_MIDDLE = 90;    // Grad
const int SWEEP_RIGHT = 0;      // TODO: rechter Winkel 100 bis 160
const unsigned long SENSOR_LOST_MS = 5000;  // ohne Echo: Stopp
Servo headServo;
CRGB leds[1];
unsigned long lastValid = 0;  // letzte gueltige Messung
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void lookAt(int angle) {
  headServo.write(angle);
  delay(450);
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
```

Fortsetzung 111_parking_guard_exercise.ino

```
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltig, nicht "frei"
  }
  return (int)(duration / 58UL);
}
bool scanAt(int angle) {
  lookAt(angle);
  int cm = readDistanceCm();
  if (cm < 0) {  // Messung ungueltig: stehen bleiben
    leds[0] = CRGB::Red;
    FastLED.show();
    Serial.println(F("Alarm Sensorfehler"));
    return false;
  }
  lastValid = millis();
  bool alarm = cm < ALARM_CM;
  leds[0] = alarm ? CRGB::Red : CRGB::Green;
  FastLED.show();
  Serial.print(alarm ? F("Alarm ") : F("Frei "));
  Serial.print(cm); Serial.println(F(" cm"));
  return true;
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  headServo.attach(PIN_SERVO);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
  lastValid = millis();
  waitForStart();
}
void loop() {
  if (millis() - lastValid > SENSOR_LOST_MS) {
    stopMotors();
    leds[0] = CRGB::Red;
    FastLED.show();
    Serial.println(F("Sensor weg - Stopp"));
    return;  // Kopf bleibt stehen
  }
  if (!scanAt(SWEEP_LEFT)) return;
  if (!scanAt(SWEEP_MIDDLE)) return;
  if (!scanAt(SWEEP_RIGHT)) return;
  if (!scanAt(SWEEP_MIDDLE)) return;
  delay(300);
```

Fortsetzung 111_parking_guard_exercise.ino

```
      return;  // Kopf bleibt stehen
    }
    if (!scanAt(SWEEP_LEFT)) return;
    if (!scanAt(SWEEP_MIDDLE)) return;
    if (!scanAt(SWEEP_RIGHT)) return;
    if (!scanAt(SWEEP_MIDDLE)) return;
    delay(300);
  }
```

Ergänze die fehlenden Zeilen.

Lösung Parkwächter

Parkwächter 111_parking_guard_solution.ino

```
// Der Kopf schwenkt, der Abstandswaechter meldet Alarm.
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;   // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;    // Ultraschall Senden
const uint8_t PIN_ECHO = 12;    // Ultraschall Empfangen
const uint8_t PIN_SERVO = 10;   // Servo
const uint8_t PIN_LED = 4;      // WS2812
const int ALARM_CM = 40;         // naeher heran: Alarm
const int SWEEP_LEFT = 30;       // Grad
const int SWEEP_MIDDLE = 90;     // Grad
const int SWEEP_RIGHT = 150;     // Grad
const unsigned long SENSOR_LOST_MS = 5000;  // ohne Echo: Stopp
Servo headServo;
CRGB leds[1];
unsigned long lastValid = 0;  // letzte gueltige Messung
void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
```

Fortsetzung 111_parking_guard_solution.ino

```
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void lookAt(int angle) {
  headServo.write(angle);
  delay(450);
}
int readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  unsigned long duration = pulseIn(PIN_ECHO, HIGH, 25000);
  if (duration == 0) {
    return -1;  // kein Echo: ungueltig, nicht "frei"
  }
  return (int)(duration / 58UL);
}
bool scanAt(int angle) {
  lookAt(angle);
  int cm = readDistanceCm();
  if (cm < 0) {  // Messung ungueltig: stehen bleiben
    leds[0] = CRGB::Red;
    FastLED.show();
    Serial.println(F("Alarm Sensorfehler"));
    return false;
  }
  lastValid = millis();
  bool alarm = cm < ALARM_CM;
  leds[0] = alarm ? CRGB::Red : CRGB::Green;
  FastLED.show();
  Serial.print(alarm ? F("Alarm ") : F("Frei "));
  Serial.print(cm); Serial.println(F(" cm"));
  return true;
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT);
  headServo.attach(PIN_SERVO);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
  lastValid = millis();
  waitForStart();
}
```

Fortsetzung 111_parking_guard_solution.ino

```
void loop() {
  if (millis() - lastValid > SENSOR_LOST_MS) {
    stopMotors();
    leds[0] = CRGB::Red;
    FastLED.show();
    Serial.println(F("Sensor weg - Stopp"));
    return;  // Kopf bleibt stehen
  }
  if (!scanAt(SWEEP_LEFT)) return;
  if (!scanAt(SWEEP_MIDDLE)) return;
  if (!scanAt(SWEEP_RIGHT)) return;
  if (!scanAt(SWEEP_MIDDLE)) return;
  delay(300);
}
```

`scanAt` dreht den Kopf, misst und entscheidet in einem Zug. Die Schleife besucht links, Mitte, rechts und noch einmal Mitte. So schaut der Kopf nach jeder Runde wieder geradeaus. Fehlt das Echo, zeigt die LED Rot und der Kopf bleibt stehen. Bleibt das Echo fünf Sekunden aus, hält das Programm ganz an, statt „frei“ anzunehmen.

Probiere

Setze `ALARM_CM` auf 25. Der Wächter meldet sich erst, wenn jemand näher kommt. Setze danach `SWEEP_LEFT` auf 60 und `SWEEP_RIGHT` auf 120. Der Kopf schaut nur noch in einen schmalen Bereich.

Tanzroboter

Ziel: Das Auto tanzt drei Strophen. Jede Strophe fährt, dreht und wackelt mit dem Kopf. Die LED zeigt die Farbe der Strophe.

Jede Strophe ist eine eigene Funktion. `move` fährt eine Zeit lang und hält danach an. `wiggleHead` schaut links, rechts und zurück. Die Zeiten stehen als Zahlen im Code. Es gibt keinen Zufall: Jeder Lauf sieht gleich aus. Musik braucht der Tanz nicht.

Deine Aufgabe

Schreibe `verseFour`. Nimm mindestens zwei Bewegungen aus `move`, eine Farbe mit `setLed` und einmal `wiggleHead`. Die vierte Strophe wird schon in `loop` aufgerufen.

Dein Code 116_dance_exercise.ino

```
// Drei Strophen sind fertig. Schreibe die vierte.
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_SERVO = 10;  // Servo
const uint8_t PIN_LED = 4;     // WS2812
const int DANCE_SPEED = 140;    // 140 von 255
const int TURN_SPEED = 130;     // 130 von 255
const int SERVO_LEFT = 55, SERVO_MIDDLE = 90, SERVO_RIGHT = 125;  // Winkel
Servo headServo;
CRGB leds[1];
void stopMotors() {
  analogWrite(PIN_PWMA, 0); analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) delay(20);
  while (buttonPressed()) delay(20);
  for (int i = 3; i > 0; i--) {
    Serial.print(i); Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void setLed(CRGB color) {
  leds[0] = color; FastLED.show();
}
void lookAt(int angle) {
  headServo.write(angle);
  delay(450);
```

Fortsetzung 116_dance_exercise.ino

```
}
void wiggleHead() {
  lookAt(SERVO_LEFT); lookAt(SERVO_RIGHT); lookAt(SERVO_MIDDLE);
}
void move(int leftSpeed, int rightSpeed, int ms) {
  drive(leftSpeed, rightSpeed); delay(ms); stopMotors(); delay(150);
}
void verseOne() {
  setLed(CRGB::Red); move(DANCE_SPEED, DANCE_SPEED, 700);
  move(-TURN_SPEED, TURN_SPEED, 500); wiggleHead();
}
void verseTwo() {
  setLed(CRGB::Blue); move(TURN_SPEED, -TURN_SPEED, 500);
  move(DANCE_SPEED, DANCE_SPEED, 900); wiggleHead();
}
void verseThree() {
  setLed(CRGB::Green); move(-DANCE_SPEED, -DANCE_SPEED, 500);
  move(-TURN_SPEED, TURN_SPEED, 800); wiggleHead();
}
void verseFour() {
  // TODO: setLed, move und wiggleHead zu einer Strophe verbinden
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  headServo.attach(PIN_SERVO); stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
  waitForStart();
}
void loop() {
  verseOne(); verseTwo();
  verseThree(); verseFour();
  setLed(CRGB::Black); delay(1000);
}
```

Ergänze die fehlenden Zeilen.

Lösung Tanzroboter

Tanzroboter 116_dance_solution.ino

```ino
// Fahren, Drehen, Farbe und Kopf-Wackeln mit dem Servo.
#include <Servo.h>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;    // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;    // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;    // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;    // linker Motor: Richtung
const uint8_t PIN_STBY = 3;    // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_SERVO = 10;  // Servo
const uint8_t PIN_LED = 4;     // WS2812
const int DANCE_SPEED = 140;    // 140 von 255
const int TURN_SPEED = 130;     // 130 von 255
const int SERVO_LEFT = 55, SERVO_MIDDLE = 90, SERVO_RIGHT = 125;  // Winkel
Servo headServo;
CRGB leds[1];
void stopMotors() {
  analogWrite(PIN_PWMA, 0); analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) delay(20);
```

Fortsetzung 116_dance_solution.ino

```
  while (buttonPressed()) delay(20);
  for (int i = 3; i > 0; i--) {
    Serial.print(i); Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void setLed(CRGB color) {
  leds[0] = color; FastLED.show();
}
void lookAt(int angle) {
  headServo.write(angle);
  delay(450);
}
void wiggleHead() {
  lookAt(SERVO_LEFT); lookAt(SERVO_RIGHT); lookAt(SERVO_MIDDLE);
}
void move(int leftSpeed, int rightSpeed, int ms) {
  drive(leftSpeed, rightSpeed); delay(ms); stopMotors(); delay(150);
}
void verseOne() {
  setLed(CRGB::Red); move(DANCE_SPEED, DANCE_SPEED, 700);
  move(-TURN_SPEED, TURN_SPEED, 500); wiggleHead();
}
void verseTwo() {
  setLed(CRGB::Blue); move(TURN_SPEED, -TURN_SPEED, 500);
  move(DANCE_SPEED, DANCE_SPEED, 900); wiggleHead();
}
void verseThree() {
  setLed(CRGB::Green); move(-DANCE_SPEED, -DANCE_SPEED, 500);
  move(-TURN_SPEED, TURN_SPEED, 800); wiggleHead();
}
void verseFour() {
  setLed(CRGB::Yellow); move(-TURN_SPEED, TURN_SPEED, 900);
  move(TURN_SPEED, -TURN_SPEED, 900); wiggleHead();
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  headServo.attach(PIN_SERVO); stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
  waitForStart();
}
void loop() {
  verseOne(); verseTwo();
  verseThree(); verseFour();
  setLed(CRGB::Black); delay(1000);
}
```

`move` bekommt links, rechts und die Dauer in Millisekunden. Ein negativer Wert dreht das Rad rückwärts. `loop` spielt alle vier Strophen hintereinander und schaltet danach die LED aus. Die vierte Strophe ist deine Lösung aus der Aufgabe.

Probiere

Tausche in `verseOne` die Farbe gegen `CRGB::Orange`. Ändere die Dauer der ersten Bewegung von 700 auf 1200 Millisekunden. Die Strophe wird langsamer und die LED zeigt eine andere Farbe.

Geheimcode

Ziel: Du tippst mit der Fernbedienung 1, dann 2, dann 3. Stimmt die Folge, fährt das Auto seine Figur und die LED wird grün. Bei einem falschen Zeichen wird sie rot und die Folge beginnt neu.

`step` merkt sich, welche Ziffer als Nächstes kommt. `SECRET` speichert die Folge. `digitFromCommand` liest `decodedIRData.command` aus IRremote 4.x und vergleicht die Tasten der Familie A: 1 ist `0x68`, 2 ist `0x98`, 3 ist `0xB0`. Ein unbekannter Code liefert `0` und wird ignoriert. Eine andere Fernbedienung misst du mit Beispiel 086 und trägst ihre Werte ein. Die 1 zählt auch als neuer Anfang: Auf 1-2-1-2-3 folgt die Figur.

Geheimcode 121_secret_code_solution.ino

```ino
#include <IRremote.hpp>
#include <FastLED.h>
const uint8_t PIN_PWMA = 5;     // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;     // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;     // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;     // linker Motor: Richtung
const uint8_t PIN_STBY = 3;     // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;  // Taste auf dem Shield
const uint8_t PIN_IR = 9;       // IR-Empfaenger
const uint8_t PIN_LED = 4;      // WS2812
const int DRIVE_SPEED = 140;     // 140 von 255
const int TURN_SPEED = 130;      // 130 von 255
const uint8_t SECRET[] = {1, 2, 3};  // geheime Ziffernfolge
const uint8_t SECRET_LEN = 3;  // Laenge der Ziffernfolge
CRGB leds[1];
uint8_t step = 0;  // aktueller Schritt
void stopMotors() {
  analogWrite(PIN_PWMA, 0); analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}
void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}
bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);  // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {  // warten, bis die Taste losgelassen ist
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
    delay(1000);
  }
  Serial.println(F("Los!"));
}
void setLed(CRGB color) {
```

Fortsetzung 121_secret_code_solution.ino

```
  leds[0] = color; FastLED.show();
}
// IRremote 4.x: command der Familie A. Andere Fernbedienung mit
// Beispiel 086 messen und die Werte hier eintragen.
uint8_t digitFromCommand(uint8_t cmd) {
  if (cmd == 0x68) return 1;  // Taste 1
  if (cmd == 0x98) return 2;  // Taste 2
  if (cmd == 0xB0) return 3;  // Taste 3
  return 0;
}
void playFigure() {
  drive(DRIVE_SPEED, DRIVE_SPEED); delay(600);
  drive(-TURN_SPEED, TURN_SPEED); delay(450);
  drive(DRIVE_SPEED, DRIVE_SPEED); delay(600);
  drive(TURN_SPEED, -TURN_SPEED); delay(450);
  stopMotors();
}
void handleDigit(uint8_t digit) {
  if (digit == SECRET[step]) {
    step++;
    Serial.print(F("Stufe ")); Serial.println(step);
    if (step == SECRET_LEN) {
      setLed(CRGB::Green);
      Serial.println(F("Offen!"));
      playFigure();
      step = 0;
      setLed(CRGB::Black);
    }
    return;
  }
  step = 0; setLed(CRGB::Red); Serial.println(F("Falsch"));
  delay(600); setLed(CRGB::Black);
  if (digit == SECRET[0]) step = 1;  // die 1 beginnt neu
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT); pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT); pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT); pinMode(PIN_BUTTON, INPUT_PULLUP);
  stopMotors();
  FastLED.addLeds<WS2812, PIN_LED, GRB>(leds, 1);
  FastLED.setBrightness(60);
  setLed(CRGB::Black);
  IrReceiver.begin(PIN_IR, false);  // D13 bleibt frei
  waitForStart();
}
void loop() {
  if (!IrReceiver.decode()) return;
  uint8_t cmd = IrReceiver.decodedIRData.command;
  IrReceiver.resume();
  uint8_t digit = digitFromCommand(cmd);
  if (digit != 0) handleDigit(digit);
```

Fortsetzung 121_secret_code_solution.ino

```
 98  }
 99  void loop() {
100    if (!IrReceiver.decode()) return;
101    uint8_t cmd = IrReceiver.decodedIRData.command;
102    IrReceiver.resume();
103    uint8_t digit = digitFromCommand(cmd);
104    if (digit != 0) handleDigit(digit);
105  }
```

`playFigure` fährt vorwärts, dreht links, fährt vorwärts und dreht rechts. Die Figur ist fest gespeichert. Nach der Figur setzt `handleDigit` den Schritt auf 0 zurück. Das Start-Gate bleibt Pflicht: Erst Taste, dann Countdown, dann nimmt der Loop Codes an.

Probiere

Ändere `SECRET` auf `{3, 1, 2}`. Jetzt öffnet 3-1-2 das Schloss. Die Ziffern selbst bleiben dieselben.

Kamera-Mission

Ziel: Dein Auto folgt einem grünen Gegenstand und stoppt unter 25 cm. Der fertige Missions-Sketch liegt unter `code/106_follow_color_uno/106_follow_color_mission/106_follow_color_mission_solution/ 106_follow_color_mission_solution.ino`. Er ist Beispiel 106 mit Ultraschall und lässt sich direkt übersetzen.

Der ESP32 sucht Grün, genau wie die Lösung von Kapitel 10. Der Uno liest `x` und `w`, lenkt mit Totzone und bremst vor Hindernissen. Drei Stellen sind wichtig: `STOP_CM = 25`, `readDistanceCm()` gibt bei fehlendem Echo `-1` zurück, und `follow` hält bei `cm < 0` oder `cm < STOP_CM` an. `setup` richtet `PIN_TRIG` als Ausgang und `PIN_ECHO` als Eingang ein. Ohne diese zwei Zeilen misst der Sensor nicht.

Die zwei wichtigen Stellen stehen hier. Der Rest ist Beispiel 106.

```
pinMode(PIN_TRIG, OUTPUT); pinMode(PIN_ECHO, INPUT); int cm = readDistanceCm(); if (cm < 0 || cm < STOP_CM) { stopMotors(); setStatus(CRGB::Red); return; }
```

Beobachte drei Fälle. Ist die grüne Fläche groß genug, folgt das Auto und die LED ist grün. Verschwindet sie aus dem Bild, sucht das Auto im Kreis und die LED wird blau. Bleibt das Telegramm drei Sekunden aus, hält das Auto an. Kommt der Gegenstand unter 25 cm, stoppt es ebenfalls. In beiden Fällen wird die LED rot. Der Ultraschall schaut nur geradeaus. Halte den Sensor frei und richte ihn nach vorn.

Häufiger Fehler

So sieht es aus	Das hilft
Compiler: multiple definition of __vector_11.	Wähle im Bibliotheksmanager FastLED 3.9.15. Mit 3.10.x belegen Servo und FastLED beide Timer1.
Die LED bleibt dunkel.	Prüfe GRB in addLeds und rufe FastLED.show() auf.
Der Geheimcode reagiert nicht.	Drücke zuerst die Taste auf dem Shield und prüfe die Codes mit dem Auslese-Sketch.
Der Parkwächter meldet ohne Hindernis Alarm.	Weiche und schräge Flächen täuschen den Sensor. Richte ihn gerade auf eine ebene Fläche.

Merken

- FastLED 3.9.15 und Servo 1.2.2 vertragen sich. Mit FastLED 3.10.x bricht der Linker mit `__vector_11` ab.
- `scanAt` verbindet Drehen, Messen und Anzeigen.
- Jede Tanzstrophe ist eine Funktion. Feste Zeiten machen den Tanz wiederholbar.
- `SECRET` und `step` prüfen eine Ziffernfolge. `digitFromCommand` liest die Familie-A-Tasten aus IRremote 4.x; andere misst du mit 086.
- Die Kamera-Mission nutzt Grün und stoppt unter 25 cm. Der fertige Sketch heißt `106_follow_color_mission_solution.ino`.

- [ ] Ich baue den Parkwächter und ändere Alarmgrenze und Schwenkbereich.
- [ ] Ich schreibe eine eigene Tanzstrophe aus `move`, `setLed` und `wiggleHead`.
- [ ] Ich gebe den Geheimcode ein und sehe die Figur.
- [ ] Ich lade die grüne Kamera-Mission und prüfe den Stopp unter 25 cm.

12
Code ordnen und eigene Klassen

Ziel

Du teilst einen großen Sketch in mehrere Dateien auf. Du verstehst, wofür Header, Includes und `#pragma once` gut sind. Du verstehst, wie die Klasse `DistanceSensor` aus Header und `.cpp` aufgebaut ist, und nutzt sie in der Notbremse aus Kapitel 3.

Mission

Räume deinen Sketch auf: Header, Quelle und Hauptprogramm liegen in getrennten Dateien. Den Ultraschallsensor übernimmt die vorgefertigte Klasse `DistanceSensor`. Am Ende fährt das Auto mit der Notbremse aus Kapitel 3, nur besser sortiert.

Wenn der Sketch wächst

Am Anfang passt alles in eine Datei. Nach ein paar Kapiteln wird sie lang. Oben stehen die Pins, darunter die Motoren, dann der Sensor, dann das Start-Gate, und ganz unten kommt `loop`. Wer eine Zeile sucht, scrollt lange.

Eine große Datei hat drei Nachteile:

- Du findest zusammengehörige Zeilen schwer wieder.
- Zwei Personen können nicht gut an derselben Datei arbeiten.
- Du kannst einen Baustein nicht in einem anderen Projekt wiederverwenden.

Die Lösung heißt: pro Aufgabe eine eigene Datei. Der Ultraschallsensor bekommt seinen eigenen Bauplan und seinen eigenen Code. Die `.ino` wird dadurch kurz und liest sich wie eine Geschichte: starten, messen, entscheiden, fahren.

Header und Includes

Ein C++-Programm besteht aus zwei Sorten von Dateien:

- Eine **Header-Datei** endet auf `.h`. Sie enthält die **Deklarationen**: Wie heißt eine Funktion, welche Parameter nimmt sie, welche Klasse gibt es? Der Header zeigt nur den Bauplan, keinen rechenbaren Code.
- Eine **Quell-Datei** endet auf `.cpp`. Sie enthält die **Definitionen**: Den Code, der wirklich rechnet. Zu jedem Eintrag im Header steht hier die passende Umsetzung.

Die `.ino` bindet den Header mit `#include "DistanceSensor.h"` ein. Die Anführungszeichen sagen: Suche die Datei zuerst im selben Ordner wie diesen Sketch. Bei `#include <Arduino.h>` stehen spitze Klammern. Damit sucht der Compiler in den mitgelieferten Bibliotheken, hier nach den Arduino-Funktionen.

Ganz oben in jeder eigenen Header-Datei steht `#pragma once`. Das ist ein Befehl an den Compiler: Lies diese Datei nur ein einziges Mal, auch wenn sie an mehreren Stellen eingebunden wird. Ohne diesen Schutz entstehen doppelte Definitionen und der Upload bricht ab.

Ein kurzer Header sieht so aus:

```
// motor.h - nur der Bauplan
#pragma once
#include <Arduino.h>

void drive(int left, int right);
void stopMotors();
```

Mehr steht nicht drin. Kein `analogWrite`, keine Schleife, nur die Ankündigung. Den Code schreibst du in die `.cpp`. So bleibt der Header kurz und du siehst sofort, was ein Baustein kann.

Eine Klasse ist ein Bauplan

Eine **Klasse** bündelt Daten und Funktionen. Die Daten heißen **Member**, die Funktionen heißen **Methoden**. Für den Sensor gehören die beiden Pins zu den Daten. Das Messen gehört zu den Methoden.

Eine Klasse hat drei wichtige Teile:

- Der **Konstruktor** trägt denselben Namen wie die Klasse und hat keinen Rückgabetyp. Er läuft einmal, wenn du ein Objekt anlegst, und speichert die Startwerte.
- `public` bedeutet: Von außen sichtbar. Diese Methoden darfst du in `setup` und `loop` aufrufen.
- `private` bedeutet: Nur innerhalb der Klasse sichtbar. Die Pins bleiben versteckt, damit niemand sie versehentlich ändert.

Ein Objekt erzeugst du mit einer Zeile wie `DistanceSensor sensor(13, 12);`. Die Zahlen sind die Pins für TRIG und ECHO. Danach rufst du `sensor.begin()` und `sensor.readCm()` auf. Der Punkt verbindet das Objekt mit seiner Methode.

DistanceSensor.h DistanceSensor.h

```
// Kapitel 12: Header mit der Deklaration der Klasse.
#pragma once                      // Datei nur einmal einbinden
#include <Arduino.h>              // uint8_t, pinMode und pulseIn

class DistanceSensor {            // Bauplan fuer einen Sensor
 public:                          // von aussen sichtbar
  DistanceSensor(uint8_t trigPin, uint8_t echoPin);  // Konstruktor
  void begin();                   // Pins als Ein-/Ausgang setzen
  int readCm();                   // Abstand in Zentimetern liefern

 private:                         // nur in der Klasse sichtbar
  uint8_t trigPin;                // Pin zum Senden
  uint8_t echoPin;                // Pin zum Empfangen
};
```

Der Header zeigt nur den Bauplan. `DistanceSensor` ist der Konstruktor. `begin` und `readCm` sind öffentlich. Die beiden Pin-Variablen sind privat.

DistanceSensor.cpp DistanceSensor.cpp

```cpp
// Kapitel 12: Hier stehen die Definitionen der Methoden.
#include "DistanceSensor.h"      // passender Header

DistanceSensor::DistanceSensor(uint8_t trigPin, uint8_t echoPin) {
  this->trigPin = trigPin;       // Wert im Objekt speichern
  this->echoPin = echoPin;       // Wert im Objekt speichern
}

void DistanceSensor::begin() {
  pinMode(trigPin, OUTPUT);      // Trigger sendet
  pinMode(echoPin, INPUT);       // Echo empfaengt
}

int DistanceSensor::readCm() {
  digitalWrite(trigPin, LOW);    // Impuls vorbereiten
  delayMicroseconds(2);          // kurze Pause
  digitalWrite(trigPin, HIGH);   // Startbefehl
  delayMicroseconds(10);         // 10 us Impuls
  digitalWrite(trigPin, LOW);    // Impuls beenden
  unsigned long dauer = pulseIn(echoPin, HIGH, 25000);  // warten
  if (dauer == 0) {              // kein Echo
    return -1;                   // -1 = ungueltig, kein Echo
  }
  return (int)(dauer / 58UL);    // Laufzeit in cm
}
```

In der `.cpp` steht vor jedem Methodennamen `DistanceSensor::`. Das sagt: Diese Funktion gehört zu dieser Klasse. Im Konstruktor speichert `this->trigPin` den übergebenen Wert im eigenen Objekt. `this` zeigt auf das Objekt, das gerade benutzt wird.

`readCm` enthält dieselbe Messung wie in Kapitel 3: 10 Mikrosekunden Trigger, `pulseIn` mit dem Limit 25000 und die Division durch 58. Bei einem Timeout liefert die Methode `-1`. Das heißt: keine gültige Messung, und das Auto stoppt.

Mission: Notbremse mit Klasse

Jetzt verbindest du beides. Die `.ino` bindet den Header ein, legt ein Objekt an und benutzt es in `loop`. Die Pins, `stopMotors`, `drive` und das Start-Gate bleiben wie in Kapitel 3.

Deine Aufgabe

Ergänze die `TODO`-Zeilen. Lege zuerst das Objekt `sensor` mit `PIN_TRIG` und `PIN_ECHO` an. Rufe dann in `setup` die Methode `begin` auf. In `loop` liest du den Abstand mit `sensor.readCm()` und entscheidest wie bei der Notbremse: unter 20 cm oder keine Messung: anhalten, sonst fahren.

Dein Code 126_classes_exercise.ino

```
// Kapitel 12, Aufgabe: Nutze die Klasse DistanceSensor.
// Ergaenze die TODO-Zeilen.
#include "DistanceSensor.h"      // Klasse einbinden

const uint8_t PIN_PWMA = 5;      // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;      // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;      // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;      // linker Motor: Richtung
const uint8_t PIN_STBY = 3;      // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;    // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;     // Ultraschall Senden
const uint8_t PIN_ECHO = 12;     // Ultraschall Empfangen
const int STOP_DISTANCE_CM = 20; // naeher heran: anhalten
const int DRIVE_SPEED = 120;     // 120 von 255

// TODO: Lege ein Objekt sensor mit PIN_TRIG und PIN_ECHO an.

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);                   // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}

void waitForStart() {
  Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
  while (!buttonPressed()) {
    delay(20);
  }
  while (buttonPressed()) {
    delay(20);
  }
  for (int i = 3; i > 0; i--) {
    Serial.print(i);
    Serial.println(F(" ..."));
```

Fortsetzung 126_classes_exercise.ino

```
    delay(1000);
  }
  Serial.println(F("Los!"));
}

void setup() {
  Serial.begin(9600);
  pinMode(PIN_PWMA, OUTPUT);     // Motorpins vorbereiten
  pinMode(PIN_PWMB, OUTPUT);
  pinMode(PIN_AIN1, OUTPUT);
  pinMode(PIN_BIN1, OUTPUT);
  pinMode(PIN_STBY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  // TODO: Setze die Sensorpins mit sensor.begin().
  stopMotors();
  waitForStart();
}

void loop() {
  int cm = 0;                    // TODO: mit sensor.readCm() fuellen
  Serial.print(cm);
  Serial.println(F(" cm"));

  if (cm < 0 || cm < STOP_DISTANCE_CM) {
    // TODO: beide Motoren anhalten
  } else {
    // TODO: geradeaus fahren
  }
  delay(200);
}
```

Ergänze die fehlenden Zeilen.

Lösung Notbremse mit Klasse

Notbremse mit Klasse 126_classes_solution.ino

```ino
// Kapitel 12, Loesung: Sensor als Klasse, Notbremse wie in Kapitel 3.
#include "DistanceSensor.h"      // Klasse einbinden

const uint8_t PIN_PWMA = 5;      // rechter Motor: Geschwindigkeit
const uint8_t PIN_PWMB = 6;      // linker Motor: Geschwindigkeit
const uint8_t PIN_AIN1 = 7;      // rechter Motor: Richtung
const uint8_t PIN_BIN1 = 8;      // linker Motor: Richtung
const uint8_t PIN_STBY = 3;      // Motortreiber ein/aus
const uint8_t PIN_BUTTON = 2;    // Taste auf dem Shield
const uint8_t PIN_TRIG = 13;     // Ultraschall Senden
const uint8_t PIN_ECHO = 12;     // Ultraschall Empfangen
const int STOP_DISTANCE_CM = 20; // naeher heran: anhalten
const int DRIVE_SPEED = 120;     // 120 von 255

DistanceSensor sensor(PIN_TRIG, PIN_ECHO);  // Objekt anlegen

void stopMotors() {
  analogWrite(PIN_PWMA, 0);
  analogWrite(PIN_PWMB, 0);
  digitalWrite(PIN_STBY, LOW);
}

void drive(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);
  digitalWrite(PIN_BIN1, leftSpeed >= 0);
  digitalWrite(PIN_AIN1, rightSpeed >= 0);
  analogWrite(PIN_PWMB, abs(leftSpeed));
  analogWrite(PIN_PWMA, abs(rightSpeed));
  digitalWrite(PIN_STBY, HIGH);
}

bool buttonPressed() {
  if (digitalRead(PIN_BUTTON) == LOW) {
    delay(30);                   // Prellen ignorieren
    return digitalRead(PIN_BUTTON) == LOW;
  }
  return false;
}
```

Fortsetzung 126_classes_solution.ino

```
40
41  void waitForStart() {
42    Serial.println(F("Bereit. Druecke die Taste auf dem Shield."));
43    while (!buttonPressed()) {
44      delay(20);
45    }
46    while (buttonPressed()) {
47      delay(20);
48    }
49    for (int i = 3; i > 0; i--) {
50      Serial.print(i);
51      Serial.println(F(" ..."));
52      delay(1000);
53    }
54    Serial.println(F("Los!"));
55  }
56
57  void setup() {
58    Serial.begin(9600);
59    pinMode(PIN_PWMA, OUTPUT);     // Motorpins vorbereiten
60    pinMode(PIN_PWMB, OUTPUT);
61    pinMode(PIN_AIN1, OUTPUT);
62    pinMode(PIN_BIN1, OUTPUT);
63    pinMode(PIN_STBY, OUTPUT);
64    pinMode(PIN_BUTTON, INPUT_PULLUP);
65    sensor.begin();                // Sensorpins setzen
66    stopMotors();
67    waitForStart();
68  }
69
70  void loop() {
71    int cm = sensor.readCm();      // Methode des Objekts
72    Serial.print(cm);
73    Serial.println(F(" cm"));
74
75    if (cm < 0 || cm < STOP_DISTANCE_CM) {
76      stopMotors();
77    } else {
78      drive(DRIVE_SPEED, DRIVE_SPEED);
79    }
80    delay(200);
81  }
```

In der Lösung siehst du den Unterschied zur Datei aus Kapitel 3: Die Messfunktion ist verschwunden. Stattdessen steht oben ein einziges Objekt, und `loop` ruft `sensor.readCm()` auf. Die `.ino` ist kürzer geworden, und die Messung steckt wiederverwendbar in der Klasse.

Probiere

Ändere in `loop` die Schwelle von 20 auf 35 cm. Ein zweiter Sensor ist nur eine **optionale Erweiterung**. Ein zweites HC-SR04 gehört nicht zur Teileliste, und D10 ist schon der Servo. Wenn du einen zweiten Sensor hast: Servo abbauen oder D10 freigeben, zwei freie Pins wählen, zum Beispiel D11 und A3, und `DistanceSensor hinten(11, A3);` anlegen. Rufe `hinten.begin()` in `setup` auf und gib beide Werte im Serial Monitor aus. Ohne zweiten Sensor übst du nur das Anlegen: Lege `DistanceSensor links(PIN_TRIG, PIN_ECHO);` und `DistanceSensor rechts(PIN_TRIG, PIN_ECHO);` an. Rufe für

> `links readCm()` auf und gib für `rechts` einen festen Testwert aus, etwa `int testwert = 40;`. So bleiben zwei getrennte Objekte, ohne dass du einen zweiten Sensor brauchst.

Häufiger Fehler

So sieht es aus	Das hilft
`DistanceSensor.h: No such file or directory`	Der Header liegt nicht im selben Ordner wie die `.ino`. Prüfe den Dateinamen und die Groß- und Kleinschreibung.
`multiple definition of ...`	Die Methode steht doppelt: einmal in der `.ino` und einmal in der `.cpp`. Lass die Definition nur in der `.cpp` stehen und setze `#pragma once` in den Header.
`'sensor' was not declared in this scope`	Das Objekt steht nicht oben bei den Konstanten, oder der Name ist falsch geschrieben. Groß- und Kleinschreibung zählt.
Der Sensor liefert `-1` oder keine Messung	Es fehlt das Echo. Prüfe, ob `begin()` aufgerufen wurde, ob TRIG und ECHO vertauscht sind (D13 und D12) und ob das Kabel sitzt. Das Auto stoppt bei `-1` absichtlich.

> **Merken**
>
> Ein Header enthält nur Deklarationen, die `.cpp` die Definitionen. `#include "..."` sucht im eigenen Ordner, `#include <...>` in den Bibliotheken. `#pragma once` verhindert doppelte Definitionen. Eine Klasse bündelt Daten und Methoden. `public` ist von außen sichtbar, `private` nur innen. Der Konstruktor hat den Namen der Klasse.

- [ ] Ich kann einen Sketch in Header, Quelle und `.ino` aufteilen
- [ ] Ich weiß, was `#pragma once` verhindert
- [ ] Ich kann den Unterschied zwischen `public` und `private` erklären
- [ ] Ich lege ein Objekt an und rufe seine Methoden auf
- [ ] Meine Notbremse benutzt die Klasse `DistanceSensor`

13
Cheatsheet

> **Ziel**
>
> Hier findest du alles Wichtige auf einen Blick: das Grundgeruest, die Sensoren, die Pins, die IR-Codes und die haeufigsten Fehler. Nutze das Kapitel zum Nachschlagen, wenn du einen Sketch baust oder etwas nicht funktioniert.

Grundgeruest und Serial Monitor

Jeder Sketch hat `setup` (laeuft einmal) und `loop` (laeuft endlos). `Serial.begin(9600)` oeffnet den Monitor. `F("...")` legt den Text in den Flash, damit der knappe RAM frei bleibt. `//` beginnt einen Kommentar.

```
void setup() {
  Serial.begin(9600);          // 9600 Baud im Monitor einstellen
  Serial.println(F("Bereit."));// F(): Text liegt im Flash
}
void loop() {
  // laeuft immer wieder
}
```

Datentypen und Operatoren

Typ	Bedeutung	Operator	Bedeutung
`int`	ganze Zahl, z. B. 30	=	zuweisen
`float`	Kommazahl, z. B. 36.5	==	gleich
`bool`	`true` oder `false`	!=	ungleich
`char`	ein Zeichen, z. B. `'A'`	<, >	kleiner, groesser
`String`	Text, z. B. `"Hallo"`	&&, ||	und, oder
		%	Rest einer Division

`31 / 2` ergibt 15, weil beide Zahlen `int` sind. `31 / 2.0` ergibt 15.5. Ein einzelnes `=` weist zu, zwei `==` vergleichen.

Ablauf: Bedingungen, Schleifen, Funktionen

```
if (cm < 20) { stopMotors(); }        // sonst else
else if (cm < 60) { drive(80, 80); }
for (int i = 0; i < 4; i++) { }       // feste Anzahl
while (!buttonPressed()) { delay(20); } // solange wahr
int readDistanceCm() { return 42; }    // Rueckgabetyp vor dem Namen
int werte[5] = {10, 20, 30, 40, 50};   // Array, Index ab 0
```

`for` zaehlt eine feste Zahl von Runden. `while` laeuft, solange die Bedingung wahr ist. Eine Funktion mit Rueckgabe nennt ihren Typ vorne und beendet sich mit `return`.

Immer wieder gebraucht

Diese Bausteine stehen in fast jedem Sketch. Kopiere sie aus dem jeweiligen Kapitel, hier stehen nur die Signaturen und die Kernzeilen.

```
const uint8_t PIN_PWMA=5, PIN_PWMB=6;  // Tempo
const uint8_t PIN_AIN1=7, PIN_BIN1=8;  // Richtung
```

```
const uint8_t PIN_STBY=3, PIN_BUTTON=2;

void stopMotors();                  // beide PWM auf 0, STBY auf LOW
void drive(int left, int right);    // -255 bis 255, Vorzeichen = Richtung
bool buttonPressed();               // INPUT_PULLUP: gedrueckt = LOW
void waitForStart();                // Taste, Loslassen, Countdown 3-2-1
```

`drive` klemmt beide Werte mit `constrain(..., -255, 255)`, setzt die Richtungspins auf $\geq$ `0` und schreibt `abs(...)` auf die PWM-Pins. `waitForStart` blockiert `loop`, bis du die Taste auf dem Shield drueckst.

Sensoren und Aktoren

Bauteil	Aufruf / Wert	Was du wissen musst
Ultraschall	`readDistanceCm()`	58 Mikrosekunden pro cm; `pulseIn` mit Timeout 25000; kein Echo = –1 (ungültig); das Auto stoppt
Linie	`analogRead(A2/A1/A0)`	schwarz 250 bis 850, weiss etwa 1000; eigene Schwelle messen; alle drei weiß = Linienende → Stopp
Servo	`attach(D10)`, `write(winkel)`	0 bis 180 Grad; nach dem Drehen 400 ms warten
MPU6050	`getEvent()`	`RAD_TO_DEG = 57.2958`; Gyro in rad/s, danach deg/s; Adresse `0x68`
LED	`CRGB`, `addLeds<WS2812,4,GRB>`	`show()` senden; jede Farbe 0 bis 255; Pin D4
IR	`begin(PIN_IR, DISABLE_LED_FEEDBACK)`	`decodedIRData.command` lesen, dann `resume()`; `DISABLE_LED_FEEDBACK` haelt D13 frei
Kamera	`{"x":160,"w":42}`	`x` ist die Mitte, `w` die Breite in Pixeln

IR-Codetabelle

Jede Taste sendet ein 8-Bit-`command`. Das Kit kennt zwei Familien. Lies deine Codes mit Beispiel 086 aus; manche Fernbedienungen weichen ab.

Taste	Befehl	Wirkung
hoch	`0x62`	vorwärts
runter	`0xA8`	rückwärts
links	`0x22`	links drehen
rechts	`0xC2`	rechts drehen
OK	`0x02`	anhalten
1	`0x68`	noch frei
2	`0x98`	noch frei
3	`0xB0`	noch frei
4	`0x30`	noch frei
5	`0x18`	noch frei
6	`0x7A`	noch frei
7	`0x10`	noch frei
8	`0x38`	noch frei
9	`0x5A`	noch frei

`0x` gehoert vor jede Zahl. Familie B sendet andere Werte; miss sie mit Beispiel 086 und trage sie an der einen `CMD_...`-Stelle im Sketch ein.

Pin-Tabelle

Pin	Bedeutung	Pin	Bedeutung
D2	Taste auf dem Shield	D9	IR-Empfaenger
D3	STBY, Treiber ein/aus	D10	Servo

Pin	Bedeutung	Pin	Bedeutung
D4	WS2812-LED	D12	Ultraschall ECHO
D5	PWM rechter Motor	D13	Ultraschall TRIG
D6	PWM linker Motor	A0	Linie rechts
D7	Richtung rechter Motor	A1	Linie mitte
D8	Richtung linker Motor	A2	Linie links
		A4/A5	I2C: SDA / SCL

Fehlerbilder

Häufiger Fehler

So sieht es aus	Das hilft
COM-Port fehlt.	USB-Kabel neu stecken, Schalter auf `Upload` stellen, Auto aus- und wieder einschalten.
Upload bricht mit `stk500_recv` ab.	Port und Schalter pruefen: `Upload` waehlen, den richtigen COM-Port nehmen, Auto aus- und wieder einschalten, dann erneut hochladen.
Auto faehrt rueckwaerts.	D7 und D8 sind vertauscht. Tausche `PIN_AIN1` und `PIN_BIN1` in `drive`.
Ultraschall liefert keine Messung / –1.	Das ECHO-Signal fehlt, oft durch ein loses Kabel an D12/D13. Das Auto stoppt absichtlich, statt mit einem Ersatzwert weiterzufahren. Kabel pruefen, Sensor gerade auf eine Wand richten.
Linie zittert.	Schwelle neu messen und `LINE_THRESHOLD` anpassen. Fuer gleichmaessiges Licht sorgen.
IR ohne Reaktion.	Start-Gate gedrueckt? Sonst mit Beispiel 086 den Befehl auslesen und die `CMD_...`-Zahlen an der einen markierten Stelle im Sketch ersetzen. Nicht die `case`-Zeilen einzeln aendern.
Kamera-WLAN fehlt.	Schalter auf `Cam`, Kamerakabel pruefen, Akku laden.
Linkerfehler `__vector_11`.	FastLED 3.10.x vertraegt sich nicht mit Servo. Nimm 3.9.15.
Auto stoppt nach kurzer Zeit ohne neue Daten.	Das ist der Watchdog: 091/096 stoppen nach 1 s ohne Befehl, 106 nach 3 s ohne Kamera, 111 nach 5 s ohne gueltiges Echo, 131 nach 30 s ohne Kreuzung.

Bibliotheksversionen

Diese Versionen passen zusammen und sind getestet.

Bibliothek / Core	Version
FastLED	3.9.15
IRremote	4.7.1
Adafruit MPU6050	2.2.9
Servo	1.2.2
ESP32-Core (nur Kamera)	2.0.17

Achtung

> Nutze bei FastLED die 3.9.15. Die 3.10.x-Reihe kollidiert mit der Servo-Bibliothek und bricht mit `multiple definition of __vector_11` ab.

Glossar

Deutsch	Englisch	Bedeutung
Schleife	`loop`	wiederholt sich endlos
Variable	variable	beschrifteter Platz im Speicher
Funktion	function	benannter Block, der eine Aufgabe loest
Bedingung	condition	Ausdruck, der `true` oder `false` liefert
Sensor	sensor	misst etwas, z. B. Abstand oder Helligkeit
Aktor	actuator	bewegt etwas, z. B. Motor oder Servo
Sketch	sketch	das Arduino-Programm
Bibliothek	library	fertiger Code, den du einbindest

Beispielnummern

Alle Sketches liegen unter `code/`. Die Nummer steht im Ordnernamen und im ersten Kommentar der `.ino`-Datei.

Nr.	Kap.	Nr.	Kap.
001	1	071	6
006	1	076	6
011	1	081	7
016	1	086	7
021	1	091	7
026	2	096	8
031	2	101	10
036	2	106	10
041	3	111	11
046	3	116	11
051	4	121	11
056	4	131	9
061	5	126	12
066	5	106 Mission	11

Merken

`setup` laeuft einmal, `loop` endlos. Ein `=` weist zu, `==` vergleicht. Vor jeder Fahrt steht das Start-Gate. Sensoren liefern Rohwerte, die du selbst kalibrierst. Pruefe bei Problemen zuerst Kabel, Schalter und Bibliotheksversion.

A
Anhang

Fuer Eltern und Lehrkraefte

Das Buch ist fuer Kinder ab 12 geschrieben, die es allein lesen koennen. Die Mission am Kapitelanfang sagt, was das Kind am Ende gebaut hat.

Kapitel	Das kannst du danach	Zeit
1	Erstes Programm, Variablen, Schleifen und Funktionen	2 Termine
2	Beide Motoren steuern, nach vorn und zurueck fahren	2 Termine
3	Notbremse vor einem Hindernis, stoppt bei ungueltiger Messung (-1)	2 Termine
4	Auto folgt der Linie mit eigener Schwelle, stoppt am Linienende (alle weiss)	2 Termine
5	Servo dreht den Kamera-Kopf nach Links und Rechts	1 Termin
6	MPU6050 erkennt Neigung und Stoesse, loest Neigungs- und Stossalarm aus	2 Termine
7	WS2812 leuchtet, IR-Taste macht das Auto fuehrbar	2 Termine
8	Kamera-WLAN sendet ein Bild, JSON-Steuerbefehle	2 Termine
9	Alle Bauteile zusammen: Linien- und Hindernis-Verhalten	2 Termine
10	Eigene ESP32-Firmware, ueberschreibt die Werksfirmware; Rueckweg erst am Geraet testen	1 Termin
11	Eigene Projekte planen und umsetzen	2 Termine
12	Code in Dateien ordnen, eine Klasse lesen und nutzen	2 Termine
13	Alles nachschlagen: Pins, Befehle, Fehler	1 Termin

So laeuft ein realistischer Zeitplan: etwa 12 Wochen mit zwei Terminen von je 45 Minuten pro Woche. Ein Termin hat zwei Aufgaben. Die Lueckenaufgabe laeuft zuerst, die Loesung kommt erst am Ende. Am letzten Tag steht das Auto, wenn alles gut gelaufen ist.

Tipp

Hilfe ohne Loesung: Das Kind liest die Zeilen selbst, du fragst nur. „Was macht diese Zeile?“ oder „Wo fehlt der Abschluss?“ reicht fast immer. Die Loesung auf der naechsten Seite vergleicht das Kind erst nach dem eigenen Versuch.

Achtung

Upload/Cam-Schalter: Bevor du einen Sketch auf das Auto loedst, stellst du den Schalter auf „Upload“. Beim Flashen steht das Auto still, das ist normal. Nach dem Flashen stellst du ihn zurueck auf „Cam“, dann ist das WLAN der Kamera wieder erreichbar.

Merken

Am Anfang: Auto ruhig auf den Boden stellen und erst dann die Spannung einschalten. Der GY-521 kalibriert sich nicht von selbst: Vor der Neigungsmessung brauchst du eine ebene Referenz. Das Servo nicht mit der Hand drehen, wenn das Auto Strom hat.

Wenn nichts geht

Die Tabelle zeigt, was du zuerst schaust. Jede Zeile ist ein reales Fehlerbild aus dem Kit.

Symptom	Ursache	Loesung
COM-Port taucht nicht auf	USB-Kabel ist nicht angeschlossen oder der Schalter steht auf „Cam“	Kabel neu an die Buchse stecken, Schalter auf „Upload“ stellen, Auto kurz aus- und wieder ein.
Upload bricht mit „stk500_recv“ ab	Falscher Port gewaehlt, der Schalter steht nicht auf „Upload“, oder der Reset beim Upload schlaegt fehl.	Port in der IDE pruefen, Schalter auf „Upload“ stellen, Auto aus- und wieder einschalten, Upload erneut starten.
Auto faehrt zurueck	Die Richtungs-Bits D8 und D7 (BIN1, AIN1) sind vertauscht.	Die beiden Pin-Kabel an einem Motor umdrehen, dann testen.
Ultraschall zeigt –1 oder keine Messung	Das ECHO-Signal kommt nicht an, meist ein loses Kabel an D12/D13.	Kabel D12/D13 neu stecken, Sensor gerade ausrichten; das Auto stoppt bei –1 absichtlich.
Linienfolger zittert	Die Schwelle passt nicht zu deiner Bahn, das Licht ist anders.	`LINE_THRESHOLD` im Sketch anpassen (nicht die App-Schwelle N=104), die Bahn gleichmaessig beleuchten.
IR reagiert nicht oder stoppt nach 1 s	Die Taste deiner Fernbedienung nutzt die andere Familie; der Sketch prueft die Familie-A-`CMD_...`-Werte. Der Watchdog stoppt ohne neuen Befehl nach 1 s.	Familie A steht in der Cheatsheet-Tabelle; andere Werte mit Beispiel 086 messen und die `CMD_...`-Zahlen ersetzen.
Kamera-WLAN taucht nicht auf	Schalter steht auf „Upload“, das Kabel zur Kamera ist lose, oder der Akku ist zu schwach.	Schalter auf „Cam“ stellen, Kamera-Kabel neu stecken, Auto kurz aus- und wieder ein.

Teileliste

Die Teileliste zeigt, was im Kit liegt und wofuer es ist.

Teil	Wofuer es ist
Arduino Uno	Der Rechner, den du programmierst. Laeuft alle Programme.
Smart-Robot-Shield (TB6612) mit GY-521	Der TB6612 treibt die beiden Motoren. Der GY-521 misst die Drehung.
HC-SR04 Ultraschallsensor	Misst die Distanz in Zentimetern, bis zu 200 cm.
3x Linienfolger (ITR20001)	Erkennen, ob der Sensor ueber der schwarzen Linie ist.
SG90 Servo	Dreht den Kamera-Kopf nach Links und Rechts.
WS2812 LED	Eine RGB-LED, die du in jeder Farbe leuchten lassen kannst.
IR-Empfaenger mit Fernbedienung	Empfaengt die Infrarot-Signale der Fernbedienung.
ESP32-WROVER mit Kamera	Sendet ueber WLAN ein Bild an das Handy.
2x Getriebemotor	Treiben die beiden Räder.

Teil	Wofuer es ist
Lithium-Akku (in der Akku-Box) mit Ladekabel	Laedt ueber USB, reicht fuer den Betrieb.
Jumper-Kabel, Stecker, Schrauben	Verbinden die Module mit dem Shield.

Beispielverzeichnis

Alle Programme aus `code/` mit Kapitel und Art. Demo heisst: Skelett und Loesung sind gleich. Aufgabe heisst: das Skelett hat Luecken.

Nr.	Titel, Kapitel, Art
001	Hallo Arduino · Kap. 1 · Demo
006	Vier Typen · Kap. 1 · Demo
011	Zu nah, passt, weit weg · Kap. 1 · Demo
016	Zaehlen mit for und while · Kap. 1 · Demo
021	Mittelwert · Kap. 1 · Aufgabe
026	Vor, zurueck und drehen · Kap. 2 · Demo
031	Rampe von 0 bis 255 · Kap. 2 · Demo
036	Quadrat · Kap. 2 · Aufgabe
041	Abstand anzeigen · Kap. 3 · Demo
046	Notbremse vor einem Hindernis · Kap. 3 · Aufgabe
051	Rohwerte anzeigen · Kap. 4 · Demo
056	Der Linie folgen · Kap. 4 · Aufgabe
061	Kopf drehen · Kap. 5 · Demo
066	Der freie Weg · Kap. 5 · Aufgabe
071	Roll und Pitch · Kap. 6 · Demo
076	Mission Stossalarm · Kap. 6 · Aufgabe, hochladbar
081	Vier Farben und einmal aus · Kap. 7 · Demo
086	Codes auslesen · Kap. 7 · Demo
091	Mission Fernbedienung · Kap. 7 · Aufgabe, hochladbar
096	Fahren mit Buchstaben · Kap. 8 · Aufgabe, hochladbar
101	Farbe suchen: erst rot, dann gruen (ESP32) · Kap. 10 · Aufgabe
106	Dem farbigen Punkt folgen (Uno) · Kap. 10 · Aufgabe
106 Mission	Gruenem Punkt folgen (Mission) · Kap. 11 · Mission
111	Parkwaechter · Kap. 11 · Aufgabe, hochladbar
116	Tanzroboter · Kap. 11 · Aufgabe, hochladbar
121	Geheimcode · Kap. 11 · Loesung
126	Notbremse mit Klasse · Kap. 12 · Aufgabe
131	Rettungsrover · Kap. 9 · Aufgabe

> **Tipp**
>
> Wer die Werksfirmware des ESP32 zurueckholen muss, folgt der Checkliste in Kapitel 10: Schalter auf `Cam`, richtigen Port waehlen, Arduino IDE mit ESP32-Core 2.0.17, Binary-Pfad aus dem Kit, dann der Pruefschritt im Seriellen Monitor. Dieser Rueckweg ist noch nicht am Geraet getestet.

Änderungen in Version 1.1

- Sensoren und Verbindungen stoppen jetzt nach einer festen Zeit ohne gueltige Daten: 091/096 nach 1 s, 106 nach 3 s, 111 nach 5 s, 131 nach 30 s.
- IR-Codes nutzen IRremote 4.x mit `decodedIRData.command` und 8-Bit- `CMD_...`-Werten; Familie B wird mit Beispiel 086 gemessen.
- Die hochladbaren Uebungssketches sind vollstaendig und werden mitgeprueft; einzelne Ausschnitte bleiben bewusst aussen vor.
- Die Kamera-Mission gibt ueber die LED Rueckmeldung: gruen folgt, blau sucht, rot stoppt.

- `0x` und die Hex-Schreibweise werden ausfuehrlich erklaert.
- Das Layout ist fuer den beidseitigen Druck (Duplex) gesetzt.
- Die Sicherheitstexte zu Akku, Servo und Upload-Schalter sind klarer.

Quellen und Weiterlesen

1. Elegoo Smart Robot Car Kit V4.0: User Manual und Tutorial-PDFs im Ordner „02 Manual & Main Code & APP“ des Kits.
2. Elegoo Smart Robot Car Kit V4.0: Firmware und Bibliotheken im Ordner „02 Manual & Main Code & APP/02 Main Program (Arduino UNO)“.
3. Elegoo Smart Robot Car Kit V4.0: Kommunikationsprotokoll („04 Related chip information/Communication protocol for Smart Robot Car.pdf“).
4. Arduino-Referenz: https://www.arduino.cc/reference/en/libraries/
5. Adafruit MPU6050 Guide: https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro/arduino
6. Adafruit_MPU6050 Bibliothek: https://github.com/adafruit/Adafruit_MPU6050
7. IRremote Bibliothek (4.x): https://github.com/Arduino-IRremote/Arduino-IRremote
8. FastLED Bibliothek: https://github.com/FastLED/FastLED
9. NewPing Bibliothek: https://github.com/teckel1/ARDUINO-NewPing
10. Elegoo offizielles GitHub-Projekt: https://github.com/elegooofficial/ELEGOO-Smart-Robot-Car-Kit-V4.0
11. ArjanOnwezen: Firmware mit strukturierten Ordnern: https://github.com/ArjanOnwezen/ElegooSmartRobotCarV4.0
12. antonioastro/Elegoo-SmartCar mit IR-Werten fuer ein TV-Gerat: https://github.com/antonioastro/Elegoo-SmartCar
13. raccrobotics/libelegoo-edu, eine Lern-Bibliothek: https://github.com/raccrobotics/libelegoo-edu
14. Toremetal/ESP32-Kamera-Beispiele, angepasst an Arduino IDE 2.1.0: https://github.com/Toremetal/ESP32_Examples-ESP32-Camera-CameraWebServer

Am Ende des Anhangs steht das Auto, der Rest ist deine Sache.